Warsaw University of Technology

FACULTY OF
ELECTRONICS AND INFORMATION TECHNOLOGY

Institute of Radioelectronics and Multimedia Technology

Final project

Postgraduate diploma thesis
Deep Learning – Applications in Digital Media

Combining Structured Knowledge Graphs with Patient-Level Statistical Evidence for Link Prediction and Clinical Prediction

Marta Jasiewicz Badowska
Monika Oskard

student record book number —

thesis supervisor
mgr inż. Grzegorz Gwardys

WARSAW 2026

Abstract

This thesis investigates how structured knowledge represented by a pharmacotherapy knowledge graph can be combined with explicit statistical evidence derived from patient-level data. The problem is studied through two complementary tasks operating on a shared synthetic pharmacotherapy benchmark. Task A addresses directed link prediction, using patient-level observations to evaluate candidate relations in the graph. Task B addresses patient-level clinical endpoint prediction, using the graph as a shared structural scaffold while individual patient observations instantiate the node-level input.

Both tasks combine graph neural representations with explicit statistical dependence information. In Task A, a heterogeneous graph encoder is fused with candidate-specific statistical context derived from patient observations. In Task B, heterogeneous message passing is augmented with endpoint-oriented statistical evidence, while different graph architectures, propagation depths, and residual connections are evaluated. HCR-derived coefficients are used as representations of empirical dependence.

On the synthetic benchmark, the final Task A model reaches an official five-scenario macro validation Average Precision of approximately 0.915 (clean-scenario validation AP approximately 0.919), with ablation results showing that explicit statistical context provides a substantial improvement over the graph representation alone. In Task B, the statistical branch is most beneficial when deeper message passing loses predictive information, whereas residual connections preserve the same information within the graph pathway; consequently, the two mechanisms are not always additive on the synthetic graph.

The generality of these observations is further examined through two cross-domain proxy experiments. On leakage-corrected Last-FM*, progressive addition of explicit candidate-specific context increases sampled NDCG@20 from approximately 0.475 for HGT alone to approximately 0.873 for the frozen model, which achieves approximately 0.276 NDCG@20 under full-catalog validation. On a Hetionet-derived endpoint-prediction proxy, the statistical branch again becomes particularly valuable for deeper models, increasing test AUROC at six layers from 0.585 for the graph-only model to 0.757, while combining statistical evidence with residual propagation reaches 0.760.

The results support a common methodological conclusion: structured graph representations and explicit empirical dependence provide complementary sources of information for both graph reconstruction and patient-level prediction. Their relative contribution depends on the task and on how effectively the graph neural network preserves relevant information through message passing. The proxy experiments provide methodological stress tests, but their findings should be interpreted cautiously before any application to clinical settings.

Keywords: knowledge graphs, graph neural networks, heterogeneous graphs, link prediction, clinical endpoint prediction, Hierarchical Correlation Reconstruction, pharmacotherapy.

Streszczenie

Przedmiotem pracy dyplomowej jest przetestowanie możliwości łączenia mechanistycznej wiedzy zakodowanej w formie grafu wiedzy (ang. knowledge graph) z danymi empirycznymi pochodzącymi z populacji pacjentów. Zagadnienie to jest rozpatrywane na dwóch uzupełniających się, w docelowym środowisku klinicznym, płaszczyznach. Zadanie A koncentruje się na predykcji krawędzi (ang. link prediction) w grafie mechanistycznym, z wykorzystaniem topologii grafu wiedzy oraz statystycznej wiedzy na temat zjawiska uzyskanej z danych pacjenckich. Natomiast Zadanie B ma wykorzystywać mechanistyczny graf wiedzy do predykcji wystąpienia klinicznych punktów końcowych, rozumianych jako zdarzenia wymagające interwencji klinicznej, wykorzystując graf jako wspólny szkielet strukturalny, podczas gdy obserwacje dotyczące poszczególnych pacjentów stanowią dane wejściowe na poziomie węzłów.

Oba zadania wykorzystują grafowe sieci neuronowe (ang. Graph Neural Networks) oraz dane empiryczne przekazane do modelu w formie statystyk, ze szczególnym uwzględnieniem oceny siły zależności między zmiennymi. Implementacje w obu zadaniach różnią się docelowymi architekturami, głębokością message passingu, połączeniami rezydualnymi, a także sposobem wykorzystania gałęzi statystycznej w architekturze sieci. W części statystycznej wspólnym mianownikiem jest jednak wykorzystanie metody Hierarchical Correlation Reconstruction (HCR), która umożliwia opis zależności pomiędzy zmiennymi różnych typów za pomocą ortonormalnych funkcji bazowych i odpowiadających im współczynników, pozwalając uwzględniać zarówno proste, jak i bardziej złożone struktury zależności.

W przypadku syntetycznego benchmarku finalny model dla Zadania A osiąga wartość Average Precision na zbiorze walidacyjnym na poziomie około 0,919, przy czym wyniki analizy ablacyjnej wskazują, że jawne uwzględnienie kontekstu statystycznego zapewnia znaczną poprawę w porównaniu z samą reprezentacją topologii grafu. W Zadaniu B moduł statystyczny przynosi największe korzyści w sytuacjach, gdy głębsze warstwy message passingu prowadzą do utraty informacji predykcyjnych, podczas gdy połączenia rezydualne pozwalają zachować te informacje w obrębie ścieżki grafowej. W rezultacie na grafie syntetycznym działanie obu mechanizmów nie zawsze się sumuje.

Ogólny charakter tych obserwacji zweryfikowano dodatkowo za pomocą dwóch międzydomenowych eksperymentów. W przypadku zbioru Last-FM* (z korektą wycieku danych) stopniowe wprowadzanie jawnego kontekstu specyficznego dla kandydata podnosi wartość wskaźnika NDCG@20 w ramach ewaluacji próbkowanej z około 0,475 dla modelu HGT do około 0,873 dla modelu pełnego, który w walidacji na pełnym katalogu osiąga wynik NDCG@20 na poziomie około 0,276. W eksperymencie porównawczym dotyczącym przewidywania klinicznych punktów końcowych, opartym na zbiorze Hetionet, moduł statystyczny ponownie okazuje się szczególnie cenny w przypadku głębszych modeli: dla konfiguracji sześciowarstwowej podnosi on wartość AUC na zbiorze testowym z 0,585 dla modelu opartego wyłącznie na grafie do 0,757, natomiast połączenie informacji statystycznej z propagacją rezydualną pozwala osiągnąć wynik 0,760.

Wyniki potwierdzają wspólny wniosek metodologiczny: ustrukturyzowane reprezentacje grafów i jawna informacja o zależnościach empirycznych stanowią atrakcyjny, uzupełniający się zestaw źródeł informacji zarówno do rekonstrukcji grafów, jak i predykcji na poziomie pacjenta. Ich względny wkład zależy od zadania oraz od tego, jak skutecznie grafowa sieć neuronowa zachowuje istotne informacje podczas kolejnych etapów propagacji i agregacji informacji w grafie. Eksperymenty na podobnych zbiorach danych stanowią metodologiczne testy obciążeniowe, jednak ich wyniki należy interpretować ostrożnie przed jakimkolwiek zastosowaniem w realnych warunkach klinicznych.

Słowa kluczowe: grafy wiedzy, grafowe sieci neuronowe, grafy heterogeniczne, predykcja linków, predykcja klinicznych punktów końcowych, Hierarchical Correlation Reconstruction, farmakoterapia.

Contents

Streszczenie **1**

List of Symbols and Abbreviations **6**

1 Introduction **8**

2 Objectives and Contributions **10**

3 Related Work **11**
3.1 Knowledge Graphs in Biomedicine . . . 11
3.1.1 Graph Neural Networks . . . 12
3.2 Statistical Dependence and Hierarchical Correlation Reconstruction . . . 15

4 Dataset **17**
4.1 Synthetic Pharmacotherapy Dataset . . . 17
4.2 Pharmacotherapy Knowledge Graph . . . 17
4.3 Patient-Level Data Generation . . . 20
4.4 Observational Scenarios . . . 21
4.5 Patient Splits and Use Across Tasks . . . 21

5 Task A. Directed Link Prediction on the Synthetic Pharmacotherapy Graph **23**
5.1 Problem Formulation . . . 23
5.2 Task-Specific Data Preparation . . . 24
5.2.1 Candidate edges and splits . . . 24
5.2.2 Message-passing graph . . . 24
5.2.3 Node features . . . 24
5.3 Model Architecture . . . 25
5.3.1 Architecture Overview . . . 25
5.3.2 Structural Branch . . . 25
5.3.3 Statistical Evidence Branch . . . 26
5.3.4 Fusion and Prediction . . . 26
5.4 Training and Evaluation . . . 27
5.4.1 Training Objective . . . 27
5.4.2 Evaluation Metrics . . . 28
5.4.3 Model and Training Configuration . . . 28
5.5 Experimental Results . . . 28
5.5.1 Architecture Comparison . . . 28
5.5.2 Contribution of Statistical Evidence . . . 29

5.5.3 Final Evaluation 29

6 Cross-Domain Validation of Task A. Last-FM* 30
6.1 Problem Formulation 30
6.2 Dataset and Collaborative Knowledge Graph 30
6.2.1 Leakage-corrected Last-FM* 30
6.2.2 Collaborative knowledge graph 31
6.2.3 Training candidate pairs 31
6.3 Model Architecture 32
6.3.1 Architecture Overview 32
6.3.2 Structural Branch 32
6.3.3 Statistical Evidence Branch / Candidate-Specific Context 32
6.3.4 Fusion and Prediction 35
6.4 Training and Evaluation 36
6.4.1 Training Objective 36
6.4.2 Checkpoint selection with sampled NDCG@20 37
6.4.3 Development full-catalog evaluation 37
6.4.4 Sealed external benchmark 37
6.4.5 Computational note 37
6.5 Experimental Results 38
6.5.1 Architecture Comparison 38
6.5.2 Contribution of Statistical Evidence 38
6.5.3 Full-catalog development ranking and classical baseline 40
6.5.4 Sealed external benchmark 40

7 Task B. Patient-Level Endpoint Prediction 43
7.1 Problem Formulation 43
7.2 Patient-Level Graph Representation 44
7.2.1 Prediction targets 45
7.3 Model Architecture 46
7.4 Statistical Evidence Branch 47
7.4.1 Scope of variables and parents 49
7.4.2 Pairwise HCR core coefficients 49
7.4.3 Extended pairwise evidence channels 49
7.4.4 Type-specific encoders and endpoint-specific fusion 49
7.4.5 Non-binary parents 50
7.4.6 Higher-order triple-path evidence 50
7.5 Training and Evaluation 51
7.5.1 Training Objective 51
7.5.2 Evaluation Metrics 51
7.5.3 Model and Training Configuration 51
7.6 Experimental Results 53
7.6.1 Architecture Comparison 53
7.6.2 Effect of Residual Connections 54
7.6.3 Contribution of Statistical Evidence 54
7.6.4 Final Evaluation 57

8 Cross-Domain Validation of Task B. Hetionet Proxy **59**
8.1 Problem Formulation and Proxy Construction 59
8.2 Data Representation 60
8.3 Training and Evaluation 62
8.4 Experimental Results 63

9 Discussion and Limitations **65**
9.1 Common Methodological Principle 65
9.2 What the Two Synthetic Tasks Show 65
9.3 Cross-Domain Proxy Experiments 66
9.4 Prediction versus Mechanistic Clinical Reasoning 67
9.5 Observational Data, Explainability and Causal Interpretation 67
9.6 Average Precision and Asymmetric Error Costs in Graph Reconstruction . 68
9.7 Limitations and Deployment Implications 69

10 Conclusions and Future Work **70**
10.1 Conclusions 70
10.2 Future Work 70

A HCR-Inspired Pair Representation: Detailed Definition **73**
A.1 Overall 40-dimensional representation 73
A.2 HCR-inspired coefficient matrix 74
A.3 Binary basis and connection to Pearson ϕ 75
A.4 Continuous variables 75
A.5 Count variables 76
A.6 Four derived dependence summaries (slots 16–19) 76
A.7 Type-specific marginal descriptors (slots 20–27) 76
A.8 Joint activation and rarity (slots 28–31) 78
A.9 Support and estimability (slots 32–33) 78
A.10 Variable-type metadata (slots 34–39) 79
A.11 Final composition 79
A.12 Connection to the Task A motif 79
A.13 Source and inspiration note 80

B Synthetic Endpoint Oracle Reference **81**

List of Figures **82**

List of Tables **84**

List of Symbols and Abbreviations

Abbreviations used in the thesis are collected in Table 1. Mathematical symbols are defined at first use in the text.

Table 1: Symbols and abbreviations by category.

Category	Abbreviation	Meaning
Clinical and pharmacological	ADR	Adverse Drug Reaction
	AKI	Acute Kidney Injury
	DILI	Drug-Induced Liver Injury
	CKD	Chronic Kidney Disease
	CNS	Central Nervous System
	GI	Gastrointestinal
	DDI	Drug–Drug Interaction
	ALT	Alanine Aminotransferase
	AST	Aspartate Aminotransferase
	NSAID	Non-Steroidal Anti-Inflammatory Drug
	ACEi	Angiotensin-Converting Enzyme Inhibitor
	ARB	Angiotensin Receptor Blocker
	SSRI	Selective Serotonin Reuptake Inhibitor
	SNRI	Serotonin–Norepinephrine Reuptake Inhibitor
	EHR	Electronic Health Record
Graph, machine learning and computation	KG	Knowledge Graph
	CKG	Collaborative Knowledge Graph
	DAG	Directed Acyclic Graph
	GNN	Graph Neural Network
	GCN	Graph Convolutional Network
	R-GCN	Relational Graph Convolutional Network
	GAT	Graph Attention Network
	HGT	Heterogeneous Graph Transformer
	GraphSAGE	Graph Sample-and-Aggregate architecture
	HCR	Hierarchical Correlation Reconstruction
	MLP	Multilayer Perceptron
	PyG	PyTorch Geometric
	KGAT	Knowledge Graph Attention Network
	LightGCN	Light Graph Convolutional Network
	1-WL	One-Dimensional Weisfeiler–Lehman Test

Continued on next page

Category	Abbreviation	Meaning
	LP	Link Prediction
Statistics and evaluation	AP	Average Precision
	AUROC	Area Under the Receiver Operating Characteristic Curve
	ROC	Receiver Operating Characteristic
	PR	Precision–Recall
	TPR	True Positive Rate
	FPR	False Positive Rate
	BCE	Binary Cross-Entropy
	MI	Mutual Information
	NMI	Normalized Mutual Information
	PMI	Pointwise Mutual Information
	NPMI	Normalized Pointwise Mutual Information
	CDF	Cumulative Distribution Function
	ECDF	Empirical Cumulative Distribution Function
	NDCG	Normalized Discounted Cumulative Gain
	MRR	Mean Reciprocal Rank
	MAP	Mean Average Precision
	SD	Standard Deviation

Chapter 1

Introduction

Modern pharmacotherapy requires the integration of well-established pharmacological knowledge across multiple biological and clinical levels. Medical knowledge graphs provide a structured representation of this knowledge, capturing the complex mechanisms through which drugs interact with the human organism, with one another, and with intermediate biological processes. These interactions may propagate through multiple mechanistic pathways and ultimately converge on a observable clinical endpoint, particularly in the context of suboptimal or complex pharmacotherapy.

The topology of a knowledge graph describes which pathways and relationships are considered plausible or mechanistically meaningful based on literature, but it does not by itself determine how strongly these relationships are supported by observations in a particular population, nor how the represented mechanisms are expressed in an individual patient. On the other hand, patient-level data provide empirical information on drug exposure, laboratory outcomes, and diagnoses, but these observations are often incomplete, noisy, and affected by selection artifacts. As a result, the observed data do not directly reveal the underlying mechanistic structure linking drug exposures to intermediate processes and, ultimately, to clinical endpoints. This creates a natural methodological distinction between two complementary sources of information – graph-structured knowledge and patient-level empirical evidence.

The central aim of this thesis is to investigate how these two sources can be integrated within a unified framework so that they interact synergistically. Based on this aim, we formulate the following theses:

Thesis 1. Empirical patient-level data, when integrated with the structural information encoded in a knowledge graph, can improve the prediction of missing directed links.

Thesis 2. The directed causal graph can serve as a shared scaffold for predicting multiple clinical events in individual patients.

As we combine graph-based structure and empirical data, this leads us to use graph neural networks (GNNs) for prediction. As the signal in GNNs can be averaged out, we explore methods for incorporating additional signal into GNNs through associative statistics. Inspired by the Wide & Deep framework proposed by Cheng et al. [5], we formulate the third thesis:

Thesis 3. Explicit empirical associations can provide additional signal to GNNs and improve prediction performance of GNNs.

The importance of these theses is reflected in the current research on patient-specific modelling and digital twins (digital representations of patients). The direction proposed is discussed in work on causal graph neural networks in healthcare, which connects mechanistic graph structure with patient-specific data. That literature also notes that models relying predominantly on statistical associations may be sensitive to changes between populations, institutions, and data-generating environments, and that causal or mechanistic structure is hypothesized to yield representations that are more stable under such changes—while emphasizing remaining challenges related to distribution shifts, validation, computational complexity, and the distinction between causal claims and statistical associations [27].

Based on that and taking into consideration formulated theses, there are defined two complementary prediction tasks.

Task A. Patient-level observations are used together with the available graph structure to evaluate candidate directed relations and perform link prediction.

Task B. Patient-level clinical outcome prediction based on the graph structure. The graph structure is treated as a shared mechanistic scaffold, while patient-specific observations are used to predict clinical endpoints for individual patients.

Together, the two tasks can be viewed as complementary parts of a broader iterative framework. In real-world applications, patient-level evidence can be used systematically to evaluate and extend the structured knowledge representation, while the resulting graph can in turn provide context for patient-level prediction. In this way, the knowledge graph is not treated only as a static repository of prior knowledge, but as a structured representation that can, in principle, be confronted with accumulating empirical evidence at scale.

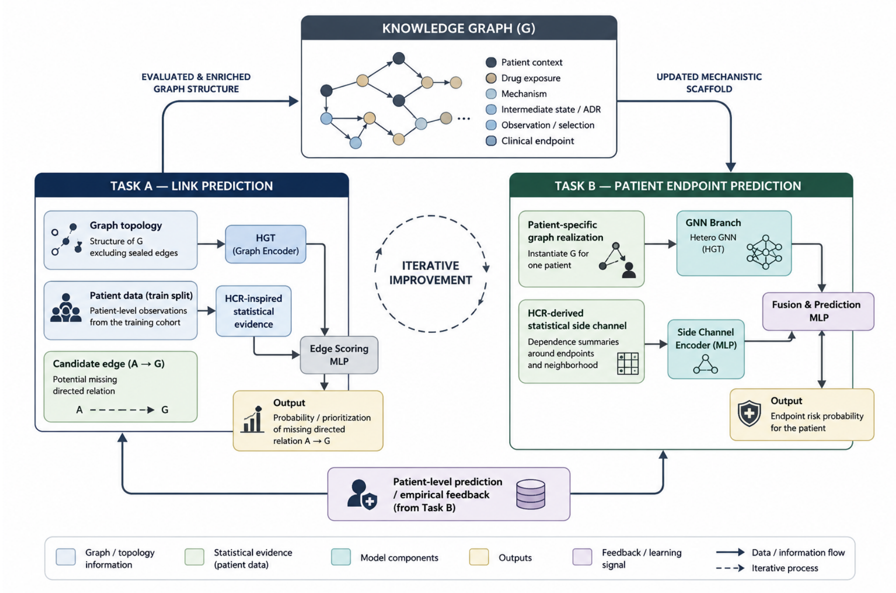

Figure 1.1: Intended research direction: an iterative loop between Task A (evaluation of graph relations) and Task B (patient-level endpoint prediction).

Chapter 2

Objectives and Contributions

The main objective of this thesis is to investigate how structured knowledge represented by a medical knowledge graph can be combined with statistical evidence derived from patient-level data. The problem is studied through two complementary tasks: graph reconstruction and patient-level clinical endpoint prediction.

The specific contributions of the thesis are:

Task A. Directed link prediction. A method is developed for evaluating candidate directed relations in a heterogeneous pharmacotherapy graph by combining graph structure with explicit empirical evidence from patient-level data. The graph component, implemented with a Heterogeneous Graph Transformer (HGT), provides a topology-constrained representation of the candidate relation and its local typed context. The statistical component characterizes the geometry and strength of dependence observed in patient-level data using coefficients derived from Hierarchical Correlation Reconstruction (HCR), together with additional statistical descriptors. Both representations are then combined and passed to a common decoder for directed link prediction. The same principle is additionally examined using Last-FM, a corrected music recommendation benchmark represented as a user–artist interaction graph, which serves as a proxy task under a substantially different graph structure and ranking objective. This allows us to assess whether the value of combining structural and explicit empirical information is specific to the pharmacotherapy setting or persists after a substantial change of domain.

Task B. Patient-level endpoint prediction. A patient-specific prediction framework is developed in which the shared pharmacotherapy graph provides the structural representation and patient observations provide individual input information. Several GNN architectures and graph depths are evaluated, including the effect of residual connections on deeper message passing. The graph representation is further complemented with an HCR-derived statistical side-channel describing dependencies between endpoints and their direct or higher-order graph neighbourhood. Different forms of this statistical branch, including pairwise, type-specific and higher-order representations, are compared. The resulting models are evaluated separately for clinical endpoints using metrics appropriate for imbalanced outcomes, including AUROC, Average Precision (AP) and lift.

In both cases, graph-based representation learning and explicit statistical dependence are treated as complementary sources of information rather than as competing alternatives.

Chapter 3

Related Work

This chapter reviews the background that motivates combining structured knowledge graphs with explicit statistical dependence estimates – starting from biomedical knowledge graphs, graph neural networks for heterogeneous data structure and Hierarchical Correlation Reconstruction served as inspiration to statistical dependence branch in this work.

3.1 Knowledge Graphs in Biomedicine

The term *knowledge graph* (KG), though traceable to proposals from the 1970s, entered widespread use after Google's 2012 announcement of its knowledge graph, built on resources such as DBpedia and Freebase [19]. Since then, the concept has been adopted far beyond web search, as it provides an interlinked representation of knowledge in a human- and machine-understandable format [2].

This growth has been particularly visible in biomedicine, where KGs are used to integrate heterogeneous sources such as drugs, genes, diseases, and mechanisms of action (the biochemical process by which a drug produces its effect) [26]. In biomedical KGs, entities (nodes V) may represent drugs, diseases, genes, proteins, mechanisms, patient characteristics, or clinical outcomes, while edges (E) describe typed relationships (e.g., *drug–treats–disease*, *gene–associates–disease*). Such graphs are naturally heterogeneous and admit the form [21]

$$G = \Big(V, E, \tau, \phi\Big), \qquad \tau\colon V \longrightarrow \mathcal{T}_V \text{ and } \phi\colon E \longrightarrow \mathcal{T}_E \tag{3.1}$$

Within this domain, knowledge graphs support a wide range of tasks, spanning drug-drug interaction prediction, drug development, and clinical decision support [7]. Among these, clinical decision support centres on patient-level data as a core, rather than drug- or trial-related data. Graph-based approaches within this patient-centric setting have been applied to different tasks such as clinical prediction tasks including in-hospital mortality and readmission [22], survival analysis [24], and multimodal risk prediction from electronic health records (EHR) [23].

At this point it is important to note that such clinical prediction models could pursue different aims, such as risk prediction or treatment effect estimation. Risk models predict the probability of an outcome under the patient's observed factual course of care, whereas treatment effect estimation asks a counterfactual question: how would the outcome differ under an alternative treatment decision [20]. This distinction motivates the design of our knowledge graph, which is structured to reflect intended causal relations between entities and patient trajectories in relation to specific treatments (drugs) and patient characteristics. The decision was motivated by Toonsi et al. [34] who propose causal knowledge graphs as a means of addressing several shortcomings of the standard modelling pipeline simultaneously. As KG's edges are intended to encode causal, rather than purely associative, relations, causal knowledge graphs are more robust to distribution shift between clinics and better suited to reasoning under incomplete documentation, while also allowing prior medical knowledge to support downstream prediction tasks [7].

Such graphs could be dynamically constructed from empirical data. Standard pipelines for constructing KG include a dedicated step for completing missing relations between entities using graph databases or link prediction models, typically applied to graphs derived from literature and curated sources [7].

Taken together, these observations motivate two complementary tasks studied in this thesis: reconstructing knowledge graphs via empirical patient-level data, and predicting clinical outcomes on the resulting structure via node classification. The tasks are addressed by GNNs described in the following section.

3.1.1 Graph Neural Networks

Graph neural networks (GNNs) learn node representations by aggregating information from neighbouring nodes through message passing [16]. After K message-passing layers, the representation of a node can incorporate information from its K-hop neighbourhood. During training, every node is represented not only by its own features, as in traditional neural networks, but also by the features of its neighbours. Unlike earlier graph-embedding methods with fixed representations, GNNs build node embeddings inductively through aggregation, so inference does not require all nodes to be present during training.

Formally, the representation $h_v^{(K)}$ of node v depends on the features of all nodes within its K-hop neighbourhood. At layer k,

$$h_v^{(k)} = \text{UPDATE}\Big(h_v^{(k-1)},\ \text{AGGREGATE}\big(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\}\big)\Big), \tag{3.2}$$

$\mathcal{N}(v)$ — neighbourhood of node v

so that information from a node u at graph distance d from v only reaches v once $K \geq d$. Different GNN architectures mainly differ in how neighbouring information and, on heterogeneous graphs, relation types are transformed and aggregated. Several architectures relevant to this thesis are reviewed below.

GraphSAGE

GraphSAGE [16] was introduced for inductive and scalable representation learning on large graphs. It instantiates the general message-passing scheme using one of three aggregation functions—mean, LSTM, or pooling—so that the same learned functions can later be applied to unseen nodes. GraphSAGE concatenates a node's own representation with the aggregated neighbour representation and L2-normalises the result after each layer. In order to improve scalability, it samples a fixed number of neighbours per node.

Relational Graph Convolutional Network

Relational Graph Convolutional Networks (R-GCNs) [30] extend graph convolutional networks to multi-relational graphs, in which edges represent different types of relationships.

Unlike a standard GCN, which applies the same transformation to messages from all neighbours, R-GCN applies a relation-specific transformation.

For a node v, the representation at layer $l+1$ is computed as

$$h_v^{(l+1)} = \sigma\left(\sum_{r\in\mathcal{R}} \sum_{u\in\mathcal{N}_r(v)} \frac{1}{c_{v,r}} W_r^{(l)} h_u^{(l)} + W_0^{(l)} h_v^{(l)}\right), \tag{3.3}$$

$\mathcal{R}$ — relation types, $\mathcal{N}_r(v)$ — neighbours of v under r, $c_{v,r}$ — normalisation

$W_r^{(l)}$ — relation-specific weights, $W_0^{(l)} h_v^{(l)}$ — self-loop, σ — activation

When the number of relation types is large, assigning a separate full weight matrix to each relation can lead to overfitting. R-GCN therefore introduces basis decomposition, in which each relation-specific matrix is expressed as a linear combination of B shared basis matrices:

$$W_r^{(l)} = \sum_{b=1}^{B} a_{rb}^{(l)} V_b^{(l)}, \tag{3.4}$$

$V_b^{(l)} \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$ — shared basis, $a_{rb}^{(l)}$ — coefficient for relation r

Transformer-based message passing

Transformer-based graph neural networks adapt the self-attention mechanism from the Transformer architecture of Vaswani et al. [35] to graph-structured data. For each node i, its input representation $\mathbf{h}_i$ is projected into query, key, and value vectors:

$$\mathbf{q}_i = \mathbf{W}_Q \mathbf{h}_i, \qquad \mathbf{k}_i = \mathbf{W}_K \mathbf{h}_i, \qquad \mathbf{v}_i = \mathbf{W}_V \mathbf{h}_i, \tag{3.5}$$

$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ — learnable projection matrices

For an edge from source node j to target node i, the attention coefficient is

$$e_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}}, \tag{3.6}$$

d_k — dimensionality of the key vectors

The attention coefficients are normalised across all neighbours of the target node using the softmax function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{r\in\mathcal{N}(i)} \exp(e_{ir})}, \tag{3.7}$$

$\mathcal{N}(i)$ — neighbourhood of target node i

The node's representation is updated by aggregating value projections of its neighbours, weighted by the learned attention coefficients:

$$\mathbf{h}_i' = \sum_{j\in\mathcal{N}(i)} \alpha_{ij} \mathbf{v}_j, \tag{3.8}$$

α_{ij} — attention weight, $\mathbf{v}_j$ — value projection of neighbour j

To capture multiple complementary interaction patterns, graph Transformers commonly use multi-head attention. The output of head m for node i is:

$$\mathbf{h}_i'^{(m)} = \sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(m)} \mathbf{v}_j^{(m)}, \tag{3.9}$$

$m\in\{1,\ldots,H\}$, $\mathbf{W}_Q^{(m)}, \mathbf{W}_K^{(m)}, \mathbf{W}_V^{(m)}$ — independent projections per head

The outputs of all H heads are combined by concatenation or averaging.

All architectures mentioned above are homogeneous from nature, but can be applied to heterogeneous graphs through an appropriate configuration of the `HeteroConv` layer in PyTorch Geometric (PyG). `HeteroConv` is a generic wrapper that applies a separately specified message-passing operator to each edge type, i.e., each meta-relation between a source node type and a target node type [37]. This mechanism is used in Task B which allowed to experiment with different graph architectures on joined setting.

Heterogeneous Graph Transformer

In response to the demand for preserving heterogeneous structure during modelling, Hu et al. [21] proposed the Heterogeneous Graph Transformer (HGT). For attention-based message passing on heterogeneous graphs, the Heterogeneous Graph Transformer (HGT) makes attention scores and message transformations dependent on the types of source and target nodes and on the edge relation. Unlike the `HeteroConv`-wrapped TransformerConv described above, which instantiates a fully independent set of attention parameters for each edge type and aggregates their outputs only after each layer has been computed separately, HGT shares a single underlying attention mechanism across all edge and node types, using type-specific projection matrices. This allows HGT to learn interactions across types directly, at a lower parameter cost as the number of types grows [21].

The HGT layer decomposes message passing into three components: heterogeneous mutual attention, heterogeneous message passing, and target-specific aggregation. Each component uses a separate set of weight matrices for every combination of source node type, edge type, and target node type – referred to as a *meta relation* $(\tau_s, \phi(e), \tau_t)$. So that HGT extends the query, key, and value projections introduced in Eq. (3.5) by conditioning each weight matrix on node type: $\mathbf{W}_Q^{\tau_t}$, $\mathbf{W}_K^{\tau_s}$, and $\mathbf{W}_M^{\tau_s}$ replace the shared $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$, so that

$$\mathbf{q}_t = \mathbf{W}_Q^{\tau_t}\mathbf{h}_t, \qquad \mathbf{k}_s = \mathbf{W}_K^{\tau_s}\mathbf{h}_s, \qquad \mathbf{m}_s = \mathbf{W}_M^{\tau_s}\mathbf{h}_s. \tag{3.10}$$

The attention coefficient additionally depends on the edge type through a relation-specific matrix $\mathbf{W}_{\text{ATT}}^{\phi(e)}$:

$$e_{ts} = \frac{\left(\mathbf{q}_t^\top \mathbf{W}_{\text{ATT}}^{\phi(e)} \mathbf{k}_s\right)}{\sqrt{d_k}}, \qquad \alpha_{ts} = \frac{\exp(e_{ts})}{\sum_{r\in\mathcal{N}(t)}\exp(e_{tr})}. \tag{3.11}$$

The message is similarly transformed by a relation-specific matrix $\mathbf{W}_{\text{MSG}}^{\phi(e)}$ and aggregated across all neighbours of t:

$$\mathbf{h}_t' = \sum_{s\in\mathcal{N}(t)} \alpha_{ts}\left(\mathbf{m}_s \mathbf{W}_{\text{MSG}}^{\phi(e)}\right) \text{ [21].} \tag{3.12}$$

Relation-specific transformations affect both the attention score and the transferred message, so that HGT assigns different importance to neighbours based not only on their features, but also on the semantic type of their connection to the target node.

As $\mathbf{h}_t'$ combines information from source nodes of potentially different types and feature distributions, it is mapped back into the target node's type-specific space via a target-type-dependent projection, followed by a non-linear activation and a residual connection to the previous layer:

$$\mathbf{h}_t'' = \mathbf{A}\text{-Linear}_{\tau_t}\left(\sigma\left(\mathbf{h}_t'\right)\right) + \mathbf{h}_t^{(l-1)} \text{ [21].} \tag{3.13}$$

This capacity to differentiate neighbours by both node type and relation semantics is particularly relevant to the knowledge graphs built for the purpose of this thesis, where clinically distinct relation types – such as drug–drug interactions versus drug–outcome relations – carry different predictive weight. HGT is the primary structural encoder in Task A and proxy to Task A, because typed pharmacotherapy and collaborative-knowledge-graph relations require type-aware message passing.

3.2 Statistical Dependence and Hierarchical Correlation Reconstruction

Empirical comparisons between graph-based and tabular models report mixed results, with gains from graph structure often modest and architecture-dependent [3, 33]. Rather than resolving this solely through architecture search, this thesis follows the Wide & Deep paradigm [6, 15], in which a linear or statistically-grounded branch complements a deep, graph-based branch. The wide branch memorises direct, low-order statistical dependencies that message passing may dilute through neighbourhood aggregation, while the deep branch generalises through learned relational representations.

Several statistical dependence measures were considered for this role. In particular the Hilbert–Schmidt Independence Criterion (HSIC) as it provides a well-established, non-parametric approach for detecting broad classes of statistical dependence, including nonlinear relationships. However, in its standard form HSIC summarizes dependence primarily through an aggregate statistic. This is useful for testing whether dependence is present, but provides less direct information about the components of the joint distribution in which that dependence is expressed. Finally, Hierarchical Correlation Reconstruction (HCR), proposed by proposed by Jarosław Duda, was selected because its orthonormal-basis representation provides a more explicit decomposition of statistical dependence. HCR represents it through a collection of coefficients associated with specific combinations of basis functions. This makes it possible to inspect which low- and higher-order components contribute to the observed dependence and therefore provides a richer and more interpretable statistical representation for downstream modelling. This property is particularly relevant in the present thesis to provide an explicit representation that can be combined with the topology-based graph encoder. A further practical advantage is computational simplicity. HCR coefficients are estimated directly as sample averages of basis-function products and can be computed independently, without iterative optimisation. The representation can therefore be expanded progressively by adding higher-order coefficients when more detail is required. These properties make HCR attractive in settings where statistical dependence must be estimated repeatedly across many candidate relations.

HCR represents probability distributions and statistical dependencies through coefficients of an orthonormal basis expansion [10, 9, 14, 11, 12]. Lower-order coefficients describe broad dependence patterns; higher-order coefficients capture more complex departures from independence.

For variables X and Y with orthonormal bases $\{\phi_j^{(X)}\}$ and $\{\phi_k^{(Y)}\}$, empirical cross-dependence coefficients are

$$a_{jk} = \frac{1}{n}\sum_{i=1}^{n} \phi_j^{(X)}(X_i)\, \phi_k^{(Y)}(Y_i). \tag{3.14}$$

For a binary variable $B \in \{0, 1\}$ with prevalence $p_B = P(B = 1)$,

$$\phi_1(B) = \frac{B - p_B}{\sqrt{p_B(1 - p_B)}}, \tag{3.15}$$

so the lowest-order coefficient a_{11} is a standardized first-order dependence descriptor. The same principle extends to other variable types via suitable marginal bases.

HCR has demonstrated practical value in domains requiring reliable, interpretable estimates of conditional distributions instead of single point predictions. In virtual screening for drug discovery, HCR-based prediction of ADMET property distributions enables inexpensive selection of compounds with properties nearly certain to fall within a desired range, while automatically flagging predictions with high uncertainty for manual review [13]. In income credibility assessment using Polish household budget survey data, HCR produced more accurate proxies for low-income households than conventional tax-record matching, showing its ability to model the conditional distribution directly instead of relying on a single regression estimate [14, 32].

Based on those results, in this work HCR is used as a statistical representation of empirical and associative dependence to complement the GNN representation. Causal direction and mechanistic interpretation are instead supplied by the directed graph schema itself built for the purpose of this thesis. In general the term HCR denotes the orthonormal-basis coefficient construction, whereas this thesis uses HCR-inspired representations – larger task-specific vectors that combine those coefficients with additional descriptors. In Task A they describe the local dependency motif around a candidate edge, while in Task B they supply an endpoint-neighbourhood evidence channel alongside the GNN representation. The common principle is combining learned graph representations with explicit statistical dependence estimated from observations.

Chapter 4

Dataset

4.1 Synthetic Pharmacotherapy Dataset

The primary dataset used in this thesis is Synthetic Pharmacotherapy Graph v3, a controlled benchmark designed to connect a known mechanistic graph structure with patient-level empirical observations. The dataset was constructed specifically to support both tasks investigated in the thesis: reconstruction of graph relations in Task A and patient-level endpoint prediction in Task B. It consists of two complementary components: a directed pharmacotherapy graph and patient-level records generated from that graph.

The use of synthetic data provides access to a known reference graph and to the complete data-generating process. This makes it possible to introduce controlled observational artefacts while preserving the underlying mechanistic structure. The dataset should therefore be interpreted as methodological ground truth for model evaluation. Graph coefficients, baseline probabilities and realised endpoint rates are simulation parameters and must not be interpreted as estimates of real clinical effects.

4.2 Pharmacotherapy Knowledge Graph

The graph was designed as a directed acyclic graph (DAG) representing pathways from patient characteristics and drug exposures, through physiological mechanisms and intermediate adverse states, to clinical outcomes. Graph construction started from clinically relevant endpoints and proceeded backwards through their expected determinants and mechanistic pathways.

Domain knowledge in pharmacology was used to define and review the initial set of drug classes, risk factors, mechanisms and clinical endpoints. Claude Fable 5 was used as an AI-assisted modelling tool to propose candidate mechanistic pathways, including drug–drug and drug–disease interaction structures. Importantly, the model was used to support pathway drafting rather than to determine the final graph automatically. Proposed relations were reviewed and could be accepted, rejected or modified before inclusion in the graph specification.

The graph was developed iteratively and subsequently audited for structural consistency. The final v3 graph contains 155 nodes and 405 directed edges and is acyclic, with no duplicate nodes, duplicate edges or references to missing nodes. The nodes are organised into six semantic categories:

These categories distinguish patient context, treatment, biological mechanisms, intermediate consequences, observation processes and final clinical events.

Table 4.1: Node types in Synthetic Pharmacotherapy Graph v3.

Node type	Number	Role
`patient_context`	24	demographics, comorbidities and other patient-level characteristics
`drug_exposure`	29	exposure to drug classes
`mechanism`	49	physiological mechanisms, interaction gates and drug-load variables
`adr_or_intermediate_state`	28	intermediate adverse or physiological states
`observation_or_selection`	12	testing, documentation and selection processes
`clinical_endpoint`	13	target clinical outcomes

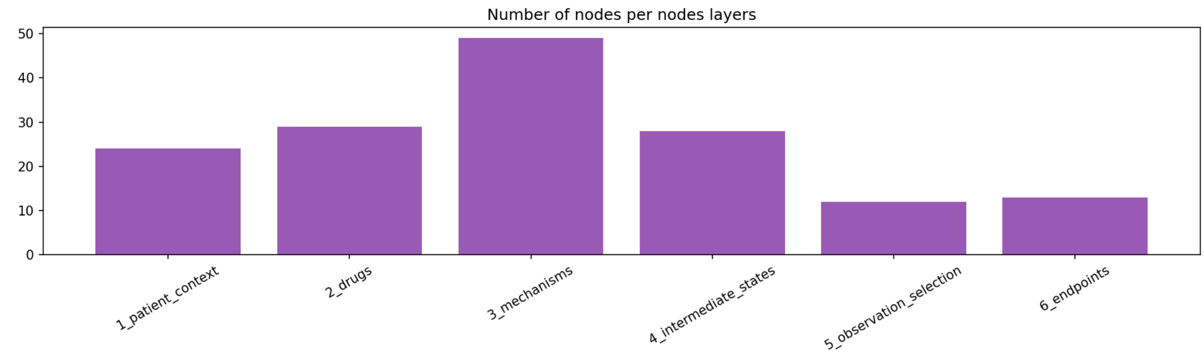

Figure 4.1: Number of nodes in each semantic layer of Synthetic Pharmacotherapy Graph v3.

Directed edges represent different semantic relationships, including background risk, treatment assignment, drug-to-mechanism effects, mechanistic progression, adverse-state progression, observation processes, and drug–drug or drug–disease interactions. Edge attributes include the simulated magnitude and sign of an effect, which are subsequently used by the patient generator.

The graph contains 13 clinical endpoints regarded as points of clinical state requiring intervention: AKI, DILI, Depression, Falls, Delirium, GI bleeding, Hyponatremia, Hyperkalemia, QT arrhythmia, Hospitalization, Serotonin syndrome, Rhabdomyolysis, and Lactic acidosis.

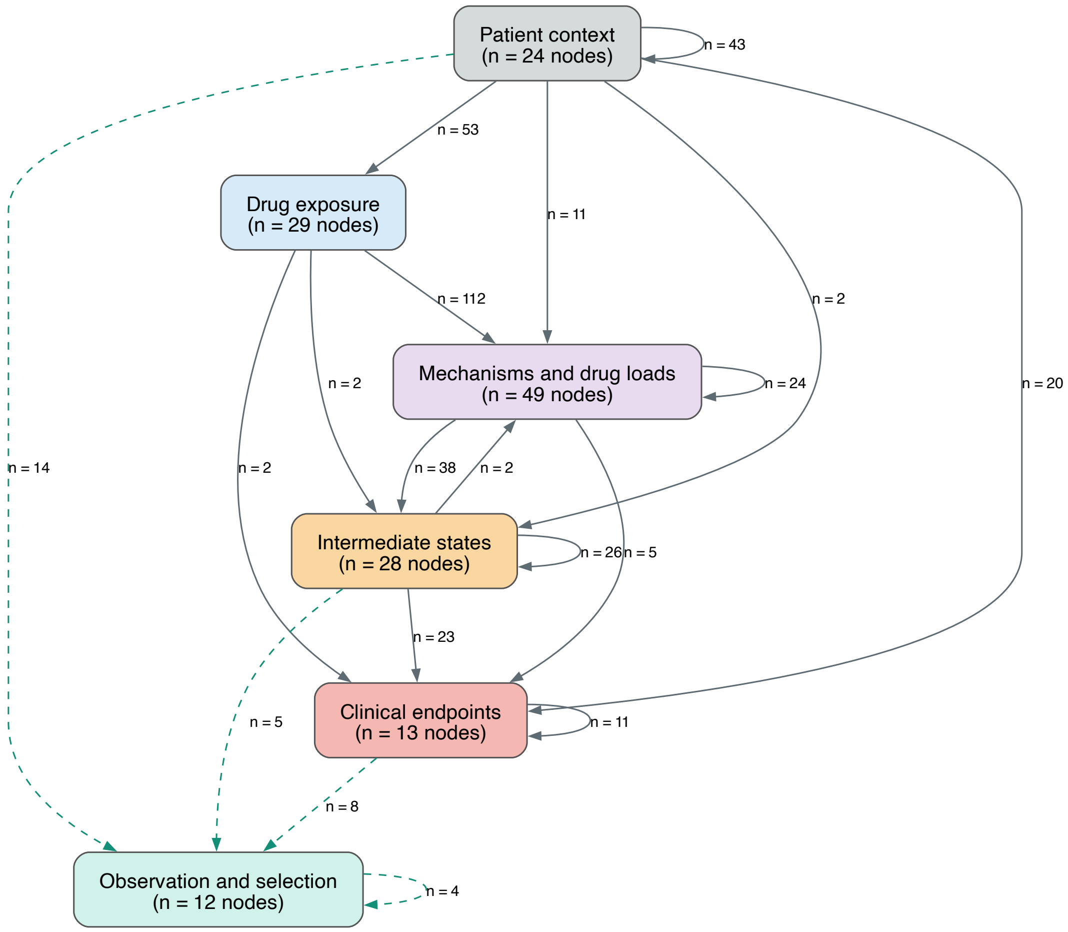

Figure 4.2: Overview of the synthetic pharmacotherapy DAG with node colours by semantic type.

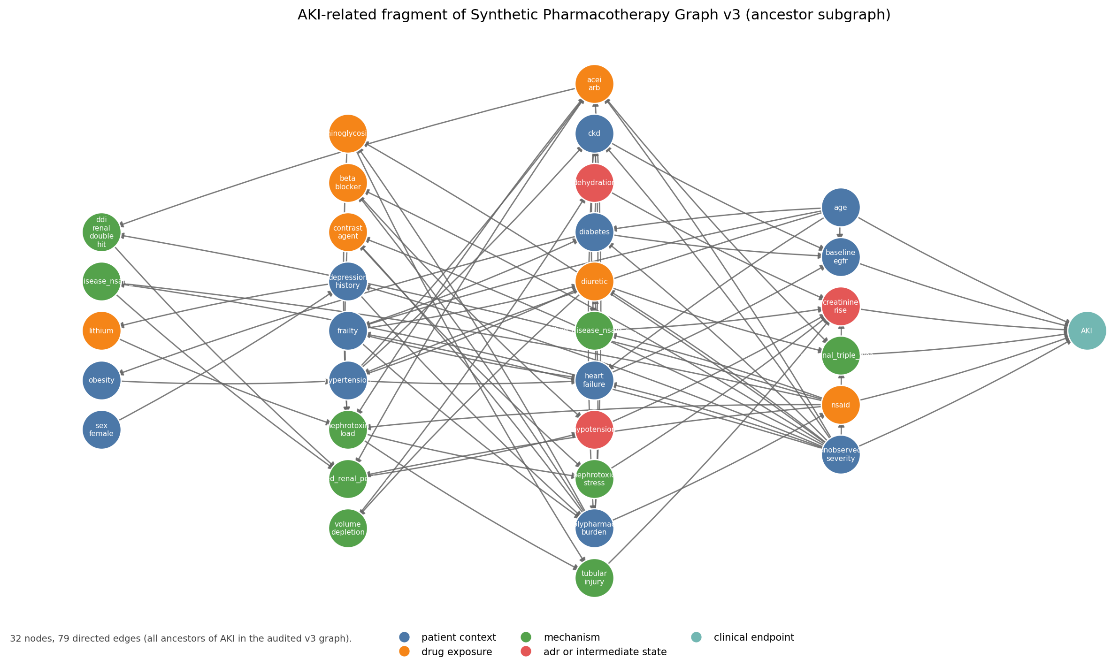

Figure 4.3: AKI-related fragment of the Synthetic Pharmacotherapy Graph v3: all audited ancestors of the AKI endpoint (32 nodes / 79 edges), drawn with NetworkX. Node colours follow the v3 ontology (patient context, drug exposure, mechanism, ADR/intermediate state, clinical endpoint). The full graph is larger; this excerpt is for orientation.

4.3 Patient-Level Data Generation

The graph defines a common mechanistic structure, whereas the patient table represents individual realisations of that structure. For each simulated patient, values are generated for the graph variables in topological order. Patient-context variables are generated first, followed by drug exposures, mechanisms, intermediate states and finally clinical endpoints. As a result, a parent variable is always realised before a downstream child variable is generated.

Most stochastic binary nodes are generated using an additive logistic model. For node i,

$$\eta_i = \operatorname{logit}(p_i^0) + \sum_{j \in \mathrm{parents}(i)} s_{ij}\, w_{ij}\, x_j, \tag{4.1}$$

followed by

$$x_i \sim \mathrm{Bernoulli}(\sigma(\eta_i))\,, \tag{4.2}$$

where p_i^0 denotes the baseline probability of the node, x_j is the realised value of parent j, and w_{ij} and s_{ij} denote the simulated effect magnitude and sign associated with the edge $j \to i$. Count and continuous parents are standardised before entering the additive predictor.

Not all variables are stochastic. The generator also contains deterministic quantities representing clinically meaningful combinations such as drug loads, drug–drug interaction gates, drug–disease interactions and cumulative adverse-event burden. For example, some interaction variables are activated only when several required drug exposures are simultaneously present.

The dataset additionally separates an underlying clinical state from whether that state is observed or documented. For example,

$$\texttt{elevated\_creatinine\_recorded} = \texttt{creatinine\_rise} \times \texttt{creatinine\_tested}. \quad (4.3)$$

A true creatinine rise can therefore occur without being present in the recorded representation if testing is not performed. This distinction allows documentation and selection artefacts to be introduced without changing the underlying mechanistic graph.

The clean dataset contains 20,000 simulated patients. Each row corresponds to one patient, while graph nodes correspond to patient-level variables. The 155 graph variables include binary, count and continuous values; the dataset additionally contains technical identifiers such as `patient_id` and `hospital_id`.

4.4 Observational Scenarios

Six scenarios were generated from the same underlying pharmacotherapy graph. They were designed to test whether the models remain robust when the patient-level evidence is affected by different observational artefacts.

Table 4.2: Observational scenarios derived from the same pharmacotherapy graph.

Scenario	Purpose	Patients
`clean`	reference setting with complete generated information	20,000
`hidden_confounder`	latent severity remains active in generation but is withheld from observed data	20,000
`selection_bias`	cohort restricted according to healthcare-contact and testing variables	14,062
`no_overlap`	selected treatment exposures become rare in particular patient subgroups	20,000
`noisy_documentation`	imperfect recording with false-negative and false-positive documentation	20,000
`multihospital`	heterogeneous treatment and documentation processes across four simulated hospitals	20,000

The scenarios differ in the observability and distribution of patient-level data, while the underlying graph semantics are preserved. Importantly, the `selection_bias` dataset is an intentional subset of the original population and therefore contains 14,062 patients.

4.5 Patient Splits and Use Across Tasks

Patients are divided into training, validation and test sets in proportions of 70%, 15% and 15%. For the clean cohort, this corresponds to 14,000 training patients, 3,000 validation patients and 3,000 test patients. The split is defined at the patient level and reused across the alternative scenarios to maintain comparable patient partitions whenever possible.

The two tasks use this split differently. In Task A, patient records are not themselves the prediction instances: the training-patient partition is used to estimate node-level statistics and pairwise dependence measures, while the prediction instances are candidate graph edges. The Task A candidate inventory, negative-sampling rule and edge-level split are defined in Chapter 5.

In Task B, the patient split itself defines the training, validation and test populations. Each patient contributes individual node values and clinical endpoint labels, while the graph provides the common mechanistic scaffold.

All statistics derived from patient-level data are computed using the appropriate training partition only. For Task B, endpoint values, post-event or definitional intermediates, and other variables unavailable at the intended prediction point are excluded from predictive inputs.

Chapter 5

Task A. Directed Link Prediction on the Synthetic Pharmacotherapy Graph

Task A asks whether directed link prediction on a heterogeneous pharmacotherapy graph improves when a learned structural representation is combined with an explicit statistical description of patient-level dependence around each candidate edge. The structural branch encodes training topology; the statistical branch encodes empirical dependence estimated from training patients; a shared decoder fuses both sources into an edge score.

5.1 Problem Formulation

We address directed link prediction among observable nodes of the Synthetic Pharmacotherapy Graph v3 (Chapter 4). For a candidate directed edge $A \to G$, the model produces a score

$$s_{AG} = f_\theta(A, G; G_{\text{train}}, X_{\text{train}}), \tag{5.1}$$

where G_{train} is the message-passing graph available during training and X_{train} denotes the training-patient observations used to construct node features and pairwise statistical descriptors. The score is transformed through the sigmoid function,

$$\hat{p}_{AG} = \sigma(s_{AG}), \tag{5.2}$$

and the resulting value is used in the binary edge-classification objective.

A positive label indicates that the directed relation is present in the audited graph, whereas a negative label denotes a type-allowed non-edge. The prediction is based on two complementary sources of evidence:

- *graph structure*: the typed neighbourhood of A and G represented in G_{train};
- *patient-derived statistical evidence*: empirical dependence patterns estimated from training patients for the local motif surrounding the candidate edge.

Directionality is determined by the ordered candidate definition and the graph schema, while the pairwise dependence coefficients provide statistical evidence about the variables involved in the candidate relation.

5.2 Task-Specific Data Preparation

The shared graph and patient scenarios are defined in Chapter 4. This section specifies how that resource is turned into prediction instances, a message-passing graph, and node features for Task A. The candidate edge inventory is held fixed across observational scenarios, so scenario differences arise from changes in patient-level observability rather than from changes in which edges are scored.

5.2.1 Candidate edges and splits

After restricting the audited graph to observable nodes, the candidate set contains 366 positive directed edges and 1,098 negatives ($r_{\text{neg}} = 3$), for 1,464 candidates in total. Negatives are drawn from non-edges that satisfy

$$u \neq v, \qquad (u, v) \notin E^+, \qquad \big(\text{type}(u), \text{type}(v)\big) \in T_{\text{allowed}}, \tag{5.3}$$

rather than from a uniform draw over all ordered node pairs.

Candidates are split independently of the patient split, at edge level, into train/validation/test partitions with proportions 70/15/15, stratified by the binary edge label. This yields 256/55/55 positives and 768/165/165 negatives. Patient records are not prediction instances: the training-patient partition is used only to estimate node statistics and pairwise dependence, while labels are attached to candidate edges.

5.2.2 Message-passing graph

The HGT encoder operates on a heterogeneous training graph G_{train} that is smaller than the full audited graph. Only observable nodes enter the encoder; latent generator nodes are excluded. Node types and relation names follow the audited v3 ontology (Chapter 4).

For each positive training relation $u \xrightarrow{r} v$, an auxiliary reverse relation $v \xrightarrow{r^{-1}} u$ is added for message passing. Reverse relations have distinct identities and enable bidirectional information flow without implying that the underlying mechanistic relation is reversible.

Message passing uses only positive training edges from the candidate set. Validation positives, test positives, and all negative candidates are excluded from G_{train}. Held-out candidates are scored by referencing their endpoints in the node embedding table; they are never inserted as edges into G_{train}.

5.2.3 Node features

Each observable node v receives a three-dimensional empirical feature vector computed from training patients only,

$$x_v = \Big[\mu_v^{\text{train}},\ \sigma_v^{\text{train}},\ m_v^{\text{train}}\Big], \tag{5.4}$$

where μ_v^{train} and σ_v^{train} are the train-patient mean and standard deviation of v, and m_v^{train} is the train missingness rate. Identity embeddings are not used. Before message passing, each node type is projected independently,

$$h_v^{(0)} = \text{Dropout}\Big(\text{ReLU}\Big(\text{LayerNorm}(W_{\text{type}(v)}\, x_v)\Big)\Big), \tag{5.5}$$

with $W_{\text{type}(v)} : \mathbb{R}^3 \to \mathbb{R}^d$. Input statistics are fixed after preprocessing; projection and convolution weights are learned.

5.3 Model Architecture

5.3.1 Architecture Overview

Figure 5.1 summarizes the full model. The structural branch maps G_{train} through HGT to a graph pair representation g_{graph}. The statistical branch maps the local $AZ/AG/ZG$ motif through role-specific encoders to g_{stat}. Concatenation and a shallow decoder produce the edge logit.

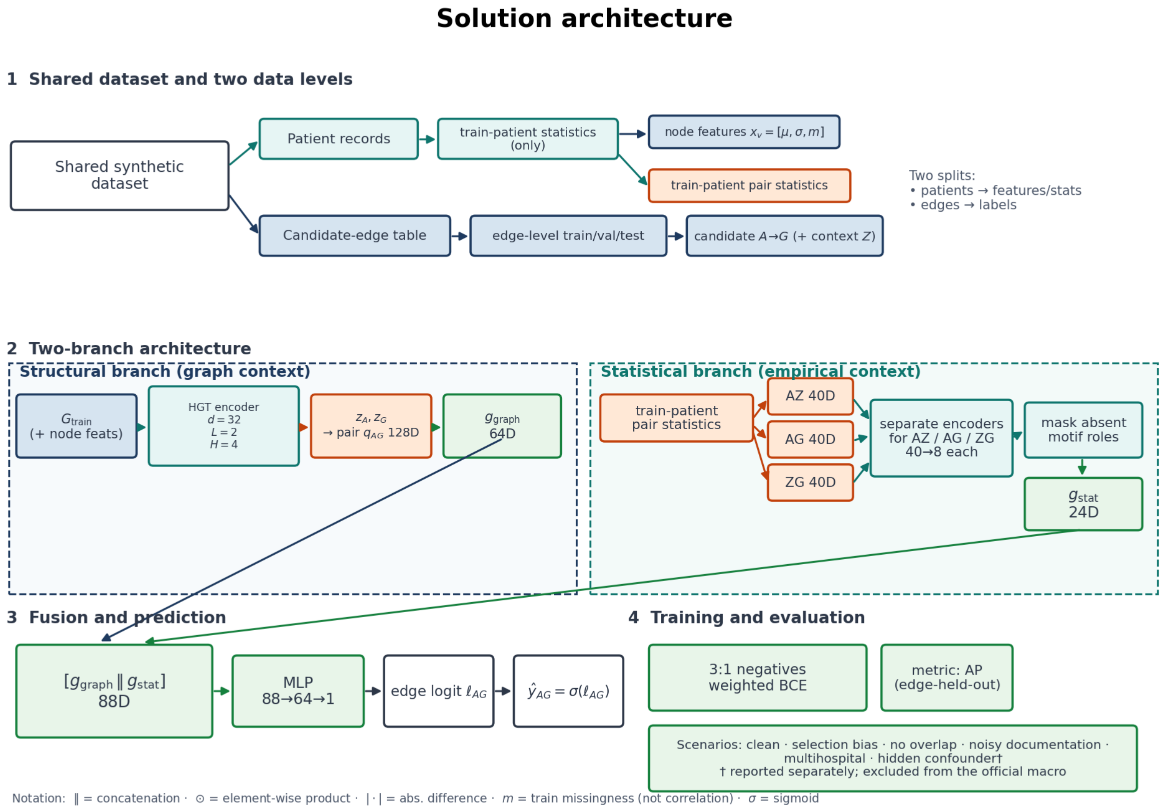

Figure 5.1: Solution architecture for Task A. Patient records yield train-only node features $x_v = [\mu, \sigma, m]$ and pair statistics; candidate edges are split independently at edge level. The structural branch maps G_{train} through HGT to $g_{\text{graph}} \in \mathbb{R}^{64}$; the statistical branch encodes roles $AZ/AG/ZG$ with separate encoders and masks absent roles to $g_{\text{stat}} \in \mathbb{R}^{24}$; fusion yields an edge logit.

5.3.2 Structural Branch

The graph encoder is a Heterogeneous Graph Transformer [21] with

$$d = 32, \qquad L = 2, \qquad H = 4, \tag{5.6}$$

dropout 0.25, LeakyReLU activations, and a residual connection with LayerNorm after each convolution layer. These residual links are standard skip connections inside the encoder; they are not studied as an anti-oversmoothing intervention.

After L layers, each observable node has an embedding $z_v \in \mathbb{R}^{32}$. For candidate $A \to G$, the ordered pair representation is

$$q_{AG} = \left[z_A \parallel z_G \parallel z_A \odot z_G \parallel |z_A - z_G| \right] \in \mathbb{R}^{128}. \tag{5.7}$$

An MLP graph branch (128 → 128 → 64 with GELU and LayerNorm) compresses q_{AG} to

$$g_{\text{graph}} \in \mathbb{R}^{64}. \tag{5.8}$$

5.3.3 Statistical Evidence Branch

The general Hierarchical Correlation Reconstruction (HCR) formulation is stated in Chapter 3. Task A uses an HCR-inspired fixed-width pair vector so that binary, count, continuous and mixed pairs share one downstream interface. HCR coefficients quantify empirical dependence; they do not identify causal direction.

For an ordered pair (U, V), let I_{UV} be the set of training patients with both variables observed and $n = |I_{UV}|$. Type-specific orthonormal bases $\{\phi_j^{(U)}\}$ and $\{\phi_k^{(V)}\}$ yield coefficients

$$a_{jk} = \frac{1}{n} \sum_{i \in I_{UV}} \phi_j^{(U)}(U_i)\, \phi_k^{(V)}(V_i), \tag{5.9}$$

zero-padded to a 4×4 matrix and stored in the first 16 dimensions of

$$h(U, V) \in \mathbb{R}^{40}. \tag{5.10}$$

A binary variable contributes one non-constant contrast; continuous and jittered count variables may contribute up to four low-order basis functions, so the active block ranges from 1×1 (binary–binary) to 4×4.

The remaining dimensions summarize dependence magnitude and complexity, marginal context, joint activity and rarity, estimability, and variable type (Table 5.1). All quantities are fitted on training patients only. An estimability indicator distinguishes unsupported pairs (coefficients zeroed) from genuine near-zero dependence. Most components are intrinsically bounded or constructed in standardized bases; LayerNorm is applied inside the nonlinear pair encoder. Complete formula-level definitions of the type-specific bases and all 40 vector components are provided in Appendix A.

Table 5.1: Grouped composition of the 40-dimensional HCR-inspired pair representation.

Block	Dim.	Purpose
HCR-inspired coefficients	16	padded dependence-structure matrix A_{pad}
Dependence summaries	4	magnitude and complexity of retained coefficients
Marginal context	8	type-dependent descriptors of U and V
Joint activity / rarity	4	co-activation enrichment and rarity
Support	2	complete-case fraction and estimability flag
Variable types	6	one-hot types of U and V

For binary–binary pairs, signed Pearson ϕ is the natural first-order dependence coefficient and corresponds to the lowest-order HCR contrast; richer blocks retain higher-order structure that a single scalar association cannot express.

5.3.4 Fusion and Prediction

For each candidate $A \to G$, a local context node Z defines three ordered roles AZ, AG and ZG (source–context, candidate, context–target). The same 40-dimensional construction is applied to each role, yielding a raw motif of 120 dimensions. Each role is mapped by an independent MLP encoder

$$f_r : \mathbb{R}^{40} \to \mathbb{R}^{16} \to \mathbb{R}^{8}, \tag{5.11}$$

followed by a hard role gate

$$e_r = m_r\, f_r(h_r), \qquad m_r \in \{0, 1\}, \tag{5.12}$$

so that absent roles (especially missing Z) contribute exactly zero. The statistical branch output is

$$g_{\text{stat}} = [e_{AZ} \parallel e_{AG} \parallel e_{ZG}] \in \mathbb{R}^{24}. \tag{5.13}$$

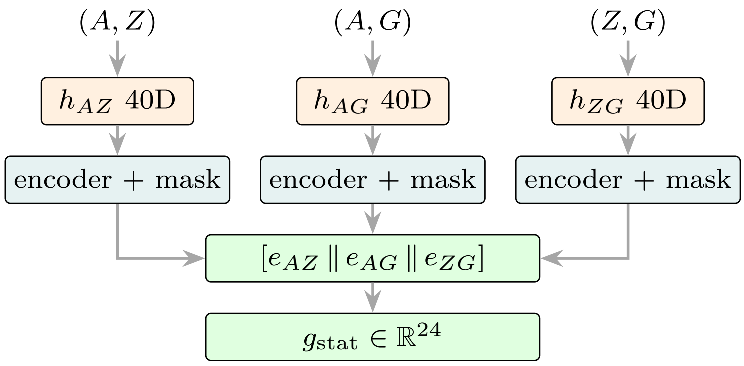

Figure 5.2: Role-aware motif for $A \to G$: three ordered pair roles, each with a separate encoder and role mask, concatenated to $g_{\text{stat}} \in \mathbb{R}^{24}$.

Fusion concatenates the branches,

$$g_{\text{fusion}} = \left[g_{\text{graph}} \parallel g_{\text{stat}}\right] \in \mathbb{R}^{88}, \tag{5.14}$$

and maps $88 \to 64 \to 1$ (GELU, LayerNorm, dropout) to a single edge logit.

5.4 Training and Evaluation

5.4.1 Training Objective

For each candidate i with logit s_i and label $y_i \in \{0, 1\}$,

$$\hat{p}_i = \sigma(s_i), \tag{5.15}$$

and parameters are optimized by weighted binary cross-entropy,

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \Big[w_+\, y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \Big], \tag{5.16}$$

with $w_+ = N_{\text{neg}}/N_{\text{pos}} \approx 3$ under the 3:1 negative-sampling ratio. Weighting balances class contributions to the loss; it does not change the candidate inventory.

5.4.2 Evaluation Metrics

Model selection uses validation Average Precision (AP) [28, 8],

$$\mathrm{AP} = \sum_k \Big(R_k - R_{k-1}\Big)\, P_k, \tag{5.17}$$

where P_k and R_k are precision and recall at successive thresholds on the ranked score list ($R_0 = 0$). This is the Average Precision summary of the ranked prediction list used throughout the thesis. AP is appropriate for imbalanced link prediction because it emphasizes precision as recall increases. The sealed test set is never used to choose architecture, representation or hyperparameters. When discrete metrics such as F_1 are reported, the classification threshold is selected on validation only.

5.4.3 Model and Training Configuration

Table 5.2 lists the model and training configuration of the full model. Six observational scenarios are evaluated; the official macro averages five scenarios and excludes the hidden-confounder scenario, which is reported for completeness only.

Table 5.2: Task A: model and training configuration.

Setting	Value
Optimizer	Adam [25]
Learning rate	10^{-3}
Weight decay	5×10^{-4}
Max epochs	200
Early stopping	patience 40 on validation AP
Gradient clip	1.0 (global norm)
Normalization	LayerNorm in HGT and decoder blocks [1]
HGT	$d = 32$, $L = 2$, $H = 4$, dropout 0.25 [21]
Statistical branch	HCR-inspired 40D pair vectors; MLP role encoders
Fusion	$64 + 24 = 88$ dimensional late fusion
Training seeds	five fixed seeds
Selection metric	validation AP
Official macro	mean over five scenarios (excludes hidden confounder)

5.5 Experimental Results

Results address three questions: which graph backbone to use, how much explicit statistical context contributes, and how the full model performs across observational scenarios. Architecture and hyperparameters were selected on validation AP before sealed-test reporting.

5.5.1 Architecture Comparison

With the statistical branch disabled and a common decoder, HGT [21], R-GCN [30], GraphSAGE [16] and GATv2 [4] were compared on the clean scenario. Mean validation AP was approximately 0.729 (HGT), 0.697 (R-GCN), 0.671 (GraphSAGE) and 0.650 (GATv2). HGT was retained as the structural encoder for all subsequent experiments.

5.5.2 Contribution of Statistical Evidence

With HGT fixed, explicit pairwise statistics were enriched progressively on the clean scenario (three seeds). Validation AP rose from approximately 0.719 with no pairwise statistics, to 0.840 with signed Pearson ϕ, 0.853 with normalized mutual information, 0.913 with an enriched multi-component representation, and 0.917 with the full 40-dimensional HCR-inspired vector. The graph-only value ≈ 0.719 in this ablation is not identical to the ≈ 0.729 reported in the backbone comparison above: the former is the HGT-only arm of the statistical-enrichment ladder under a shared decoder and seed set, whereas the latter compares alternative graph encoders under a matched backbone-selection protocol. Across the five official scenarios the full representation reached a macro validation AP of approximately 0.915. Removing the four derived dependence-summary dimensions reduced validation AP by about 0.008 on average; the full vector was retained.

5.5.3 Final Evaluation

Figure 5.3 reports the full model under the official multi-scenario protocol. Official macro test AP is 0.894 and test AUROC is 0.967. Performance remains stable under selection bias, noisy documentation and related observational artefacts; the largest train–test AP gap appears in the multi-hospital scenario.

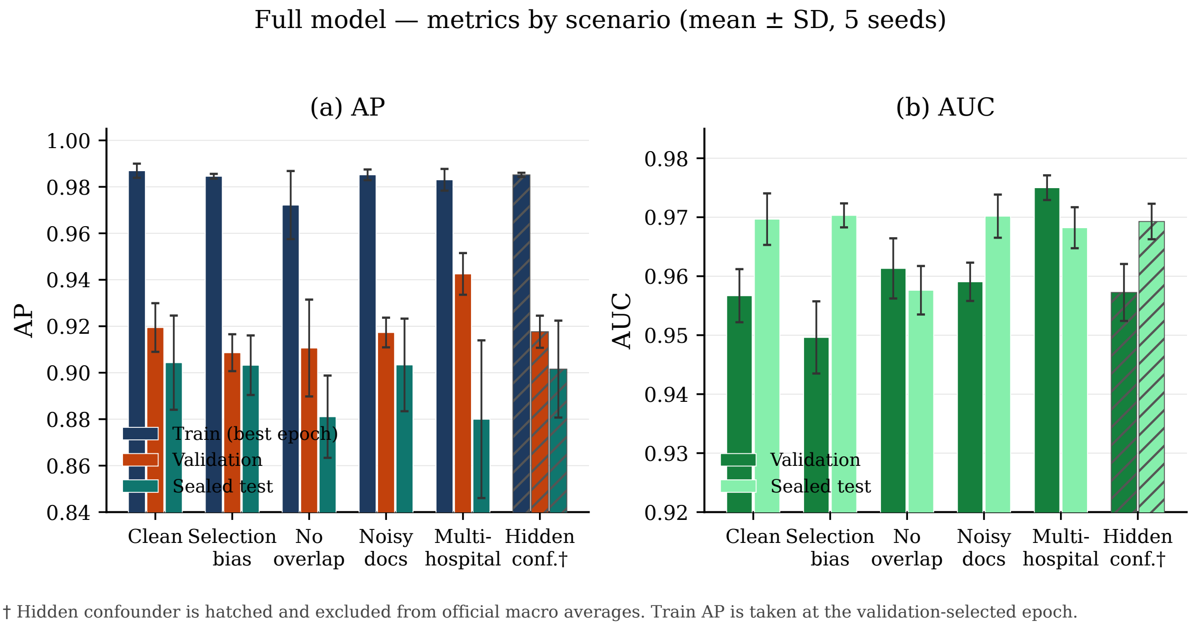

Figure 5.3: Task A: final performance across observational scenarios. Panel (a): AP on the training partition (at the validation-selected epoch), on validation, and on the sealed test set. Panel (b): AUROC on validation and sealed test. Bars are means over five training seeds; whiskers are one standard deviation. The hatched scenario is excluded from official macro averages. Official macro test AP is 0.894 and test AUROC is 0.967 (five-scenario mean; hidden confounder excluded).

Chapter 6

Cross-Domain Validation of Task A. Last-FM*

Last-FM* serves as a cross-domain validation of the Task A modelling pattern: a heterogeneous graph encoder combined with explicit candidate-specific empirical context. The transferred template is structural representation plus candidate-conditioned statistics fused in a decoder; the domain and prediction object change from pharmacotherapy edge classification to user–item ranking.

6.1 Problem Formulation

We consider an implicit-feedback user–item ranking problem on a heterogeneous collaborative knowledge graph built from Last-FM listening events and Freebase-style music relations. For a user u and a candidate catalog item X, the model assigns a ranking score

$$s(u, X) = f_\theta(u, X; G_{\text{train}}, D_{\text{train}}), \tag{6.1}$$

where G_{train} is the collaborative knowledge graph available to the model and D_{train} denotes the training interaction data. Higher scores indicate items ranked as more compatible with the user's observed training context.

The methodological question is whether ranking improves when an HGT representation of collaborative graph structure is combined with explicit candidate-specific empirical context derived from user interactions.

6.2 Dataset and Collaborative Knowledge Graph

6.2.1 Leakage-corrected Last-FM*

Experiments use Last-FM*, the leakage-corrected variant of the Last-FM benchmark associated with KGAT [36, 31]. Overlapping train–test interactions are removed and interactions are re-split, yielding zero train–test interaction overlap. Table 6.1 compares the original dump with the corrected resource; Table 6.2 states the working splits.

Validation is carved exclusively from the corrected training set under a fixed seed and a minimum-history constraint. Full-catalog validation uses 23,163 users with at least one validation positive. Working matrices contain binary implicit feedback only.

Table 6.1: Last-FM dump versus leakage-corrected Last-FM*.

Quantity	Original Last-FM dump	Last-FM*
Users	23,566	23,529
Items	48,123	48,123
Official train interactions	1,289,003	1,235,905
Official test interactions	423,635	306,914
Users with train/test overlap	17,290	0

Table 6.2: Working splits used in the Last-FM* experiment. Official test remains sealed.

Split	Interactions	Use
model train	1,111,803	CKG construction, histories, statistical fitting, training
validation	124,102	checkpoint selection and full-catalog validation
test	306,914	sealed

6.2.2 Collaborative knowledge graph

The HGT encoder operates on a heterogeneous collaborative knowledge graph with two node classes: user and entity. Catalog items form a subset of the entity nodes; remaining entities come from the external music knowledge graph.

Message-passing relations comprise user–item interaction edges from the model-train split (and auxiliary reverse relations), together with entity–entity knowledge-graph relations under a deterministic cap of 250,000 edges (and their reverses). Validation and test user–item interactions are absent from the message-passing graph.

6.2.3 Training candidate pairs

Positive training pairs are the observed model-train user–item interactions. For each positive pair, four negative user–item pairs are sampled ($r_{\text{neg}} = 4$), excluding the user's known interaction history and known positive items. The same candidate table is reused across representation ablations so that experiments change features rather than training examples.

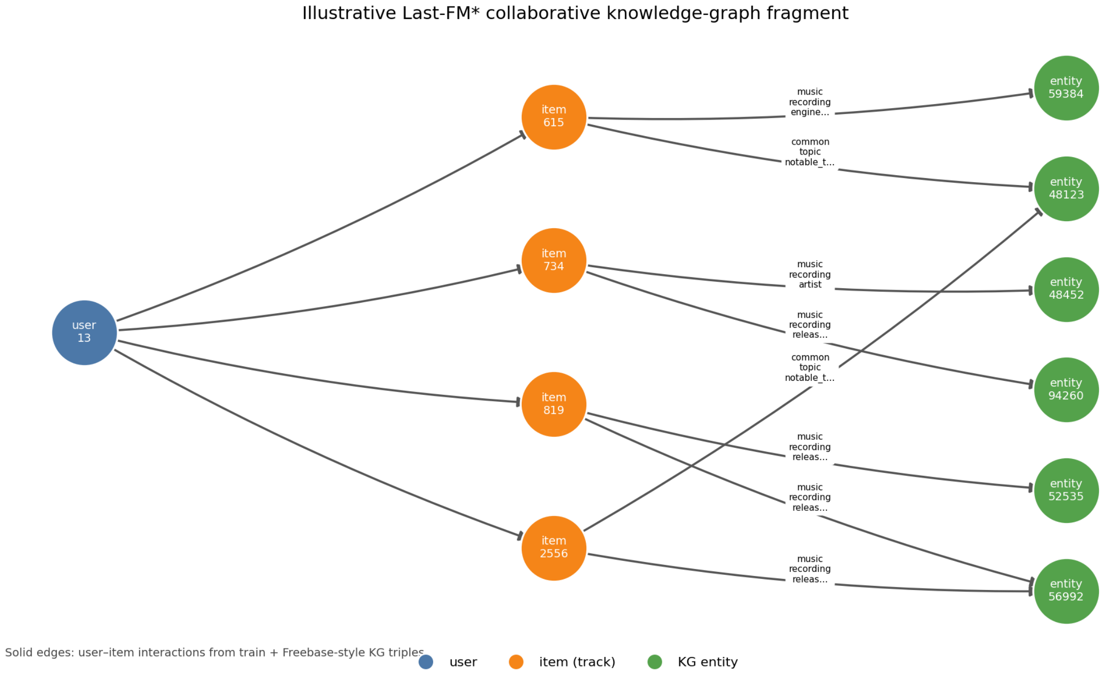

Figure 6.1: Illustrative fragment of the Last-FM* collaborative knowledge graph: a user, training interactions, and Freebase-style entity neighbours of catalog items.

6.3 Model Architecture

6.3.1 Architecture Overview

Figure 6.2 summarizes the complete model. An HGT encodes the collaborative knowledge graph; A5 and H3 supply candidate-specific empirical context; late fusion produces a main score; a zero-initialized LEG_{K2} residual adds a second-order correction.

6.3.2 Structural Branch

Each user and each entity receives a learnable embedding of dimension $d = 64$, initialized with Xavier uniform weights and updated jointly with the decoder. HGT node inputs are therefore learned embeddings; popularity and co-occurrence statistics are not injected into these tables.

The encoder uses

$$d = 64, \qquad L = 2, \qquad H = 2, \tag{6.2}$$

with dropout 0.1. After message passing on the training collaborative graph, let $z_u, z_X \in \mathbb{R}^{64}$ denote the embeddings of user u and candidate item X. The ordered pair representation is

$$q_{uX} = \left[z_u \parallel z_X \parallel z_u \odot z_X \parallel |z_u - z_X| \right] \in \mathbb{R}^{256}, \tag{6.3}$$

together with the scalar interaction $d_{uX} = \langle z_u, z_X \rangle$, for 257 structural dimensions in total.

6.3.3 Statistical Evidence Branch / Candidate-Specific Context

Candidate context is defined relative to the user's training history H_u at several resolutions: A5 compares the complete history with the candidate co-occurrence neighbourhood;

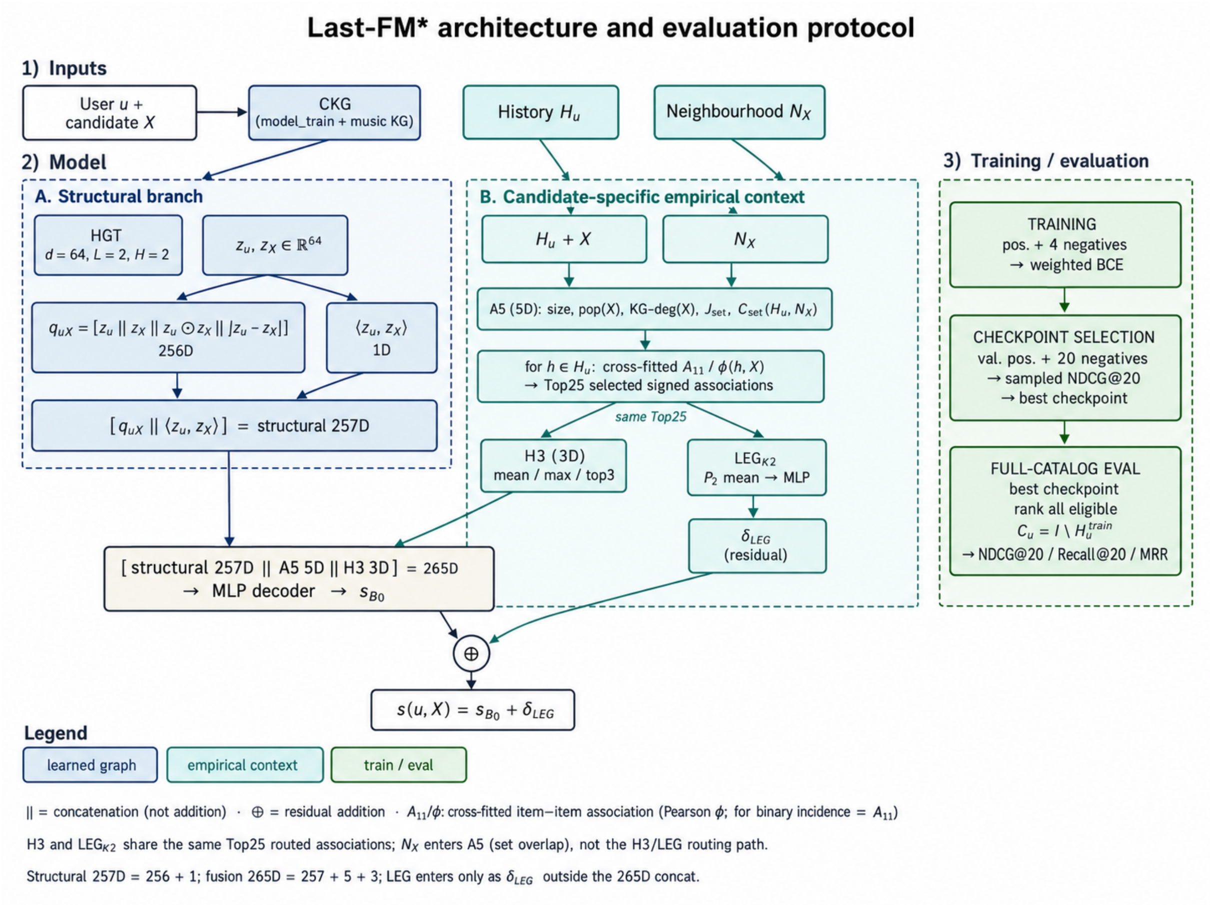

Figure 6.2: Last-FM* architecture. Structural HGT yields $[q_{uX} \parallel \langle z_u, z_X \rangle] \in \mathbb{R}^{257}$. A5 uses H_u, X and N_X; routed Top25 associations feed both H3 and LEG_{K2}, with LEG entering only as residual δ_{LEG}. Late fusion is 265-dimensional ($\parallel$ = concatenation; $\oplus$ = residual addition). The right column summarizes training and the two development evaluation levels; the sealed external benchmark is defined in Section 6.4.

Pearson ϕ (A11) scores individual history items against X for routing; H3 summarizes the signed associations of the selected items; LEG_{K2} summarizes their second-order magnitude.

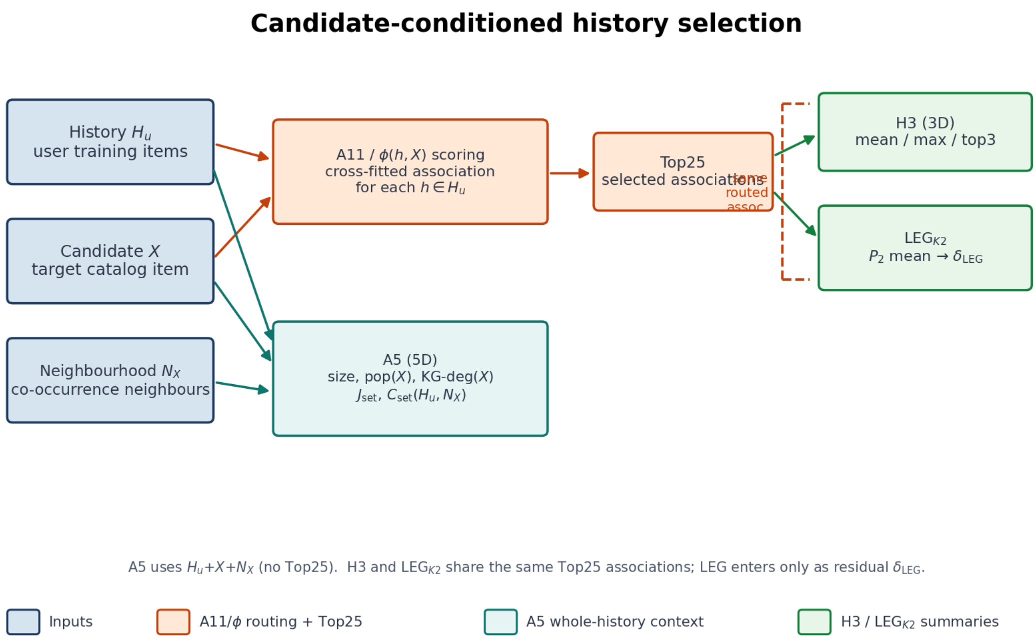

Figure 6.3: Candidate-conditioned history selection: A5 summarises whole-history context with N_X; Top25 routing by cross-fitted A_{11}/ϕ feeds both H3 and LEG_{K2} from the same selected associations.

Cross-fitted item–item association. For history item h and candidate X, co-occurrence across users yields contingency counts from which

$$\phi(h, X) = \frac{N n_{11} - n_h n_X}{\sqrt{n_h (N - n_h)\, n_X (N - n_X)}}, \qquad \phi \in [-1, 1]. \tag{6.4}$$

Association counts for user u are computed from a five-fold partition of training interactions, excluding the fold that contains u. Validation and test interactions never enter these counts. The candidate co-occurrence neighbourhood is $N_X = \{j \neq X : n_{11}(j, X) > 0\}$.

A5: full-history candidate context.

$$\begin{aligned} A(u, X) = \Big[\log(1 + |H_u|),\ \log(1 + \mathrm{pop}(X)),\ \log(1 + \deg_{\mathrm{KG}}(X)), \\ J_{\mathrm{set}}(H_u, N_X),\ C_{\mathrm{set}}(H_u, N_X)\Big] \in \mathbb{R}^5, \end{aligned} \tag{6.5}$$

where

$$J_{\mathrm{set}}(H_u, N_X) = \frac{|H_u \cap N_X|}{|H_u \cup N_X|}, \qquad C_{\mathrm{set}}(H_u, N_X) = \frac{|H_u \cap N_X|}{\sqrt{|H_u|\,|N_X|}}, \tag{6.6}$$

with empty-set cases mapped to zero. A5 is standardized on training pairs only.

Candidate-conditioned routing and H3. For each $h \in H_u \setminus \{X\}$, the routing score is $v(h, X) = \phi(h, X)$. Alternative association criteria (Jaccard, cosine, co-occurrence, MI, NPMI, G^2) were screened under a fixed Top25 budget; signed Pearson ϕ was retained. The Top$K$ items with $K = \min\bigl(25, |H_u \setminus \{X\}|\bigr)$ are summarized by

$$H_3(u, X) = \Bigl[\text{mean}(v),\ \max(v),\ \text{top3mean}(v)\Bigr] \in \mathbb{R}^3, \tag{6.7}$$

using signed values; an empty routed history is $[0, 0, 0]$. H3 is standardized on training pairs only.

LEG$_{K2}$ residual. The same Top-K association coefficients selected for H3 are additionally summarized using the second Legendre polynomial,

$$L_2(u, X) = \frac{1}{K} \sum_{k=1}^{K} P_2(v_k), \qquad P_2(v) = \frac{3v^2 - 1}{2}. \tag{6.8}$$

While H3 preserves signed first-order summaries of the selected associations, L_2 provides a complementary nonlinear summary of their magnitude. Because P_2 is an even function, strong positive and negative associations both contribute through their squared magnitude, allowing the residual branch to capture structure that is not represented by the signed mean or maximum alone.

Higher-order Legendre terms were also considered during model development. The third-order term P_3 retains sign and therefore overlaps more strongly with information already represented by the signed H3 statistics, whereas the fourth-order term P_4 places greater emphasis on extreme coefficients and can be more sensitive to variation in the relatively small routed set ($K \leq 25$). The second-order term was therefore retained as the most compact complementary summary of the routed association context:

$$\text{Top25} \longrightarrow \begin{cases} \text{H3}: & \text{signed summary,} \\ L_2: & \text{nonlinear magnitude summary.} \end{cases}$$

The scalar $L_2(u, X)$ is standardized using training pairs and passed through a small residual MLP producing $\delta_{\text{LEG}}(u, X)$. The final linear layer of this branch is initialized to zero, so the residual contribution is initially neutral and is learned only when supported by the training signal.

6.3.4 Fusion and Prediction

The main late-fusion input is

$$g_{uX} = \Bigl[q_{uX} \parallel d_{uX} \parallel A(u, X) \parallel H_3(u, X)\Bigr] \in \mathbb{R}^{265}. \tag{6.9}$$

A decoder $265 \rightarrow 128 \rightarrow 64 \rightarrow 1$ (LayerNorm, GELU, dropout) produces $s_{B0}(u, X)$. LEG$_{K2}$ is not concatenated into g_{uX}; it enters only as an additive residual. The final ranking score is

$$s(u, X) = s_{B0}(u, X) + \delta_{\text{LEG}}(u, X). \tag{6.10}$$

6.4 Training and Evaluation

The Last-FM* experiments use three evaluation levels corresponding to different stages of model development. Sampled validation is used for checkpoint selection, full-catalog validation for architectural ablation, and the sealed external benchmark for the final comparison with classical recommenders. Although NDCG@20 is used in more than one stage, the corresponding values are interpreted within their respective candidate universes and experimental roles.

6.4.1 Training Objective

For each training pair with ranking score s_i and label $y_i \in \{0, 1\}$,

$$\hat{p}_i = \sigma(s_i), \tag{6.11}$$

is used only inside weighted binary cross-entropy ($w_+ \approx 4$ under the 4:1 candidate construction). The quantity $s(u, X)$ itself is a ranking score, not a calibrated interaction probability. Optimization uses Adam (learning rate 10^{-3}, weight decay 10^{-4}), ReduceLROnPlateau on sampled validation NDCG@20, batch size 4096, and early stopping with patience 8 on the same sampled metric. Figure 6.4 shows a representative development learning curve for seed 303, with the selected checkpoint marked. Architecture and per-seed epochs were frozen under this sampled rule before any sealed-test evaluation; full-catalog development metrics and the sealed external benchmark are computed only after freeze and are never used to re-select the epoch.

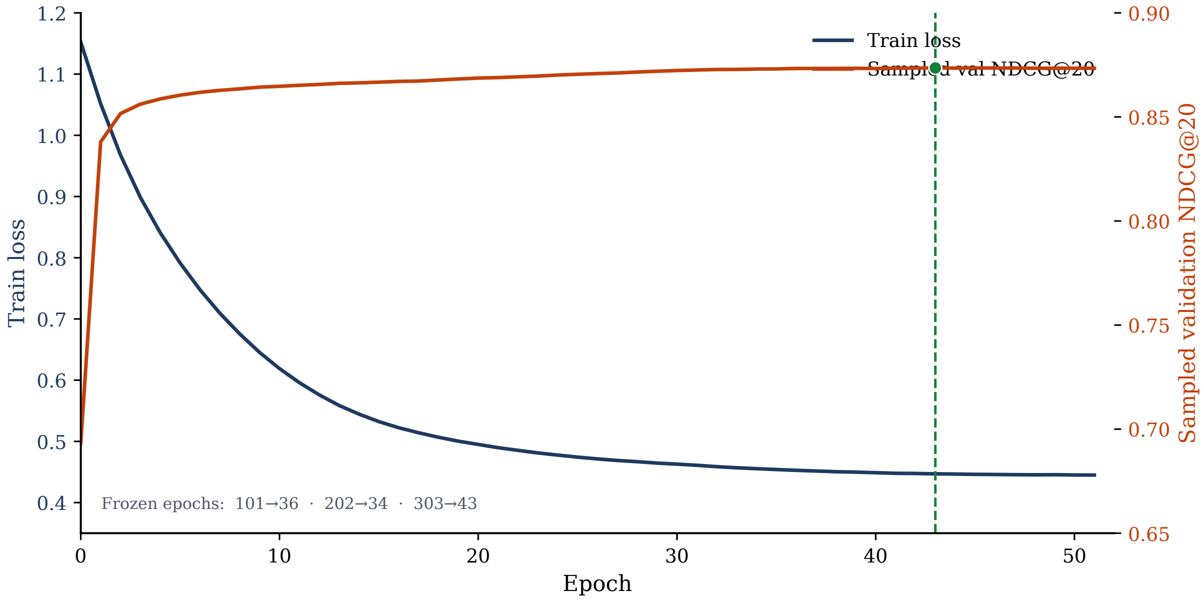

Figure 6.4: Development training dynamics for seed 303 of the full Last-FM* model: training loss and sampled validation NDCG@20 versus epoch. The dashed line marks the selected checkpoint (epoch 43). Seeds 101 and 202 were frozen under the same sampled rule at epochs 36 and 34.

6.4.2 Checkpoint selection with sampled NDCG@20

Sampled validation NDCG@20 was used for checkpoint selection and early stopping during model development. For each validation positive, the candidate set consisted of the relevant item and 20 sampled negatives not present in the user's observed history. To limit the contribution of highly active users, at most 10 validation positives per user were included.

This sampled protocol provides a computationally efficient ranking signal during training. It is used exclusively for model selection and is not interpreted as catalog-scale recommendation performance.

6.4.3 Development full-catalog evaluation

After checkpoint selection, architectural variants were evaluated against the full eligible catalog. For user u, the candidate set was defined as

$$\mathcal{C}_u = I \setminus H_u^{(\text{model\_train})}, \tag{6.12}$$

where I denotes the item catalog and $H_u^{(\text{model\_train})}$ contains items already observed for that user in the model-training data.

Performance was reported using NDCG@20, Recall@20, and MRR. This evaluation was used to quantify the contribution of the candidate-specific empirical components—A5, H3, and the LEG_{K2} residual—relative to the structural HGT baseline under catalog-scale ranking.

6.4.4 Sealed external benchmark

The final external benchmark was conducted only after the model specification and per-seed training schedule had been fixed on development data. The final models were refitted on the complete corrected training population,

$$\mathcal{D}_{\text{train}}^{\text{external}} = \mathcal{D}_{\text{model\_train}} \cup \mathcal{D}_{\text{validation}}, \tag{6.13}$$

and evaluated on the previously unseen corrected Last-FM* test split using the pinned IntentAwareRS holdout evaluator.

The proposed model and the classical recommendation baselines were evaluated under the same full-catalog candidate-space rules and reported using NDCG@20, Recall@20, and MRR. This stage therefore provides the matched external comparison used for the final benchmark results.

6.4.5 Computational note

Full-catalog neural evaluation on Last-FM* was computationally demanding on the available local hardware because scores had to be produced for a catalog of 48,123 items for each eligible user. This constrained the breadth of additional convergence and hyperparameter studies. The external comparison therefore follows the prespecified model specification rather than an extensive post-hoc search over training budgets.

6.5 Experimental Results

6.5.1 Architecture Comparison

HGT width, depth and number of attention heads were compared under a common development protocol with the decoder family held fixed. The selected configuration is $d = 64$, $L = 2$, $H = 2$ with dropout 0.1; larger width, deeper stacks or additional heads did not justify replacement. Table 6.3 summarizes the frozen training and model settings of the full Last-FM* specification used in all subsequent development and sealed evaluations.

Table 6.3: Last-FM*: model and training configuration.

Setting	Value
Optimizer	Adam [25]
Learning rate	10^{-3}
Weight decay	10^{-4}
LR schedule	ReduceLROnPlateau on sampled validation NDCG@20; factor 0.5, patience 3, min_lr = 6.25×10^{-5}
Batch size	4096
Max epochs	120
Early stopping	patience 8 on sampled validation NDCG@20
Loss	weighted binary cross-entropy ($w_+ \approx 4$; 4:1 candidate construction)
HGT	$d = 64$, $L = 2$, $H = 2$, dropout 0.1 [21]
Structural pair	$q_{uX} \in \mathbb{R}^{256}$ plus graph dot $\Rightarrow$ 257D
Explicit context	A5 (5D), H3 (3D), LEG$_{K2}$ residual
Main decoder	$265 \to 128 \to 64 \to 1$ (LayerNorm, GELU, dropout 0.2)
LEG$_{K2}$ branch	$1 \to 16 \to 1$ (GELU; final linear zero-initialized)
Training seeds	$\{101, 202, 303\}$
Selection metric	sampled validation NDCG@20
Frozen epochs	seed 101 $\to$ 36, seed 202 $\to$ 34, seed 303 $\to$ 43

6.5.2 Contribution of Statistical Evidence

With the HGT backbone fixed, a controlled ablation ladder isolates explicit candidate context (Table 6.4, Figure 6.5), using seeds $\{101, 202, 303\}$:

1. HGT only (257-dimensional structural input);
2. HGT + A5;
3. HGT + A5 + H3;
4. full model (A5 + H3 plus the LEG$_{K2}$ residual).

The controlled ablation demonstrates that explicit candidate-specific empirical context substantially improves the neural graph model relative to its HGT-only counterpart. A5 introduces the first large improvement on both development protocols. H3 produces the largest full-catalog increase. LEG$_{K2}$ changes sampled NDCG@20 by only about 0.0004 relative to HGT+A5+H3, while adding a further full-catalog gain of about 0.044 NDCG@20. The development ablation addresses the contribution of the proposed representation within the neural model family. It does not by itself establish superiority over strong collaborative-filtering baselines.

Table 6.4: Last-FM*: development ablation results. Sampled NDCG@20 is the checkpoint-selection metric; full-catalog NDCG@20 and Recall@20 evaluate the same checkpoints on the eligible validation catalog. These are development validation results and are used to isolate the contribution of explicit candidate-specific context. They are not the final external benchmark.

Variant	Main decoder	Sampled NDCG@20	Full-cat. NDCG@20	Full-cat. Recall@20
HGT only	257D	0.4755	0.0093	0.0169
HGT + A5	262D	0.7908	0.0640	0.1128
HGT + A5 + H3	265D	0.8729	0.2321	0.3647
Full model (+ LEG$_{K2}$)	265D + residual	0.8733	0.2764	0.3768

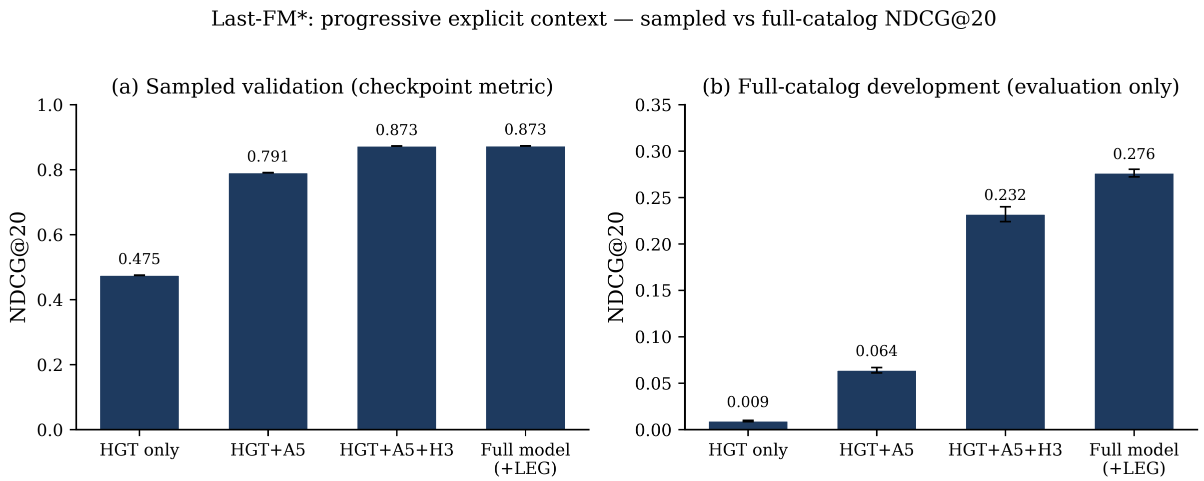

Figure 6.5: Last-FM* development ablation ladder. Panel (a): sampled validation NDCG@20 used for checkpoint selection. Panel (b): full-catalog development NDCG@20 of the same checkpoints. Error bars show the seed standard deviation over three training seeds. The panels use different y-axis scales because they evaluate different candidate universes under the same NDCG@20 formula. This figure is not an external baseline comparison.

6.5.3 Full-catalog development ranking and classical baseline

Epochs were selected using sampled validation NDCG@20; full-catalog development metrics were then computed without re-selection (Table 6.5).

Table 6.5: Full-model full-catalog development metrics by seed (model train $\rightarrow$ validation). Sampled NDCG@20 selected the epoch; full-catalog metrics are evaluation only.

Seed	Best epoch	Sampled NDCG@20	Full-cat. NDCG@20	Recall@20	MRR
101	36	0.873224	0.277432	0.375599	0.356988
202	34	0.873253	0.279879	0.378722	0.359681
303	43	0.873515	0.271886	0.376197	0.343391
Mean $\pm$ SD	—	0.8733 ± 0.0002	0.2764 ± 0.0041	0.3768	0.3534

Table 6.6: Full-catalog development NDCG@20 by seed for the ablation layers without LEG_{K2}.

Seed	HGT only	HGT+A5	HGT+A5+H3
101	0.008548	0.064571	0.227969
202	0.009520	0.060866	0.241332
303	0.009935	0.066630	0.227054
Mean	0.0093	0.0640	0.2321

For the full model, mean full-catalog development NDCG values were additionally NDCG@5 = 0.2336 and NDCG@10 = 0.2503; HitRate@20 was approximately 0.664.

Under the same full-catalog development protocol, a matched RP3β reference already reached NDCG@20 = 0.3227, Recall@20 = 0.4122 and MRR = 0.4183, exceeding the proposed full model (NDCG@20 = 0.2764). Importantly, RP3β already outperformed the proposed model under full-catalog development evaluation. The sealed external benchmark therefore evaluates whether this qualitative ordering persists on the previously unseen corrected test split.

6.5.4 Sealed external benchmark

Table 6.7 reports classical baselines on the sealed corrected test under the pinned holdout evaluator after fitting on TRAIN_EXTERNAL. RP3β is the strongest classical baseline (NDCG@20 = 0.2318, Recall@20 = 0.2588).

Table 6.8 reports the proposed model under the same sealed protocol. All three prespecified seeds are complete; the mean $\pm$ SD is computed over seeds 101, 202 and 303.

Figure 6.6 compares classical baselines and the proposed mean under the same sealed protocol.

On the sealed external test, RP3β was the strongest classical baseline, reaching NDCG@20 = 0.2318 and Recall@20 = 0.2588. The proposed model reached mean NDCG@20 = 0.1718 ± 0.0040 and Recall@20 = 0.2139 ± 0.0024 across seeds 101, 202 and 303 (per-seed NDCG@20: 0.1685, 0.1762, 0.1706). Seed 202 is the strongest of the three, but all three seeds remain below RP3β, ItemKNN and P3α on NDCG@20, which is consistent with the development ordering in which RP3β already exceeded the proposed

Table 6.7: Last-FM*: sealed external benchmark results for classical recommenders fitted on TRAIN_EXTERNAL and evaluated once on the corrected Last-FM* test with the pinned holdout evaluator. Primary comparison metrics are NDCG@20 and Recall@20.

Model	NDCG@20	Recall@20
TopPop	0.0091	0.0153
UserKNN	0.1622	0.1819
ItemKNN	0.2051	0.2369
P3α	0.2035	0.2385
RP3β	**0.2318**	**0.2588**

Table 6.8: Last-FM*: sealed external benchmark results for the proposed full model under the frozen epoch schedule. Mean ± SD is over the three prespecified training seeds.

Seed	Frozen epoch	NDCG@20	Recall@20
101	36	0.1685	0.2123
202	34	0.1762	0.2167
303	43	0.1706	0.2127
Mean ± SD	—	0.1718 ± 0.0040	0.2139 ± 0.0024

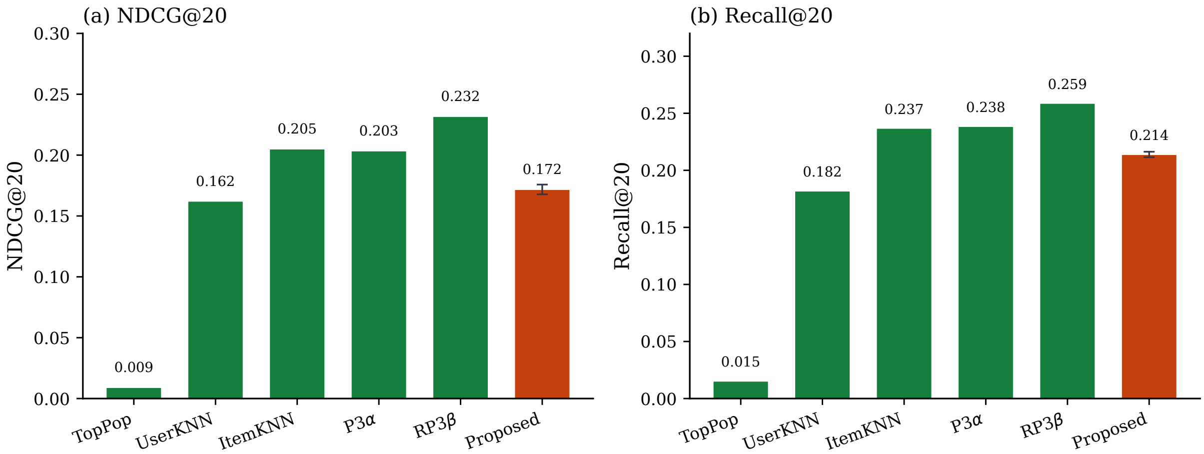

Figure 6.6: Sealed external Last-FM* benchmark. NDCG@20 and Recall@20 for classical recommendation baselines and the proposed model under the same full-catalog holdout evaluation protocol. The proposed-model bars report the mean across the three prespecified training seeds, with error bars indicating one standard deviation.

model. The external benchmark highlights an important distinction between improving a neural graph model and achieving benchmark dominance: explicit candidate-specific statistical context provides substantial ranking signal beyond HGT alone, but strong classical collaborative-filtering methods remain competitive and, in this benchmark, can outperform the more complex neural architecture. Full-catalog development and sealed external numbers must not be subtracted as if they were the same experiment.

Chapter 7

Task B. Patient-Level Endpoint Prediction

7.1 Problem Formulation

Task B is formulated as multi-label node classification on a heterogeneous, per-patient graph, in order to predict clinical outcomes. Instead of building one global graph and treating patients as additional nodes, every patient p is represented as a copy of the same fixed SPG v3, with node feature vectors derived from that patient's covariates.

Formally, patient p is represented by the attributed graph $G_p = (V, E, x_p)$, where x_p denotes the patient-specific node features. The topology (V, E) is shared across the cohort. The node set V comprises six semantic categories:

- *patient_context* — baseline characteristics such as age, sex, or alcohol use;
- *drug_exposure* — medications taken, such as anticoagulants, diuretics, or opioids;
- *mechanism* — biological or pharmacodynamic processes such as slowed drug breakdown, sodium–water imbalance, or poor kidney blood flow;
- *adr_or_intermediate_state* — visible medical states such as a rise in creatinine, ECG-measured QT prolongation, or bleeding;
- *observation_or_selection* — testing, documentation, and care-selection variables such as laboratory monitoring or recorded clinical checks;
- *clinical_endpoint* — clinical outcomes being predicted, described in section 7.2.1.

These node types encode a causal pathway towards a clinical outcome: signals propagate along the edges of SPG v3 from context and drug exposure, through mechanisms and visible medical states, to the endpoints. In general, heart failure (*patient_context*) together with an NSAID (*drug_exposure*) may reduce kidney blood flow (*mechanism*), become visible as a rise in creatinine (*adr_or_intermediate_state*), and culminate in AKI (*clinical_endpoint*); laboratory monitoring of creatinine is an *observation_or_selection* node on the same pathway. Which of those nodes are actually on for a given patient is encoded in x_p. Figure 7.1 shows one such realisation ending in AKI: filled nodes are this patient's active context, drugs and mechanisms, whereas outlined nodes (for example NSAID, heart failure, or a creatinine rise) remain inactive mediators on the shared schema. The aim of Task B is to estimate endpoint risk from the entire patient-specific graph as described above.

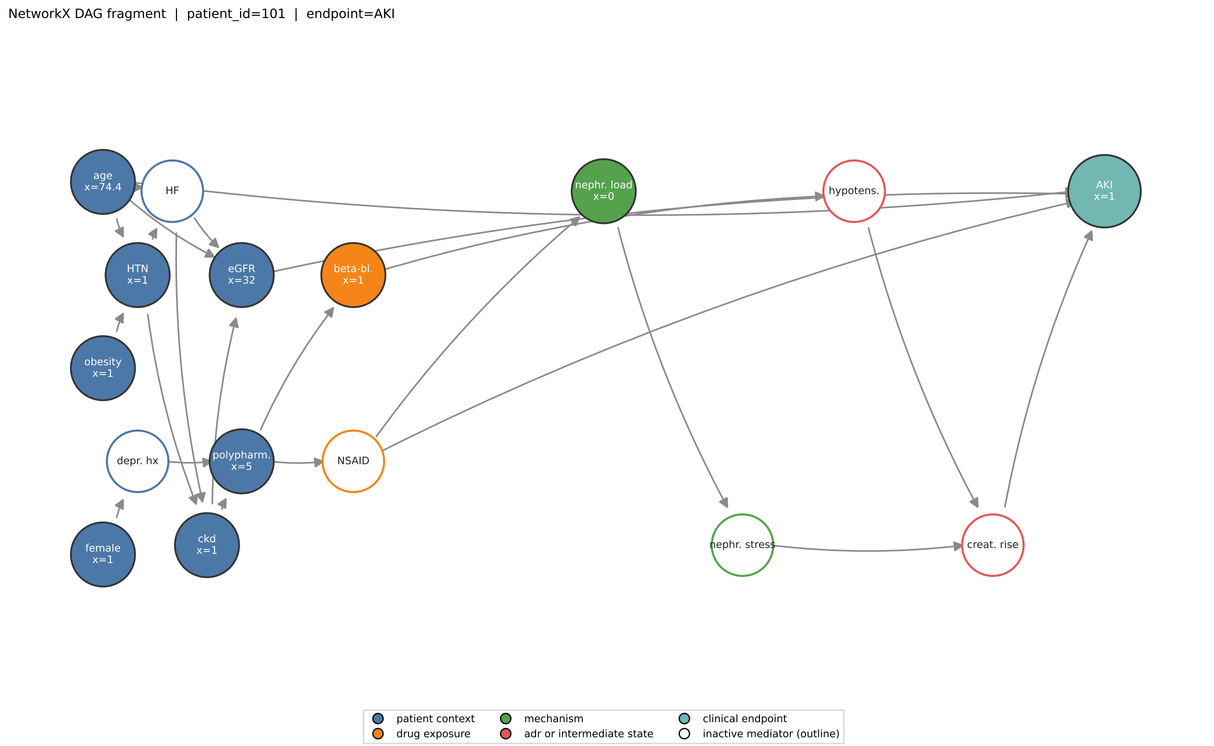

Figure 7.1: Exemplary per-patient causal DAG for one synthetic patient (id 101). The full SPG v3 is shared across the cohort; this figure keeps only the AKI-relevant fragment so that the left-to-right causal layers remain readable. Filled nodes are active for this patient; outlined nodes are inactive mediators on the same paths.

Assuming that $\mathcal{V}_{\text{target}} \subset V$ is the set of endpoint nodes used as prediction targets, for each patient p and target node $v \in \mathcal{V}_{\text{target}}$ the binary label $y_{p,v}$ indicates whether the endpoint occurs. A shared heterogeneous GNN f_θ produces a separate probability for each endpoint from the same encoded graph; v is a concrete clinical endpoint such as QT arrhythmia, Falls, Serotonin syndrome, or AKI:

$$\hat{y}_{p,v} = \sigma\Big(f_\theta(G_p)_v\Big), \quad v \in \mathcal{V}_{\text{target}}, \tag{7.1}$$

G_p=(V,E,x_p), f_θ — heterogeneous GNN shared across patients, v — clinical endpoint node

7.2 Patient-Level Graph Representation

The input graph for the node classification task is built from:

a) **nodes**

b) **edges**

c) **per-scenario patient sample table**

These components are assembled into a heterogeneous per-patient graph representation whose elements are summarised in table 7.1.

Table 7.1: Components of the per-patient heterogeneous graph representation.

Component	Description
Node types	Six semantic categories of nodes shared across all patient graphs: `patient_context`, `drug_exposure`, `mechanism`, `adr_or_intermediate_state`, `observation_or_selection`, and `clinical_endpoint`.
Relations	Keyed by `(src_type, "to", dst_type)`; `edge_type` is encoded as a one-hot block inside `edge_attr`. This keeps the number of distinct convolutional relation modules in `HeteroConv` small while still letting edge-type information reach the message function.
Edge attributes (`edge_attr`)	Fixed-length vector per edge (see table 7.2): signed effect `effect_size`×`effect_sign`, audit scalars `activation_frequency`, `mechanistic_confidence`, `evidence_weight`, and a one-hot `edge_type` block.
Node features (`x`)	The patient's observed values for a node

Patient observations are encoded as node features. The AKI fragment in figure 7.1 is the running example of Section 7.1: filled nodes follow this patient's x_p, outlined nodes are inactive mediators on the same directed paths.

Two structural exclusion mechanisms are applied before any feature is computed:

a) nodes flagged `exclude_from_observed_model` or `is_latent` (hidden confounders);

b) nodes that are structural descendants of any endpoint in the DAG (e.g. documentation nodes such as `hospital_contact`, `fall_reported`, which are caused by the endpoint rather than causal parents of it)

All normalization statistics (mean/std for continuous node values) and all statistical evidence features (section 7.4) are computed exclusively on the training split of patients, to prevent both node-level and patient-level information leakage across splits.

7.2.1 Prediction targets

Although the knowledge graph defines 13 clinical outcomes nodes, the model is trained and evaluated on a subset of 11 clinical endpoints, excluding `Hospitalization` and `Falls`. They are retained in the per-patient graph topology and in patient labels, but are excluded from the multi-target prediction head.

This choice follows both the graph structure and the clinical scope of the task. `Hospitalization` was excluded because the DAG models it as a downstream consequence of other endpoints (`endpoint_to_hospitalization`), which makes it a poor fit for parallel multi-target prediction alongside those same outcomes. `Falls`, by contrast, is a geriatric outcome that has different causal pathways than other endpoints that originate from exposures and mechanistic intermediates in the pharmacotherapy graph. On these grounds, Falls and Hospitalization are treated as downstream, multifactorial events outside that primary prediction scope.

For reporting, all eleven targets are trained, but macro-averaged metrics (AUROC, AP, % of oracle AUROC; equation (7.4)) are computed on eight endpoints only. `Serotonin_syndrome`, `Rhabdomyolysis`, and `Lactic_acidosis` are omitted from the aggregate because their extreme rare prevalence makes split-level AUROC and AP unstable. The resulting reporting hierarchy is therefore 13 graph endpoints → 11 trained targets → 8 endpoints included in macro metrics.

7.3 Model Architecture

The core architecture is a stack of `HeteroConv` layers (`HeteroConv` layers (a wrapper applying a homogeneous message-passing operator per relation type; section 3.1.1) applied to the SGP v3. Each layer instantiates a separate message-passing operator per relation type; the same backbone weights are shared across all patient graphs in the cohort. Three operators were benchmarked against each other: GraphSAGE, R-GCN, and TransformerConv. All three are applied to SGP v3 via `HeteroConv`. Each directed edge in the graph carries a fixed `edge_attr` vector assembled at graph-build phase from the audited edge table (table 7.2). In model design relations are grouped by node-type pairs `(src_type, to, dst_type)` and the semantic `edge_type` is retained as a one-hot suffix in `edge_attr`. This vector is consumed differently depending on the message-passing operator (table 7.3).

Table 7.2: Layout of the Task B `edge_attr` vector.

Block	Columns	Source
Signed effect	1	`effect_size` × `effect_sign`
Audit scalars	3	`activation_frequency`, `mechanistic_confidence`, `evidence_weight`
Edge-type one-hot	|`edge_type`|	semantic edge category from the audited DAG

Table 7.3: Use of `edge_attr` by convolution backbone in Task B.

Backbone	Edge features used in message passing
GraphSAGE	none (topology only; `edge_attr` stored but not passed to `SAGEConv`)
TransformerConv	full `edge_attr` via `edge_dim`
R-GCN	relation identity via separate modules per edge type, not via the Task B `edge_attr` vector

A design issue handled during the experiments is the duplication of the self-transform across relations. Several PyG message-passing layers (SAGEConv, TransformerConv) internally add their own linear self-loop term, wrapped per-relation inside `HeteroConv`. This contribution of the node's own representation is duplicated once per incoming relation type. The implementation applied removes this internal duplication (internal self-loop weights disabled for SAGE/Transformer) and replaces it with a single, explicit, controlled self-transform per layer per node type.

The architecture begins with the linear layer that maps each node's raw features into a hidden vector of fixed width (the input projection). Several clinical endpoints in SPG v3—serotonin syndrome, rhabdomyolysis, and lactic acidosis—share identical audited attributes and similar neighbourhoods, so a message-passing GNN cannot recognise them apart. This limitation is the one-dimensional Weisfeiler–Lehman colouring: nodes with equal features and equivalent neighbourhoods receive the same representation [37]. To break the symmetry, a learnable node-identity embedding x' is added after the input projection and before the first message-passing layer:

$$x' = Wx + e_{\text{type}}[\text{index}], \tag{7.2}$$

x — raw node features, W — input projection, $e_{\text{type}}[\text{index}]$ — identity vector for that node in the shared DAG

For each node type, a table of vectors is indexed by the node's fixed local index $e_{\text{type}}[\text{index}]$ in SPG v3 and the looked-up vector is added to the projected features so that similar endpoints remain distinguishable even when their static attributes coincide. The sequence is presented in figure 7.2.

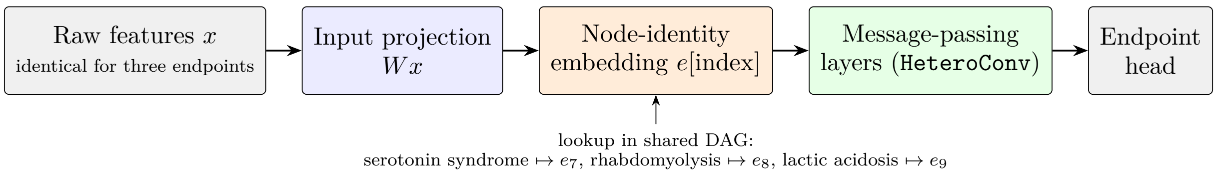

Figure 7.2: Order of operations in the Task B encoder.

A separate issue at greater depth is oversmoothing. When several message-passing layers are stacked, repeated aggregation can make node representations increasingly similar [29]. Residual connections, Jumping Knowledge, DropEdge, and PairNorm were tested as countermeasures; residual connections were the most effective and are used in the results reported below.

7.4 Statistical Evidence Branch

In addition to the deep graph representation learned by the GNN, endpoint prediction includes a statistical evidence branch: a wide side-channel that supplies cohort-level pairwise dependence statistics. The construction is *HCR-inspired* (section 3.2): some parent–endpoint dependencies are well captured by interpretable statistics computed on the training cohort, and combining them with the GNN representation keeps a direct-parent signal that message passing can dilute.

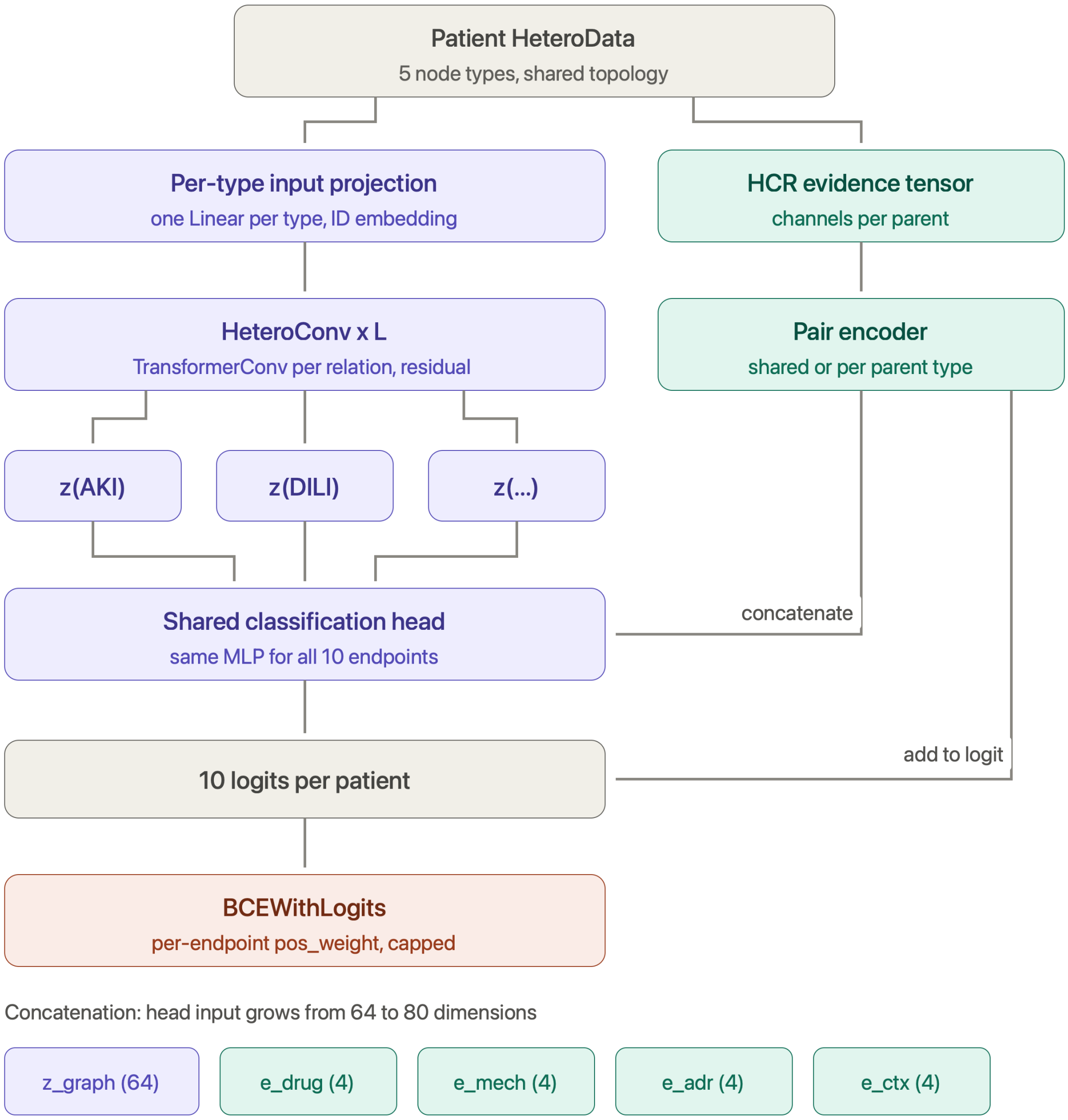

Figure 7.3: Task B architecture: heterogeneous GNN path with a shared endpoint head, HCR-inspired evidence tensors with pair encoders, and optional concatenation or logit-level fusion.

Figure 7.3 summarises the dual-path layout. The GNN produces an endpoint embedding z_{graph}. In the nonlinear and typed configurations, a pair encoder (a small MLP) maps each parent–endpoint evidence vector to an embedding and mean-pools over parents. There may be a single encoder for all parents, or one encoder per parent type (drug, mechanism, ADR, patient context) – e_{drug}, e_{mech}, e_{adr} and e_{ctx} presented at the bottom of the figure. The resulting embeddings are either concatenated with z_{graph} as input to the classification head, or reduced to a scalar and added to the GNN logit. The different fusion modes tested are specified below (table 7.4) and subsections below represents detailed info about configurations.

7.4.1 Scope of variables and parents

For each target endpoint, evidence is computed over its *direct parents* in the patient graph (subject to the structural exclusions described in section 7.2). Pairwise statistics relate one parent to one endpoint at a single hop. Higher-order evidence over *grandparent–parent–endpoint* paths ($G \to P \to Y$, where G, P, and Y denote the grandparent, parent, and endpoint nodes on SGP v3). is computed separately in the *triple* extension. It is stored in its own evidence tensor (13 channels per triple slot) and fused through a dedicated encoder block. Several evidence dimensionalities were evaluated, including binary-only parents (compact 10-channel evidence) and all parent value types (binary, count, and continuous; 42-channel evidence).

7.4.2 Pairwise HCR core coefficients

For a binary parent P and a binary endpoint Y, the primary association coefficient is the φ-coefficient derived from the training-cohort contingency table. For each patient p, the parent activation is the standardized binary score $\phi_1(R)$, where R denotes the parent. The HCR patient evidence for that parent–endpoint pair is the product $s_{p,rv} = a_{11,rv}\, \phi_1(R_p)$ (channel `s_evidence`). In *linear* mode the endpoint-wide score is the sum over eligible direct parents,

$$s_{p,v} = \sum_{r \in \mathrm{Pa}(v)} a_{11,rv}\, \phi_1(R_p),$$

and a single learned scalar per endpoint scales this score before it is added to the GNN logit.

7.4.3 Extended pairwise evidence channels

Beyond the HCR core, the same contingency table yields supplementary association measures that provide complementary views of parent–endpoint dependence for a learned encoder: normalized mutual information, a tanh-scaled log odds ratio, risk difference, tanh-scaled log lift, parent and endpoint prevalence, and support (the fraction of training patients with both variables observed). These are assembled into a fixed-order evidence vector per (patient, parent, endpoint) slot. Two pairwise dimensionalities were used:

- **10 channels**: compact binary-parent representation with explicit `s_evidence`;
- **42 channels**: enriched representation including matrix-energy summaries, marginal entropies, type indicators, and the patient activation channel.

7.4.4 Type-specific encoders and endpoint-specific fusion

In *nonlinear* and *triple* modes, a shared pairwise evidence encoder (PairEncoder in figure 7.3) processes each evidence slot with a two-layer MLP (`Linear`→`ReLU`→`Linear`), mean-pools over parent or triple slots using a padding mask, and collapses the pooled embedding to a scalar logit contribution. ReLU activation is used as in the graph encoder. The MLP widths (hidden 32, output 8) were selected after preliminary runs with a narrower encoder (hidden 8, output 4) performed worse; they were then held fixed (table 7.5). In *nonlinear* mode only, that contribution is further scaled by a learned per-endpoint weight.

In *typed_concat* (and *typed+triple*), the architecture differs in two respects. First, evidence is partitioned by *parent node type*: each of the four parent categories (`drug_exposure`, `mechanism`, `adr_or_intermediate_state`, `patient_context`) is encoded by its own pairwise evidence encoder applied to a separate evidence tensor (type-specific evidence tensors). These encoders are shared across endpoints; they differ only in the semantic role of the parents they read. Second, fusion is *endpoint-specific*: a learnable gate matrix $\in \mathbb{R}^{|\mathcal{V}_{\text{target}}| \times B}$ (with $B = 4$ parent-type blocks, or $B = 5$ when a triple block is enabled) scales each type-level embedding separately for every target endpoint before concatenation. The gated embeddings are concatenated with the GNN target node embedding and passed jointly to the classification head. In *typed+triple*, triple-path evidence is encoded by an additional triple-path encoder and included as a fifth block, subject to its own per-endpoint gate column in the endpoint-specific gate matrix.

7.4.5 Non-binary parents

Count- and continuous-valued direct parents are handled outside the binary φ-coefficient framework. Values are mapped to comparable pseudo-uniform scores via a train-only empirical CDF transform; count variables receive deterministic, patient-ID-seeded jitter before transformation. Non-linear dependence on continuous parents is represented through a Legendre polynomial basis. These channels extend the binary *patient activation* in the 42-dimensional branch.

7.4.6 Higher-order triple-path evidence

To capture higher-order dependencies, third-order moments are computed over graph-consistent triples $G \to P \to Y$. The main coefficient a_{111} is estimated from a $2 \times 2 \times 2$ contingency table over three binary nodes on each DAG-consistent path $G \to P \to Y$ (grandparent, parent, and endpoint). It generalizes the pairwise φ-coefficient, which uses only two variables at a time. The triple vector also retains the three pairwise marginals ($a_{11}^{GP}, a_{11}^{GY}, a_{11}^{PY}$), prevalence terms, support, patient activations, and an explicit patient evidence product $a_{111}\,\phi_G\,\phi_P$. Triple evidence uses the same slot-based encoder interface as pairwise evidence.

The fusion modes connect the statistical branch to the GNN; they differ in what evidence tensor is built and how it is combined with the deep representation (table 7.4).

Table 7.4: Fusion modes for the statistical evidence branch in Task B.

Mode	**Evidence tensor**	**Encoder**	**Fusion with GNN**
`linear`	Scalar HCR score $s_{p,v} = \sum_r a_{11,rv}\,\phi_1(R_p)$ per endpoint	None (precomputed wide scores)	Added to endpoint logit; one learned scalar per endpoint (learned wide-branch weight)
`nonlinear`	Pairwise 10D or 42D vectors per (patient, parent, endpoint) slot	Shared `HCRPairEncoder`; mean-pool over parent slots	Encoder output added to logit; optional per-endpoint scale (learned per-endpoint scale)
`typed_concat`	42D pairwise vectors, split into four type-specific tensors (`drug_exposure`, `mechanism`, `adr_or_intermediate_state`, `patient_context`)	One pairwise evidence encoder per parent *type* (shared across endpoints); endpoint-specific gate matrix $[\lvert\mathcal{V}_{\text{target}}\rvert \times 4]$	Gated type embeddings concatenated with GNN target embedding *before* the classification head
`triple`	13-channel vectors per (patient, triple, endpoint) slot along paths $G \to P \to Y$	Shared pairwise evidence encoder on triple slots (replaces pairwise evidence in this mode)	Encoder output added to logit (no per-endpoint scalar)
typed+triple	Typed 42D pairwise blocks *and* 13-channel triple block (with triple-path evidence enabled)	Four typed encoders plus triple-path encoder; gates $[\lvert\mathcal{V}_{\text{target}}\rvert \times 5]$ (one column per block, including triple)	All gated blocks concatenated with GNN embedding before the head (same join as `typed_concat`)

7.5 Training and Evaluation

7.5.1 Training Objective

Training iterates over patient graphs batched via PyG DataLoader. Models were optimized using class-weighted binary cross-entropy with logits. Endpoint weights were computed from the training-split prevalence, since clinical outcomes are rare and highly imbalanced. Model selection uses ReduceLROnPlateau on validation performance. Exploratory runs with focal loss were not retained in the final reported protocol.

7.5.2 Evaluation Metrics

Evaluation is reported separately per endpoint and in aggregate, using AUROC, AP, and prevalence-normalized lift. As the synthetic dataset generated for this project contains rare events and class imbalance, AP was chosen as a key performance metric. It focuses on the positive class and reflects the trade-off between precision and recall. The evaluation metrics are defined as follows.

AUROC is the area under the receiver operating characteristic curve and is defined as:

$$\mathrm{AUROC} = \int_0^1 \mathrm{TPR}(\mathrm{FPR})\, d\mathrm{FPR}, \tag{7.3}$$

where TPR is the true positive rate and FPR is the false positive rate.

AP is defined analogously like in Task A (section 5.4.2).

Lift is the AP divided by the prevalence baseline, that is, the factor by which the model improves over a random ranking, e.g. at a mean prevalence of 5.5% a lift of 2.3 corresponds to an AP of 0.125.

In addition to the traditional metrics described above, a *generator-based oracle AUROC* was computed for the node-classification task. As the dataset is synthetic, the logistic generative model for each endpoint is known: each patient is scored with the true parent effects from the audited DAG and AUROC is evaluated on the resulting probabilities. Model performance is reported relative to this reference via *% of oracle AUROC*:

$$\frac{\mathrm{AUROC} - 0.5}{\mathrm{AUROC}_{\mathrm{Oracle}} - 0.5} \cdot 100\%, \tag{7.4}$$

where $\mathrm{AUROC}_{\mathrm{Oracle}} = 0.651$ is the macro-averaged oracle AUROC over the same eight endpoints used in aggregate reporting (table B.1). This metric expresses the share of ranking signal recoverable under the known generator, relative to random performance at 0.5.

7.5.3 Model and Training Configuration

The experimental configurations share a common training protocol. All experiments share the same encoder structure and differ only in a small number of hyperparameters: the number of layers L, the presence of a residual connection and the statistical-evidence variant. Everything else was held fixed. The shared configuration is listed in table 7.5, and the parameters specific to individual convolutions in table 7.6.

Table 7.5: Task B: shared model and training configuration. Values were fixed across the experiments reported in this chapter.

Group	Hyperparameter	Value
Encoder	Hidden units per convolutional layer	64
	Number of layers L	1, 2, 3, 4, 5, 6
	Aggregation across relations	sum
	Activation	ReLU
	Batch normalisation	yes
Pair encoder	Hidden units	32
	Output embedding	8
	Dropout	0.0
Regularisation	Dropout	0.2
	Weight decay	5×10^{-4}
Classification head	Number of layers	2
	Hidden units	64 (equal to encoder width)
	Output units	1 per endpoint
Optimisation	Optimiser	Adam
	Learning rate	1×10^{-3}
	Batch size	128
	Epochs (maximum)	200
	Loss	weighted BCE with logits
	Positive weight cap	30
	Gradient clipping	disabled
	Seed	42
Model selection	Selection metric	validation AP
	Early-stopping patience	20 epochs
	Minimum improvement	0.001

Table 7.6: Task B: architecture-specific configuration.

Hyperparameter	R-GCN	GraphSAGE	TransformerConv
Attention heads	—	—	4
Units per head	—	—	16
Output width per relation	64	64	$4 \times 16 = 64$
Basis decomposition	8 bases	—	—
Uses edge attributes	relation id only	none (stored, unused)	full `edge_attr`
Weight matrices per relation	shared through bases	2	3 (query, key, value)

All three convolutions are wrapped in `HeteroConv`, which instantiates a separate convolution for each of the 19 relation types in the graph. The parameter counts therefore scale with the number of relations. In attention-based layers the hidden width is split across heads, so the output width stays at 64 regardless of the number of heads.

The entries in tables 7.5 and 7.6 were held fixed so that the reported comparisons isolate depth, residual connections, and the statistical-evidence variant. Adam with learning rate 10^{-3} follows the standard setting of Kingma and Ba [25]; ReLU, batch normalisation, and the 64-dimensional width are the same conventions as in the graph encoder, so that R-GCN, GraphSAGE, and TransformerConv differ only in the message-passing operator. `HeteroConv` uses sum aggregation across relation types after preliminary checks in which mean and max aggregation did not change the conclusions. The pair-encoder widths are those selected in section 7.4. Batch size 128 was used as a stable setting for patient-graph batches; the remaining optimisation details (including the choice of Adam) were not ablated.

7.6 Experimental Results

7.6.1 Architecture Comparison

The first set of experiments established how much the choice of GNN architecture matters for this task. Three message-passing layers were compared: R-GCN, GraphSAGE and TransformerConv. All three are defined for homogeneous graphs, so each was wrapped in `HeteroConv`, which instantiates a separate embedding for every relation type and aggregates the results at the target node and allows to preserve the heterogenous structure.

The results for those experiments are reported in table 7.7. All of metrics reported are described in section 7.3. The epoch is selected on the validation set and test metrics are reported from that epoch.

Table 7.7: Task B: message-passing architecture comparison without residual connections.

		Validation				Test			
Architecture	L	AUROC	AP	Lift	% oracle	AUROC	AP	Lift	% oracle
R-GCN	1	0.623	0.124	2.28	81.6	0.624	0.120	2.21	82.0
GraphSAGE	1	0.622	0.124	2.28	80.9	0.619	0.120	2.21	78.9
TransformerConv	1	0.622	0.125	2.30	81.2	0.620	0.120	2.20	79.6
R-GCN	3	0.542	0.076	1.40	28.0	0.542	0.068	1.26	27.7
GraphSAGE	3	0.549	0.079	1.45	32.2	0.549	0.072	1.32	32.8
TransformerConv	3	0.550	0.083	1.53	33.2	0.552	0.073	1.35	34.5
R-GCN	6	0.503	0.055	1.00	2.2	0.500	0.054	1.00	0.0
GraphSAGE	6	0.530	0.065	1.20	20.0	0.510	0.061	1.12	6.6
TransformerConv	6	0.527	0.065	1.20	18.0	0.500	0.054	1.00	0.0

The experiments were conducted with multiple number of layers as patient path is up to 6 degrees and it is assumed that every point can have an influence in casual path. For each experiment all number of layers were calculated, but for reporting reasons in table 7.7 there are included layers 1,3 and 6 to show the trend with the increased number of layers. The reporting convention is that unless the results from other layers are significant for conclusion.

Without residual connections the three architectures diverge sharply at L=6, collapse to constant predictions (AUROC=0.50).

7.6.2 Effect of Residual Connections

That's why the next step was application of residual connections to see the capabilities of models with higher number of layers. The results are presented in table 7.8.

Table 7.8: Task B: message-passing architecture comparison with residual connections.

		Validation				Test			
Architecture	L	AUROC	AP	Lift	% oracle	AUROC	AP	Lift	% oracle
R-GCN	1	0.625	0.125	2.30	82.7	0.625	0.121	2.22	83.2
GraphSAGE	1	0.621	0.124	2.27	80.6	0.619	0.120	2.21	78.7
TransformerConv	1	0.623	0.126	2.31	81.3	0.619	0.120	2.21	79.0
R-GCN	3	0.630	0.133	2.44	86.0	0.627	0.125	2.29	84.2
GraphSAGE	3	0.623	0.132	2.42	81.4	0.616	0.124	2.27	77.0
TransformerConv	3	0.628	0.135	2.47	85.2	0.620	0.123	2.27	79.7
R-GCN	6	0.629	0.135	2.48	85.6	0.626	0.123	2.27	83.9
GraphSAGE	6	0.628	0.134	2.46	84.9	0.624	0.127	2.33	82.3
TransformerConv	6	0.627	0.134	2.46	84.2	0.620	0.122	2.24	79.4

With residual connections all three architectures converge to comparable performance (table 7.8): 77–86% of the oracle AUROC (equation (7.4)) on the test set, with differences of about 0.01 AUROC. The more informative factors for model assessment are AP and Lift, which show that the model performs nearly 2.3 times better than a random baseline. Since the architectures achieve comparable performance, the choice of architecture for the remaining experiments was therefore made on structural grounds. TransformerConv is the only one of the three backbones benchmarked here that consumes the full `edge_attr` vector in message passing (see table 7.3). Since edge-level audit evidence is important for graph representation, TransformerConv was selected for all subsequent experiments with the statistical evidence branch.

7.6.3 Contribution of Statistical Evidence

The statistical evidence branch was introduced to supply the model with empirical dependence signals about grandparent/parent–endpoint pairs that the message-passing path has to infer on its own (section 7.4). The configurations examined here vary along three axes, each addressing a separate question. The first is the form of the wide branch: a single learned coefficient per endpoint applied to a precomputed dependence score, against a small nonlinear encoder applied to each parent separately. The second is the scope of the evidence: binary parents only, or all node types, in which case continuous and count variables are expanded in a Legendre basis. The third is the order of the statistic: pairwise coefficients between a parent and an endpoint, or third-order moments over grandparent–parent–endpoint paths, which capture interactions that pairwise coefficients cannot represent. The relevant Abbreviations used in the result tables are given in table 7.9; the corresponding scores are reported in table 7.10.

Table 7.9: Notation used for the statistical-evidence variants in the result tables.

Label	Join point	Description
—	–	No statistical evidence branch
linear	sum at logit	Precomputed dependence score, all parent types; one learned scalar per endpoint, no encoder
nonlin. all 42D	sum at logit	42 evidence channels, all parent types; shared pair encoder
nonlin. bin. 10D	sum at logit	10 evidence channels, binary parents only; shared pair encoder
nonlin. bin. 42D	sum at logit	42 evidence channels, binary parents only; shared pair encoder
triple	sum at logit	13 channels of third-order moments over grandparent–parent–endpoint paths, replacing the pairwise evidence
typed+triple	concatenation	42 pairwise channels split across four non-shared encoders, one per parent type, each with a learned per-endpoint gate, plus a third-order block with its own per-endpoint gate column

Table 7.10: Task B: statistical-evidence variants without residual connections (TransformerConv backbone).

		Validation				Test			
L	Evidence variant	AUROC	AP	Lift	% oracle	AUROC	AP	Lift	% oracle
1	—	0.622	0.1251	2.30	81.19	0.620	0.1197	2.20	79.61
1	linear 42D	0.622	0.1247	2.29	81.10	0.617	0.1191	2.19	77.83
1	nonlin. all 42D	0.622	0.1243	2.28	81.17	0.618	0.1192	2.19	78.57
1	typed+triple	0.634	**0.1348**	2.48	**89.05**	0.626	0.1241	2.28	**83.46**
2	nonlin. all 42D	0.625	0.1242	2.28	82.77	0.615	0.1180	2.17	76.15
2	typed+triple	0.628	0.1328	2.44	84.91	0.621	0.1235	2.27	80.16
3	—	0.550	0.0833	1.53	33.19	0.552	0.0734	1.35	34.49
3	linear 42D	0.604	0.1173	2.15	69.28	0.612	0.1031	1.90	74.51
3	triple	0.602	0.1131	2.08	67.92	0.590	0.1021	1.88	59.81
3	nonlin. all 42D	0.621	0.1310	2.41	80.13	0.629	0.1197	2.20	85.63
3	typed+triple	0.625	**0.1362**	2.50	**82.90**	0.620	**0.1252**	2.30	79.54
4	nonlin. all 42D	0.616	0.1214	2.23	76.90	0.609	0.1168	2.15	72.30
4	typed+triple	0.631	0.1350	2.48	86.65	0.614	0.1237	2.27	75.53
5	nonlin. all 42D	0.617	0.1183	2.17	77.67	0.611	0.1132	2.08	73.37
5	typed+triple	0.627	0.1320	2.42	84.31	0.617	0.1225	2.25	77.58
6	—	0.527	0.0653	1.20	17.96	0.500	0.0544	1.00	0.00
6	linear 42D	0.616	0.1206	2.21	76.91	0.603	0.0955	1.76	68.22
6	triple	0.607	0.1154	2.12	70.71	0.579	0.0967	1.78	52.56
6	nonlin. all 42D	0.569	0.0955	1.75	45.53	0.500	0.0544	1.00	0.00
6	typed+triple	0.630	0.1326	2.43	86.53	0.618	0.1218	2.24	78.35

Without the statistical evidence branch the model degrades sharply with depth and predicts a constant at $L = 6$. Every statistical-evidence variant recovers a substantial part of the lost performance, and the gain grows with depth: +1.8, +42.5 and +74.5 percentage points of the oracle AUROC (equation (7.4)) at $L = 1$, 3 and 6 respectively. The concatenated variant is the only one that remains flat across all six depths.

The reason for improved results with the statistical evidence branch is that it reads the dependence statistics of the direct parents and grandparents from a precomputed tensor and never passes through a graph convolution, so its contribution is identical at $L = 1$ and at $L = 6$.

Residual connections address the same failure from the opposite side: they preserve the signal inside the propagation channel instead of bypassing it. Both residual connections and the statistical-evidence variants are effective on this benchmark, but they are not complementary in a consistent way. With residual connections enabled, the evaluated statistical-evidence variants did not provide a consistent additional improvement over the corresponding graph-only residual models on the synthetic benchmark (table 7.11).

Table 7.11: Task B: statistical-evidence variants with residual connections (TransformerConv backbone).

		Validation				Test			
L	Evidence variant	AUROC	AP	Lift	% oracle	AUROC	AP	Lift	% oracle
1	—	0.623	0.1255	2.31	81.30	0.619	0.1202	2.21	79.02
1	nonlin. bin.	0.622	0.1238	2.27	81.12	0.617	0.1176	2.16	77.82
1	nonlin. 42D	0.621	0.1250	2.30	80.56	0.616	0.1186	2.18	76.95
1	typed+triple	**0.631**	0.1335	2.45	**86.94**	**0.621**	0.1240	2.28	**80.30**
2	typed+triple	0.631	0.1322	2.43	86.77	0.619	0.1230	2.26	79.05
3	—	0.628	0.1346	2.47	85.23	0.620	0.1233	2.27	79.66
3	nonlin. bin.	0.627	0.1339	2.46	84.06	0.617	0.1242	2.28	77.48
3	nonlin. 42D	0.630	0.1343	2.47	85.95	0.619	**0.1247**	2.29	78.80
3	typed+triple	0.629	0.1350	2.48	85.64	0.618	0.1246	2.29	78.05
4	typed+triple	0.627	**0.1367**	**2.51**	84.10	0.621	**0.1256**	**2.31**	80.08
5	typed+triple	0.626	0.1361	2.50	83.88	**0.623**	**0.1256**	**2.31**	**81.54**
6	—	0.627	0.1337	2.46	84.19	0.620	0.1219	2.24	79.37
6	nonlin. bin.	0.614	0.1177	2.16	75.90	0.605	0.1082	1.99	69.84
6	nonlin. 42D	0.624	0.1301	2.39	82.21	0.618	0.1214	2.23	78.23
6	typed+triple	0.629	0.1347	2.47	85.62	0.620	0.1244	2.29	79.34

For understanding how the statistical evidence branch contributes to individual endpoint prediction, the learned gate weights are shown in figure 7.4. Each cell gives the scalar by which the model scales the evidence block of one parent type before it enters the classification head; the gates were initialised at 1.0, so values below unity indicate reduced reliance on that channel.

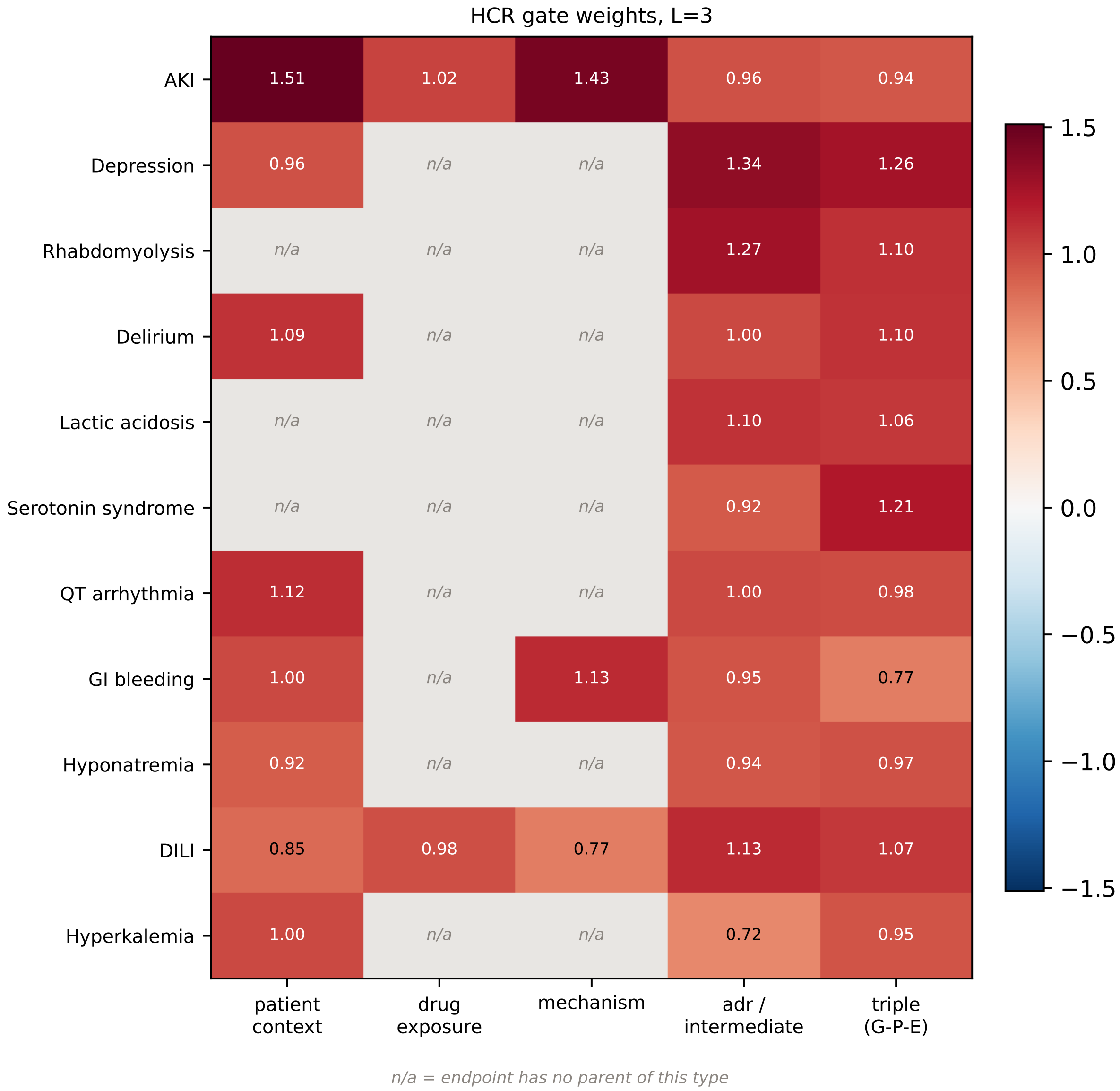

Figure 7.4: Statistical-evidence gate matrix: influence of parent node type per endpoint.

The gate ordering aligns with the number of endpoints that have at least one direct parent of each type in the graph, suggesting that the model allocates more weight to parent types that are structurally more informative.

7.6.4 Final Evaluation

In summary, models with a statistical evidence branch reached results comparable to models with residual connections applied. The gate matrix indicates that the evidence branch is used unevenly across endpoints and parent types, in line with differences in parent availability and predictive utility on the synthetic task. In general the value of the statistical evidence branch therefore does not lie in raising the peak, but in bringing stability to learning across all depths and additional information useful for clinical event prediction. Across the six depths evaluated, *typed+triple* is the only variant that never collapses, retaining between 82% and 89% of the oracle AUROC, while both the baseline and *nonlin. all 42D* fall below 80.00% at $L = 6$. The configuration *typed+triple* is mostly stable on depth and additionally brings the most informative value. The highest validation score in table 7.10 is obtained by *typed+triple* at $L = 1$ and $L = 4$ (89.05% and 86.65% of the oracle AUROC). Taken together, the *typed+triple* statistical-evidence variant is selected as the preferred configuration.

In table 7.12 there are presented results per endpoint.

Table 7.12: Task B: per-endpoint results for the selected model (*typed+triple* at $L = 4$ without residual connections).

		Oracle		Validation			Test		
Endpoint	Prev.	AUROC	AP	AUROC	AP	% oracle	AUROC	AP	% oracle
AKI	0.131	0.811	0.454	0.794	0.406	94.3	0.785	0.421	91.6
Depression	0.085	0.652	0.166	0.655	0.165	101.6	0.639	0.170	91.0
Hyponatremia	0.079	0.638	0.168	0.613	0.132	81.5	0.607	0.142	77.5
Hyperkalemia	0.049	0.590	0.083	0.595	0.080	105.7	0.550	0.074	55.2
Delirium	0.037	0.668	0.100	0.649	0.093	89.0	0.687	0.098	93.0
GI bleeding	0.021	0.570	0.080	0.570	0.080	99.0	0.528	0.029	39.6
DILI	0.018	0.653	0.086	0.563	0.067	41.3	0.579	0.036	51.6
QT arrhythmia	0.016	0.622	0.075	0.591	0.050	74.1	0.536	0.019	29.4
Macro	0.055	0.651	0.151	0.629	0.134	85.4	0.614	0.124	75.5
Trained but excluded from the macro average									
Lactic acidosis	0.0057	0.523	0.011	0.560	0.009	–	0.549	0.008	–
Rhabdomyolysis	0.0037	0.530	0.011	0.536	0.005	–	0.724	0.033	–
Serotonin syndrome	0.0027	0.554	0.008	0.646	0.010	–	0.642	0.008	–

Chapter 8

Cross-Domain Validation of Task B. Hetionet Proxy

8.1 Problem Formulation and Proxy Construction

Task B is evaluated on a benchmark dataset derived from Hetionet [17]. Hetionet is a heterogeneous network (*hetnet*) that integrates biomedical knowledge from 29 publicly available resources. It represents biological and pharmacological entities as nodes and documented relationships between them as typed edges.

Hetionet v1.0 contains 47,031 nodes belonging to 11 node types and 2,250,197 relationships of 24 types [18]. The graph includes compounds, diseases, genes, anatomies, pathways, biological processes, molecular functions, cellular components, pharmacologic classes, side effects, and symptoms. This heterogeneous structure makes Hetionet suitable for node classification task described further. The proxy construction and experimental protocol are specified in the remainder of this chapter.

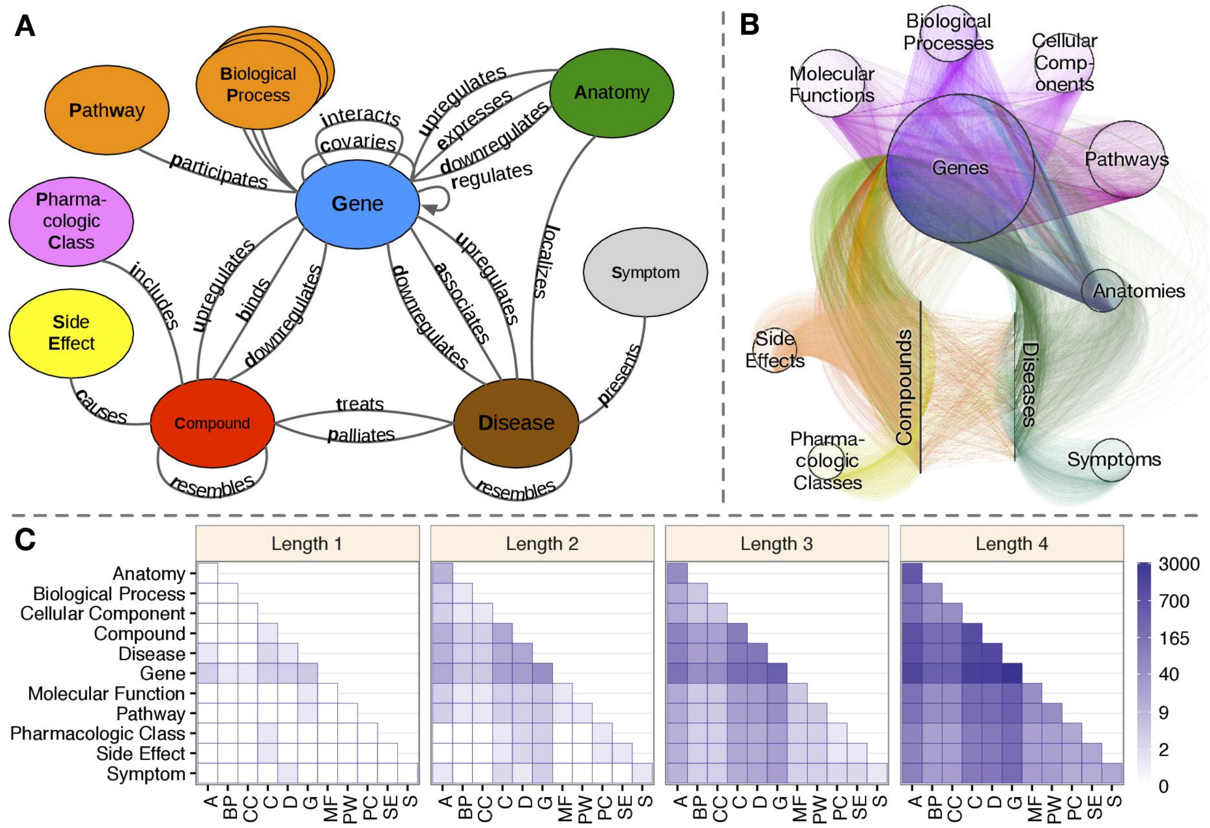

Figure 8.1: Hetionet v1.0 knowledge graph (adapted from [18]).

Figure 8.1 shows the Hetionet v1.0 network structure. Hetionet was selected because it is heterogeneous and supports a multi-label prediction setup analogous to the synthetic task.

8.2 Data Representation

The benchmark on this dataset is a structural proxy—the pipeline, layer slots, and statistical evidence branch are reused, but the graph is reconstructed to resemble the synthetic schema in topology. The reconstruction proceeds as follows.

(i) **Instance.** A compound takes the role of the patient: the topology is shared and a compound is described by the edges linking it to the remaining entities. A compound is therefore *not* a node of the graph, which means an entity can serve as a feature column only if it connects to compounds directly.

(ii) **Target.** The predicted endpoint is a disease node, labelled positive when a `treats` or `palliates` edge links the compound to that disease (the two relations were merged). Prevalences range from 1.29% to 4.51% over the twenty most frequent diseases.

(iii) **Layer mapping.** Node types were mapped onto the slots of the synthetic schema as presented in table 8.1.

(iv) **Direction.** Hetionet edges are undirected associations. For pipeline compatibility, every retained edge was oriented from the lower to the higher synthetic layer and intra-layer edges were discarded, guaranteeing acyclicity.

(v) **Path length.** Two changes lengthen the paths to a disease. First, a pathway and biological-process layer was inserted as the third layer, while the direct `DaG` edges were kept. Second, class-to-gene edges were added, because pharmacologic classes connect only to compounds in Hetionet and would otherwise have no edges at all. Such an edge is created when at least three training compounds of a class bind the same gene.

Table 8.1: Mapping of Hetionet entities onto the synthetic layer schema.

Layer	Slot	Hetionet entity	Observed
1	`patient_context`	Pharmacologic Class (filtered)	yes
2	`drug_exposure`	Gene, bound (`CbG`)	yes
3	`mechanism`	Pathway, Biological Process	no (latent)
4	`adr_or_intermediate_state`	Gene, associated (`DaG`)	no (latent)
5	`clinical_endpoint`	Disease	target

Figure 8.2 shows the updated layer mapping.

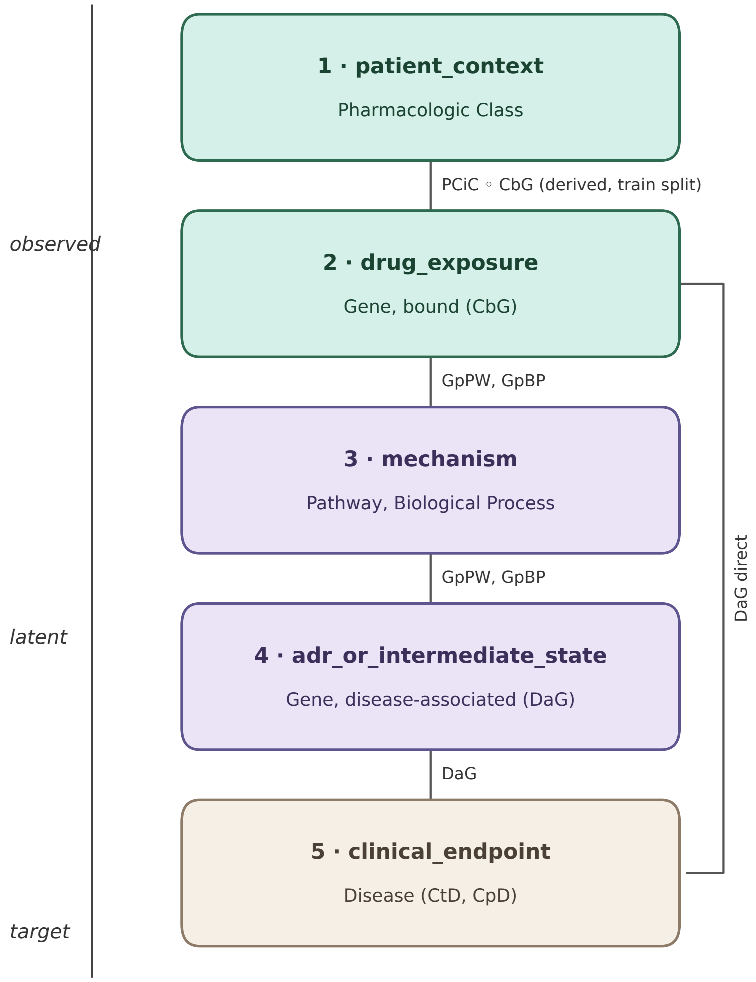

Figure 8.2: Hetionet layer mapping after reconstruction onto the synthetic schema.

8.3 Training and Evaluation

The Hetionet proxy reuses the Task B pipeline, layer slots, and statistical evidence branch. Experiments use the TransformerConv backbone with hidden width 64, batch size 8, learning rate 1×10^{-3}, DropEdge disabled, and three random seeds (42, 20256, 202567). The proxy cut comprises 1,552 compounds and twenty disease endpoints. Model selection follows the same validation-driven protocol as on the synthetic Task B cohort.

Table 8.2: Hetionet proxy: model and training configuration (differences from the Task B synthetic protocol).

Setting	Value
Backbone	TransformerConv (as in Task B)
Hidden width	64
Batch size	8
Learning rate	1×10^{-3}
DropEdge	disabled
Seeds	42, 20256, 202567
Compounds (instances)	1,552
Disease endpoints	20

8.4 Experimental Results

The benchmark reproduces several trends observed on the synthetic dataset – depth sensitivity without stabilisers, partial recovery with the statistical evidence branch – under a different graph semantics and without oracle metrics. Reported scores in table 8.3 are the mean ± sample standard deviation over up to three seeds. The gain from statistical evidence is conditional on the state of the propagation path: at $L = 6$ without residual connections the graph-only model reaches 0.615 ± 0.026 test AUROC, while statistical-evidence variants recover part of the lost ranking quality (0.703–0.712). At $L = 1$, where the receptive field of an endpoint coincides with the input of the statistical evidence branch, graph-only and statistical-evidence variants remain close (test AUROC 0.667–0.672).

At $L = 6$, residual connections already restore much of the graph pathway (0.728 ± 0.010 without the statistical branch). Adding statistical evidence on top of residuals yields 0.726–0.742, so the two mechanisms are only weakly additive on this proxy: the residual path carries most of the depth-related gain, while the statistical branch helps mainly when that path is absent.

The reason is the structure of the two graphs. In the synthetic graph the direct parents of an endpoint are observed, so the statistical evidence branch and the one-hop receptive field read the same variables and the two mechanisms address the same bottleneck. In the Hetionet-derived graph depth is more crucial because some layers are latent. Information must therefore traverse several hops, and the residual path and the HCR bypass contribute at different points of that route.

Overall, the proxy experiment suggests that combining message passing with the statistical evidence branch remains useful with multi-hop reasoning. Whether an additional residual path helps or hinders depends on where the statistical evidence enters the model.

Table 8.3: Hetionet proxy: endpoint-prediction results under the reconstructed layer schema.

			Validation			Test		
L	Res.	Evidence variant	AUROC	AP	Lift	AUROC	AP	Lift
1	no	—	0.757 ± 0.005	0.218 ± 0.027	9.20 ± 1.15	0.668 ± 0.006	0.125 ± 0.010	6.78 ± 0.53
1	no	nonlin. bin.	0.758 ± 0.005	0.219 ± 0.020	9.23 ± 0.83	0.667 ± 0.008	0.128 ± 0.007	6.90 ± 0.39
1	no	nonlin. all	0.757 ± 0.004	0.213 ± 0.012	8.99 ± 0.52	0.667 ± 0.002	0.135 ± 0.005	7.27 ± 0.24
1	no	typed+triple	0.765 ± 0.011	0.216 ± 0.049	9.10 ± 2.05	0.668 ± 0.006	0.132 ± 0.026	7.17 ± 1.40
1	yes	—	0.756 ± 0.002	0.213 ± 0.014	8.96 ± 0.61	0.671 ± 0.004	0.132 ± 0.029	7.13 ± 1.55
1	yes	nonlin. bin.	0.756 ± 0.005	0.215 ± 0.019	9.06 ± 0.81	0.672 ± 0.007	0.142 ± 0.006	7.69 ± 0.35
1	yes	nonlin. all	0.757 ± 0.001	0.222 ± 0.018	9.38 ± 0.77	0.667 ± 0.003	0.137 ± 0.002	7.41 ± 0.08
1	yes	typed+triple	0.755 ± 0.004	0.210 ± 0.046	8.84 ± 1.94	0.671 ± 0.009	0.136 ± 0.034	7.37 ± 1.83
2	no	nonlin. bin.*	0.761 ± 0.004	0.254 ± 0.000	10.70 ± 0.01	0.683 ± 0.007	0.160 ± 0.004	8.66 ± 0.20
2	no	nonlin. all	0.760 ± 0.005	0.261 ± 0.018	11.03 ± 0.75	0.686 ± 0.009	0.165 ± 0.029	8.93 ± 1.56
2	no	typed+triple	0.768 ± 0.008	0.262 ± 0.002	11.06 ± 0.07	0.685 ± 0.007	0.169 ± 0.005	9.14 ± 0.28
2	yes	nonlin. bin.*	0.764 ± 0.004	0.277 ± 0.004	11.70 ± 0.17	0.691 ± 0.009	0.181 ± 0.011	9.78 ± 0.58
2	yes	nonlin. all	0.758 ± 0.003	0.241 ± 0.015	10.15 ± 0.62	0.685 ± 0.004	0.162 ± 0.007	8.75 ± 0.39
2	yes	typed+triple	0.768 ± 0.004	0.258 ± 0.022	10.86 ± 0.93	0.691 ± 0.001	0.189 ± 0.014	10.21 ± 0.75
3	no	—	0.742 ± 0.012	0.200 ± 0.008	8.45 ± 0.35	0.689 ± 0.029	0.132 ± 0.027	7.13 ± 1.46
3	no	nonlin. bin.	0.798 ± 0.009	0.275 ± 0.003	11.59 ± 0.13	0.692 ± 0.008	0.172 ± 0.005	9.28 ± 0.28
3	no	nonlin. all	0.791 ± 0.016	0.264 ± 0.006	11.13 ± 0.24	0.700 ± 0.002	0.165 ± 0.001	8.91 ± 0.08
3	no	typed+triple	0.801 ± 0.006	0.266 ± 0.025	11.23 ± 1.06	0.702 ± 0.002	0.167 ± 0.001	9.05 ± 0.06
3	yes	—	0.802 ± 0.004	0.270 ± 0.045	11.41 ± 1.90	0.700 ± 0.014	0.179 ± 0.014	9.70 ± 0.76
3	yes	nonlin. bin.	0.799 ± 0.020	0.270 ± 0.036	11.40 ± 1.54	0.712 ± 0.003	0.179 ± 0.023	9.67 ± 1.25
3	yes	nonlin. all	0.805 ± 0.004	0.274 ± 0.013	11.54 ± 0.54	0.696 ± 0.017	0.177 ± 0.005	9.59 ± 0.30
3	yes	typed+triple	0.803 ± 0.003	0.289 ± 0.036	12.20 ± 1.52	0.711 ± 0.004	0.185 ± 0.006	10.00 ± 0.34
4	no	nonlin. bin.*	0.760 ± 0.001	0.254 ± 0.005	10.71 ± 0.20	0.683 ± 0.006	0.169 ± 0.002	9.13 ± 0.12
4	no	nonlin. all	0.774 ± 0.016	0.237 ± 0.006	9.99 ± 0.26	0.702 ± 0.022	0.152 ± 0.001	8.21 ± 0.06
4	no	typed+triple	0.778 ± 0.025	0.264 ± 0.004	11.12 ± 0.17	0.708 ± 0.033	0.172 ± 0.008	9.30 ± 0.46
4	yes	nonlin. bin.*	0.815 ± 0.003	0.297 ± 0.002	12.52 ± 0.09	0.729 ± 0.007	0.198 ± 0.001	10.70 ± 0.03
4	yes	nonlin. all	0.798 ± 0.022	0.264 ± 0.022	11.14 ± 0.94	0.709 ± 0.015	0.188 ± 0.022	10.16 ± 1.18
4	yes	typed+triple	0.808 ± 0.008	0.285 ± 0.039	12.03 ± 1.67	0.737 ± 0.023	0.185 ± 0.015	9.99 ± 0.85
5	no	—*	0.605 ± 0.000	0.133 ± 0.004	5.63 ± 0.16	0.628 ± 0.001	0.083 ± 0.005	4.50 ± 0.27
5	no	nonlin. bin.*	0.796 ± 0.000	0.249 ± 0.003	10.50 ± 0.11	0.751 ± 0.002	0.161 ± 0.001	8.72 ± 0.03
5	no	nonlin. all	0.775 ± 0.017	0.241 ± 0.002	10.15 ± 0.07	0.693 ± 0.021	0.159 ± 0.007	8.59 ± 0.36
5	no	typed+triple	0.792 ± 0.013	0.262 ± 0.016	11.06 ± 0.69	0.714 ± 0.038	0.165 ± 0.002	8.92 ± 0.12
5	yes	—*	0.807 ± 0.008	0.293 ± 0.015	12.36 ± 0.63	0.735 ± 0.021	0.190 ± 0.002	10.29 ± 0.10
5	yes	nonlin. bin.*	0.816 ± 0.003	0.326 ± 0.017	13.76 ± 0.73	0.763 ± 0.020	0.212 ± 0.025	11.49 ± 1.37
5	yes	nonlin. all	0.812 ± 0.004	0.299 ± 0.008	12.60 ± 0.31	0.728 ± 0.001	0.179 ± 0.009	9.67 ± 0.48
5	yes	typed+triple	0.820 ± 0.010	0.292 ± 0.023	12.29 ± 0.97	0.731 ± 0.014	0.197 ± 0.021	10.67 ± 1.13
6	no	—	0.608 ± 0.002	0.128 ± 0.006	5.40 ± 0.25	0.615 ± 0.026	0.079 ± 0.002	4.27 ± 0.11
6	no	nonlin. bin.	0.782 ± 0.013	0.237 ± 0.023	9.98 ± 0.97	0.707 ± 0.038	0.165 ± 0.009	8.94 ± 0.48
6	no	nonlin. all	0.771 ± 0.028	0.239 ± 0.005	10.07 ± 0.23	0.703 ± 0.020	0.153 ± 0.004	8.29 ± 0.20
6	no	typed+triple	0.777 ± 0.022	0.261 ± 0.002	11.01 ± 0.09	0.712 ± 0.039	0.167 ± 0.007	9.05 ± 0.35
6	yes	—	0.811 ± 0.006	0.263 ± 0.029	11.11 ± 1.20	0.728 ± 0.010	0.168 ± 0.022	9.09 ± 1.20
6	yes	nonlin. bin.	0.820 ± 0.011	0.302 ± 0.023	12.74 ± 0.97	0.742 ± 0.016	0.186 ± 0.015	10.06 ± 0.81
6	yes	nonlin. all	0.814 ± 0.005	0.278 ± 0.037	11.71 ± 1.56	0.739 ± 0.009	0.180 ± 0.014	9.71 ± 0.76
6	yes	typed+triple	0.815 ± 0.006	0.298 ± 0.042	12.57 ± 1.78	0.726 ± 0.010	0.204 ± 0.007	11.05 ± 0.38

Chapter 9

Discussion and Limitations

9.1 Common Methodological Principle

The two prediction tasks investigated in this thesis address different objects, but are based on a common methodological principle: structured graph knowledge and explicit empirical statistical evidence provide different, potentially complementary descriptions of the same system.

In Task A, the prediction object is a candidate directed relation. The structural branch describes the position of the candidate nodes within the heterogeneous graph, while the statistical branch describes empirical dependence patterns observed in the patient population. In Task B, the graph topology is shared across patients and the prediction object becomes a patient-specific clinical endpoint. Patient observations are propagated through the graph, while an explicit statistical evidence branch provides an additional description of endpoint-related dependencies.

The common modelling template can therefore be summarized as

$$\text{structured graph representation} + \text{explicit empirical evidence} \;\rightarrow\; \text{task-specific prediction.} \tag{9.1}$$

The relative contribution of the two components is task dependent. Statistical evidence is particularly useful when relevant information is not adequately represented or preserved by the graph pathway, whereas a sufficiently effective graph encoder may already capture part of the same signal. On the synthetic Task B benchmark this distinction is concrete: when residual connections preserve endpoint-relevant information inside the message-passing stack, the evaluated statistical-evidence variants do not provide a consistent additional improvement over the corresponding graph-only residual models, whereas the same branch recovers substantial performance when deep models degrade without residual propagation.

9.2 What the Two Synthetic Tasks Show

Task A: confronting structured knowledge with observational evidence

Task A demonstrates that patient-derived statistical evidence can provide substantial information beyond heterogeneous graph topology alone. Progressive enrichment of the candidate-specific statistical representation improves directed edge ranking relative to the graph-only HGT baseline.

Importantly, statistical dependence does not determine edge direction. Direction is supplied by the ordered candidate definition and the mechanistic graph schema. HCR-derived coefficients provide empirical evidence about relationships between variables, but are not interpreted as causal effects. Task A can therefore be understood as confronting prior structured knowledge with population-level observational evidence under a controlled reconstruction objective. It does not perform causal discovery.

Residual connections inside the Task A HGT encoder are standard skip connections with LayerNorm; they are architectural details of the structural branch and are not studied as an anti-oversmoothing intervention in that chapter.

Task B: structural context for patient-level endpoints

Task B examines whether a shared mechanistic graph can provide useful structural context for predicting patient-level clinical endpoints.

The experiments show that the usefulness of the statistical branch depends strongly on the state of the graph propagation pathway. Without residual connections, deeper models can lose endpoint-relevant information, while the explicit statistical branch can provide a direct alternative information route. When residual propagation preserves the graph signal, the statistical branch does not provide a consistent additional improvement on the synthetic benchmark. With residual connections enabled, the evaluated statistical-evidence variants did not provide a consistent additional improvement over the corresponding graph-only residual models on the synthetic benchmark.

9.3 Cross-Domain Proxy Experiments

The proxy experiments test whether the architectural principle survives substantial changes in graph semantics and prediction objective rather than establishing direct domain transfer.

Last-FM* replaces pharmacotherapy variables with users, items and knowledge-graph entities and changes the objective from directed graph reconstruction to recommendation ranking. The development ablation shows that candidate-specific empirical context substantially improves the neural HGT model. The matched sealed benchmark additionally shows that improving a neural graph model does not imply superiority over strong classical recommenders: development gains over HGT are not equivalent to sealed-test dominance, and the proposed full model remains below the strongest classical baseline (RP3β) under the sealed external protocol.

The three Last-FM* result levels therefore answer different questions and must not be interchanged. Sampled validation NDCG@20 is a cheap checkpoint-selection proxy; full-catalog development ranking isolates the contribution of candidate-specific context relative to HGT alone; only the sealed external evaluation is comparable to published recommender numbers under the same evaluator. In particular, the LEG residual in Last-FM* is an additive correction in decoder score space and must not be conflated with residual connections inside a message-passing stack: on the sampled checkpoint metric LEG changes NDCG@20 by only about 0.0004, whereas on full-catalog development ranking it raises mean NDCG@20 from about 0.232 (HGT+A5+H3) to about 0.276 (full model).

The Hetionet-derived proxy performs the analogous stress test for Task B. It preserves the endpoint-prediction logic while introducing different graph semantics and longer, partially latent information pathways. Under this setting, explicit statistical evidence becomes particularly useful for deeper models, and statistical evidence and residual propagation can become complementary: at six layers, test AUROC rises from 0.585 for the graph-only model to 0.757 with statistical evidence alone and 0.760 when both mechanisms are combined. Hetionet remains a methodological proxy rather than clinical validation.

9.4 Prediction versus Mechanistic Clinical Reasoning

Predictive performance alone does not fully determine the clinical usefulness of a model. A patient-level predictor primarily addresses the question of whether a clinical event is likely to occur. Clinical decision making additionally requires understanding which observed factors and plausible mechanisms may contribute to that risk and which of them could potentially be modified.

This distinction is especially important in pharmacotherapy. The same clinical endpoint may arise through different combinations of drug exposure, comorbidity, physiological state, laboratory abnormalities and intermediate adverse effects. A predicted risk of an endpoint is therefore potentially more informative when it can be related to plausible mechanistic routes than when it is presented as an isolated score.

The graph-based formulation investigated in this thesis provides a structural basis for such an extension. The present work evaluates graph reconstruction and endpoint prediction; it does not yet identify patient-specific causal mechanisms. A natural next stage would be to determine which regions and pathways of the mechanistic graph are supported by the observations of an individual patient.

Instead of treating the predicted endpoint as the final output, prediction becomes one component of a broader reasoning process:

$$\text{endpoint risk} \;\rightarrow\; \text{plausible mechanistic routes} \;\rightarrow\; \text{modifiable factors} \;\rightarrow\; \text{possible intervention hypothe} \tag{9.2}$$

The present architecture does not perform intervention analysis; the sequence above is a motivating interpretation for future work.

9.5 Observational Data, Explainability and Causal Interpretation

Clinical data are generated through a healthcare process that includes decisions about testing, treatment, documentation and follow-up. Consequently, recorded associations may reflect both the patient's underlying condition and institution-specific patterns of observation or clinical practice.

This issue motivates the controlled observational scenarios used in the synthetic benchmark, including selection bias, noisy documentation and multi-hospital variation. In the synthetic setting these processes can be modified while the underlying graph remains fixed. In real clinical data, the corresponding observation mechanisms would be substantially more complex and only partially observable.

Mechanistic prior knowledge may provide useful structure when empirical associations are unstable across institutions or populations. It does not remove confounding or establish causal direction, but it provides an explicit hypothesis about how variables are related that can be evaluated against observations.

Model explainability and mechanistic explanation should also remain conceptually distinct. Feature attribution can indicate which model inputs contributed to a prediction, but this does not establish that those variables constitute the mechanism responsible for the clinical event. A mechanistic graph provides a different form of explanation by explicitly representing hypothesized relationships and pathways; causal interpretation still requires additional assumptions and validation.

The present thesis therefore remains at the level of predictive modelling supported by mechanistic priors and empirical dependence. Future work could connect this framework with dedicated causal-discovery or causal-inference methods, but such extensions should not be inferred from predictive performance alone.

9.6 Average Precision and Asymmetric Error Costs in Graph Reconstruction

Average Precision provides a threshold-independent summary of the trade-off between recovering existing relations and avoiding unsupported ones. It is well suited to model comparison and checkpoint selection in the present link-prediction setting. However, it does not explicitly assign different downstream costs to false-positive and false-negative errors.

This distinction becomes important when link prediction is interpreted as graph reconstruction rather than only as binary classification. A false-negative prediction leaves a potentially valid relation unresolved. A false-positive prediction introduces into the reconstructed graph a relation that is absent from the audited reference structure.

This difference can be expressed conceptually as

$$C_{\mathrm{FP}} > C_{\mathrm{FN}}, \tag{9.3}$$

when the predicted graph is intended to support subsequent structural or mechanistic reasoning. The inequality is not imposed by the current training objective; rather, it represents a task-dependent consideration for downstream use of reconstructed edges.

A missed edge preserves uncertainty about a part of the graph. A spurious edge can instead convert uncertainty into an incorrect structural assertion and may alter the set of paths and neighbourhoods available for subsequent reasoning. In applications in which predicted relations are incorporated into a knowledge graph, it may therefore be preferable to recover a smaller set of high-confidence edges rather than to maximize recall at the cost of introducing unsupported structure.

AP evaluates the overall quality of the candidate-edge ranking, but it does not determine the operating point at which a predicted edge should be accepted into an expanded graph. Future graph-completion studies could therefore complement AP with precision-oriented operating-point measures, for example precision at a prescribed recall level, precision among the highest-ranked candidates, or a validation-selected threshold τ in a selective acceptance rule

$$\hat{E}_{\mathrm{add}} = \Big\{(A, G) : s_{AG} \geq \tau\Big\}. \tag{9.4}$$

Consequently, graph completion intended for knowledge enrichment should ultimately be treated not only as a ranking problem, but also as a selective knowledge-acceptance problem. A high-scoring candidate should become an asserted graph relation only when the available evidence satisfies an acceptance criterion appropriate to its downstream use. The present study does not implement such a deployment rule.

9.7 Limitations and Deployment Implications

The primary limitation of the present study is the synthetic nature of the pharmacotherapy benchmark. The known graph and data-generating process provide a controlled environment for methodological evaluation, but cannot reproduce the full heterogeneity, measurement noise and confounding of real clinical records. At the same time, the controlled setting is useful for studying architectural questions that would be difficult to isolate when both the graph structure and observation process are unknown. The synthetic benchmark should therefore be interpreted as methodological ground truth rather than as a substitute for clinical evidence.

The Last-FM* and Hetionet experiments extend the methodological stress testing beyond the original synthetic setting, but neither constitutes validation on real patient data. Generator-based oracle references in Task B are synthetic artefacts and should not be read as clinical ceilings or as strict theoretical upper bounds.

A practical feature of the proposed framework is its modularity. The graph encoder and explicit statistical branch can be modified independently, allowing alternative message-passing operators, dependence estimators or domain-specific evidence channels to be evaluated without redesigning the complete architecture.

The explicit statistical evidence branch is comparatively lightweight once cohort-level statistics have been precomputed, although the computational cost of the complete system depends on graph size, message-passing architecture and the number of candidate predictions required.

Separating structural and statistical evidence also improves inspectability. Individual dependence coefficients, evidence blocks and learned gates can be examined independently of the latent GNN representation. This does not make the complete model intrinsically interpretable, but provides an explicit evidence layer that can support auditing and hypothesis generation.

The patient-specific graph formulation can also be viewed as one possible methodological building block toward future mechanistic digital-twin systems. The present architecture is not a digital twin: it lacks temporal state updating, validated intervention models, counterfactual estimation and real-world clinical calibration. Nevertheless, representing patient observations on a shared mechanistic graph provides a possible intermediate step between static risk prediction and dynamic patient-specific mechanistic modelling.

Chapter 10

Conclusions and Future Work

10.1 Conclusions

This thesis investigated whether structured graph knowledge and explicit empirical statistical evidence can be integrated within a common modelling framework for two complementary problems: reconstruction of directed graph relations and prediction of patient-level clinical endpoints.

In Task A, a heterogeneous graph encoder was combined with a candidate-specific HCR-inspired statistical representation derived from patient observations. The results showed that explicit empirical context provided substantial predictive information beyond graph topology alone. The Last-FM* proxy demonstrated that the same broad modelling principle could be transferred to a different domain and ranking objective, while the sealed benchmark also showed that improvements within a neural graph model do not necessarily translate into superiority over strong classical methods.

In Task B, patient-specific observations were propagated over a shared pharmacotherapy graph and complemented with endpoint-oriented statistical evidence. The results showed that the value of this evidence depends on the ability of the graph neural network to preserve predictive information across message-passing layers. Residual connections and the statistical evidence branch can address related information-loss mechanisms on the synthetic graph, while the Hetionet-derived proxy showed that their effects may become complementary when relevant information must traverse longer or partially latent paths.

Taken together, the experiments support a methodological rather than clinical conclusion. Structured graph representations and explicit empirical dependence can provide complementary information, but their contribution depends on the graph topology, prediction task, information pathway and evaluation protocol. The synthetic and proxy experiments establish a foundation for further investigation on real clinical data; they do not establish clinical utility or causal validity.

10.2 Future Work

Real-world clinical validation. The most important next step is evaluation on real longitudinal clinical data. Such studies would need to address irregular observation, missingness, heterogeneous coding, treatment-selection effects, institutional variation and unmeasured confounding. The primary question would be which components of the architecture remain useful when both the graph structure and empirical evidence are uncertain. Multicentre validation would be especially important for distinguishing relationships that remain stable across institutions from correlations that reflect local observation or treatment practice.

Higher-order statistical dependence. The present experiments make limited use of higher-order HCR coefficients, particularly for continuous and mixed variables. Richer clinical datasets containing laboratory measurements and longitudinal physiological variables would provide a more informative setting in which to evaluate whether higher-order orthonormal coefficients capture dependence patterns that cannot be represented adequately by first-order association measures.

Causal discovery and graph refinement. A further direction is to combine mechanistic graph priors with dedicated causal-discovery methods. HCR should remain an empirical dependence representation rather than a causal estimator, but its coefficients could provide candidate statistical evidence to methods that introduce the additional assumptions required for directional inference. Approaches based on additive-noise models, LiNGAM-type methods, or other suitable causal-discovery frameworks could therefore be investigated alongside graph priors, with careful attention to variable type and assumption validity rather than automatic application across the graph.

Patient-specific mechanistic path activation. Clinical endpoint prediction alone does not identify which mechanisms are responsible for the predicted risk. A natural extension is therefore to move from endpoint prediction toward patient-specific activation of mechanistic paths. For patient p and endpoint Y, let

$$\mathcal{P}_p(Y) = \{\pi_1, \pi_2, \dots, \pi_K\} \tag{10.1}$$

denote plausible mechanistic routes, with patient-conditioned activation weights $w_p(\pi_k)$. Several routes should be allowed to remain simultaneously active. This is particularly important in polypharmacy, where multiple drugs, risk factors and intermediate mechanisms may contribute concurrently to the same adverse endpoint. A future temporal aggregation layer could model how these routes accumulate, interact and converge over time rather than forcing the system to select a single dominant pathway.

Active information acquisition and missing evidence. Missing information could also be treated as part of the reasoning problem rather than an incidental preprocessing detail. When several mechanistic paths remain compatible with the observed patient state, the model could identify which unobserved variable or clinical measurement would be most informative for distinguishing between them. Such an approach would connect prediction with targeted information acquisition: the system would not only estimate risk, but identify which additional observation could most reduce uncertainty about the mechanism responsible for that risk.

Entropy-based path weighting. Entropy-based graph-walk methods, including maximal-entropy random walks, provide one possible starting point for weighting alternative routes through a mechanistic graph. Standard topology-driven random walks would, however, require modification for the present setting because clinical path relevance should depend on patient-specific evidence, not graph structure alone. A future patient-conditioned finite-path formulation could therefore combine structural accessibility with observed activation of the variables along a route. Such a method should permit several routes to contribute simultaneously to the same endpoint.

KAN as an alternative nonlinear encoder. Kolmogorov–Arnold Networks (KANs) could be evaluated more systematically as an alternative to the MLP blocks used for statistical encoding and fusion. Their learnable univariate transformations may be attractive for heterogeneous statistical descriptors and nonlinear interactions. Exploratory MLP-versus-KAN comparisons for Task A role encoders did not establish an advantage of KAN over MLPs under the reported protocol, so this remains an open architectural comparison rather than a settled design choice.

Towards mechanistic digital twins. The combination of patient-specific graph instantiation, mechanistic prior knowledge and explicit empirical evidence is conceptually related to emerging medical digital-twin research. The goal should not be understood as constructing a complete virtual replica of a human patient. A more realistic intermediate objective is a dynamically updated representation of the clinically relevant mechanisms, exposures, observations and uncertainties associated with a specific decision problem. A clinically useful digital-twin framework would ultimately require temporal updating, intervention modelling, uncertainty quantification, external validation and careful governance. The present thesis addresses only some of the representational prerequisites for such a system.

Iterative Task A–Task B loop. Finally, Task A and Task B could be connected into an iterative framework. High-confidence candidate relations identified during graph reconstruction could be introduced into a provisional graph and subsequently evaluated through patient-level endpoint prediction. Conversely, systematic endpoint-prediction errors could identify regions of the graph in which structural knowledge requires further examination. Such a loop would move the framework from static graph reconstruction and endpoint prediction toward continual evaluation of mechanistic knowledge against accumulating empirical evidence.

The longer-term objective is therefore not only to predict what may happen, but to connect prediction with the mechanisms, uncertainty and potentially modifiable pathways that make the prediction clinically meaningful.

Appendix A

HCR-Inspired Pair Representation: Detailed Definition

The statistical branch in Task A represents each ordered variable pair (U, V) by a fixed 40-dimensional vector. Its core construction is inspired by Hierarchical Correlation Reconstruction (HCR), where dependence is represented through coefficients of products of orthonormal basis functions [10, 9, 14]. The implementation extends this coefficient representation with fixed-width summaries describing dependence magnitude, marginal distributions, joint activation, data support, estimability and variable type. This appendix gives the complete formula-level definition used by the final model.

Two layers must be kept distinct.

(1) **HCR inspiration.** Dependence is encoded through coefficients of products of type-specific orthonormal basis functions, following the HCR principle of representing statistical dependence in an orthonormal expansion [10, 9, 14].

(2) **Thesis-specific extensions.** Fixed-width padding, derived dependence summaries, marginal descriptors, joint-activation descriptors, support and estimability features, and type metadata provide a common interface for heterogeneous binary, count and continuous variables.

The resulting object is therefore called an *HCR-inspired pair representation*. It is a statistical dependence descriptor and does not identify causal direction.

A.1 Overall 40-dimensional representation

Define

$$h(U, V) \in \mathbb{R}^{40}. \tag{A.1}$$

Let

$$I_{UV} = \{i : U_i \text{ and } V_i \text{ are both observed}\} \tag{A.2}$$

and

$$n = |I_{UV}|, \tag{A.3}$$

with n_{train} denoting the total number of training patients. All bases and statistical descriptors are fitted from training patients only.

The slot composition is

Slots	Content
0–15	padded HCR-inspired coefficient matrix
16–19	derived dependence summaries
20–23	marginal descriptors of U
24–27	marginal descriptors of V
28–31	joint activation / enrichment / rarity
32	complete-case support
33	estimability indicator
34–36	one-hot type of U
37–39	one-hot type of V

Dimension check:

$$16 + 4 + 4 + 4 + 4 + 1 + 1 + 3 + 3 = 40. \tag{A.4}$$

Table A.1: Block layout of the 40-dimensional HCR-inspired pair representation.

Slots	Dim.	Block
0–15	16	padded coefficient matrix A_{pad}
16–19	4	derived dependence summaries
20–23	4	marginal descriptors $m(U)$
24–27	4	marginal descriptors $m(V)$
28–31	4	joint activation / enrichment / rarity
32	1	complete-case support S
33	1	estimability indicator I_{sup}
34–36	3	one-hot type of U
37–39	3	one-hot type of V

A.2 HCR-inspired coefficient matrix

For each variable construct type-specific non-constant orthonormal basis functions

$$\phi_j^{(U)}, \quad j = 1, \ldots, d_U, \qquad \phi_k^{(V)}, \quad k = 1, \ldots, d_V. \tag{A.5}$$

Define

$$a_{jk} = \frac{1}{n} \sum_{i \in I_{UV}} \phi_j^{(U)}(U_i)\, \phi_k^{(V)}(V_i). \tag{A.6}$$

Equivalently, with design matrices Φ_U and Φ_V whose rows are the evaluated basis functions on complete cases,

$$A_{UV} = \frac{1}{n} \Phi_U^{\mathsf{T}} \Phi_V. \tag{A.7}$$

Zero-pad the active block to

$$A_{\text{pad}} \in \mathbb{R}^{4 \times 4} \tag{A.8}$$

and store it row-wise (row-major) in slots 0–15:

$$A_{\text{pad}} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}. \tag{A.9}$$

Active block sizes by pair type are

Pair type	Active block
binary $\times$ binary	1×1
binary $\times$ continuous	1×4
binary $\times$ count	1×4
count $\times$ count	up to 4×4
count $\times$ continuous	up to 4×4
continuous $\times$ continuous	up to 4×4

The coefficient a_{11} describes the lowest-order dependence mode; higher-order coefficients allow the representation to capture additional nonlinear structure rather than reducing every pair to one correlation number. Individual higher-order coefficients are not assigned unique causal or clinical meanings.

A.3 Binary basis and connection to Pearson ϕ

For $B \in \{0, 1\}$, with training prevalence $p_B = P(B = 1)$, define the standardized non-constant basis function

$$\phi_1^{(B)}(B) = \frac{B - p_B}{\sqrt{p_B(1 - p_B)}}. \tag{A.10}$$

For a binary–binary pair,

$$a_{11} = \frac{p_{11} - p_U p_V}{\sqrt{p_U(1 - p_U)\, p_V(1 - p_V)}}, \tag{A.11}$$

which is equivalent to the Pearson ϕ coefficient. For binary pairs, a_{11} therefore reduces to Pearson ϕ. This is the conceptual link between the simple binary association used elsewhere and the more general HCR-inspired coefficient representation.

A.4 Continuous variables

Continuous variables are transformed using the empirical training distribution. The training rank / pseudo-observation construction is

$$u_i = \mathrm{clip}\left(\frac{\frac{1}{2}(\ell_i + r_i) + 0.5}{N_{\text{train}} + 1}, 10^{-6}, 1 - 10^{-6}\right), \tag{A.12}$$

where ℓ_i and r_i are the left and right rank positions of tied observations in the sorted training sample. The transformed value is expanded in shifted orthonormal Legendre functions

$$\psi_j(u) = \sqrt{2j + 1}\, P_j(2u - 1), \qquad j = 1, \ldots, 4, \tag{A.13}$$

where P_j is the Legendre polynomial of order j. The rank transformation removes dependence on the original marginal scale, while the orthonormal polynomial expansion provides several low-order modes with which joint dependence can be represented. Legendre polynomials are the chosen orthonormal basis in this implementation; they are not themselves the HCR method.

A.5 Count variables

Counts are capped using the training 99.5th percentile,

$$C^c = \min(C, c_{\max}), \qquad c_{\max} = q_{0.995}. \tag{A.14}$$

Repeated integer values are handled with deterministic distributional jitter,

$$u_i = F_C(c_i^-) + \xi_i\, P(C = c_i), \qquad \xi_i \in (0, 1), \tag{A.15}$$

where ξ_i is deterministic and reproducible. The transformed values are then passed through the same four shifted Legendre basis functions used for continuous variables. Count variables are *not* simply binarized in the final model.

A.6 Four derived dependence summaries (slots 16–19)

The following four deterministic summaries of the active coefficient matrix are stored in slots 16–19.

For slot 16 (total energy),

$$D_{\text{tot}} = \|A\|_F^2 = \sum_{j,k} a_{jk}^2. \tag{A.16}$$

For slot 17 (mean energy),

$$D_{\text{mean}} = \frac{\|A\|_F^2}{d_U\, d_V}. \tag{A.17}$$

For slot 18 (low-order energy fraction),

$$R_{\text{low}} = \frac{\|A_{1:2,1:2}\|_F^2}{\|A\|_F^2 + \varepsilon}, \tag{A.18}$$

with $\varepsilon = 10^{-12}$ in the implementation. For slot 19 (strongest coefficient),

$$M_{\max} = \max_{j,k} |a_{jk}|. \tag{A.19}$$

Interpretation: D_{tot} is overall dependence magnitude; D_{mean} normalizes that magnitude by the number of active coefficients; R_{low} is the fraction of dependence concentrated in low-order modes; $M_{\max}$ is the strength of the strongest individual dependence mode. These four values are deterministic summaries of A, not independent statistical estimators.

A paired ablation that removed slots 16–19 (reducing the vector from 40 to 36 dimensions) lowered validation Average Precision (AP) by approximately 0.008 on average, which supports retaining the summaries while treating them as derived rather than independent evidence.

A.7 Type-specific marginal descriptors (slots 20–27)

Each variable contributes a four-dimensional marginal vector

$$m(U) \in \mathbb{R}^4, \qquad m(V) \in \mathbb{R}^4, \tag{A.20}$$

with slots 20–23 storing $m(U)$ and slots 24–27 storing $m(V)$.

Binary

1. prevalence $p_1 = P(B = 1)$;
2. normalized binary entropy

$$H_2(p_1) = \frac{-p_1 \ln p_1 - (1 - p_1)\ln(1 - p_1)}{\ln 2}; \tag{A.21}$$

3. bounded imbalance $\tanh\Big(\mathrm{logit}(p_1)/4\Big)$, where

$$\mathrm{logit}(p) = \ln \frac{p}{1 - p}; \tag{A.22}$$

4. positive-state rarity $(n_1 + 1)^{-1/2}$.

Count

Using the capped count C^c:

1. zero mass $P(C^c = 0)$;
2. scaled count intensity

$$\frac{\mathbb{E}[\log(1 + C^c)]}{\log(1 + c_{\max})}; \tag{A.23}$$

3. normalized empirical count entropy $H(\hat{\pi})/\log K_{\mathrm{obs}}$, where

$$K_{\mathrm{obs}} = \max\Big(|\mathrm{unique}(C^c)|, 2\Big) \tag{A.24}$$

and

$$H(\hat{\pi}) = -\sum_r \hat{\pi}_r \log \hat{\pi}_r; \tag{A.25}$$

4. bounded dispersion descriptor

$$\tanh\left(\log \frac{\mathrm{Var}(C^c) + \varepsilon}{\mathbb{E}[C^c] + \varepsilon}\right). \tag{A.26}$$

Continuous

Define the robust standardized value

$$z_r = \frac{x - \mathrm{median}(x)}{1.4826\,\mathrm{MAD}(x) + \varepsilon}, \tag{A.27}$$

with

$$\mathrm{MAD}(x) = \mathrm{median}\Big(|x - \mathrm{median}(x)|\Big). \tag{A.28}$$

Let

$$z_0 = \frac{z_r - \mathbb{E}[z_r]}{\mathrm{std}(z_r) + \varepsilon}, \tag{A.29}$$

using the population standard deviation over finite training observations, and set

$$\gamma_1 = \mathbb{E}[z_0^3], \qquad \gamma_2 = \mathbb{E}[z_0^4]. \tag{A.30}$$

The four continuous descriptors are

1. bounded skewness $\tanh(\gamma_1/3)$;
2. bounded excess kurtosis / tail-weight descriptor $\tanh((\gamma_2 - 3)/10)$;
3. extreme tail mass $P(|z_r| > 1.5)$;
4. signed tail asymmetry $P(z_r > 1.5) - P(z_r < -1.5)$.

Skewness and kurtosis use z_0; the tail descriptors continue to use z_r. The hyperbolic tangent is used only as a bounded monotonic transform, not as an additional statistical measure.

A.8 Joint activation and rarity (slots 28–31)

Define a type-specific activation indicator

$$\mathrm{act}(X) = \begin{cases} \mathbf{1}[X \geq 0.5], & X \text{ binary}, \\ \mathbf{1}[X > 0], & X \text{ count}, \\ \mathbf{1}[|z_r| > 1.5], & X \text{ continuous}. \end{cases} \tag{A.31}$$

Marginal activation rates are

$$p_+(U) = \frac{1}{n}\sum_i \mathrm{act}(U_i), \qquad p_+(V) = \frac{1}{n}\sum_i \mathrm{act}(V_i), \tag{A.32}$$

and joint activation is

$$p_{++} = \frac{1}{n}\sum_i \mathbf{1}\Big[\mathrm{act}(U_i) = 1 \wedge \mathrm{act}(V_i) = 1\Big]. \tag{A.33}$$

- slot 28: joint activation rate p_{++};
- slot 29: excess joint activation over independence $\Delta = p_{++} - p_+(U)p_+(V)$;
- slot 30: log-lift
$$L = \log \frac{p_{++} + \varepsilon}{p_+(U)p_+(V) + \varepsilon}; \tag{A.34}$$
- slot 31: joint-rarity descriptor
$$n_{++} = \sum_i \mathbf{1}\Big[\mathrm{act}(U_i) = 1 \wedge \mathrm{act}(V_i) = 1\Big], \qquad R_{++} = (n_{++} + 1)^{-1/2}. \tag{A.35}$$

The quantity R_{++} is an engineered rarity descriptor, not a classical rarity estimator.

A.9 Support and estimability (slots 32–33)

For slot 32,

$$S = \frac{n_{\mathrm{complete}}}{n_{\mathrm{train}}}, \tag{A.36}$$

which communicates how much of the training population supports the pair estimate. For slot 33, $I_{\mathrm{sup}} \in \{0, 1\}$. The estimability indicator distinguishes a genuinely small or near-zero dependence estimate from an unavailable estimate that was zero-filled because support was insufficient.

In the frozen implementation a pair is unsupported if

$$n_{\text{complete}} < 50, \tag{A.37}$$

and, for binary–binary pairs, if the minimum of the four level counts

$$|\{U = 0\}|,\ |\{U = 1\}|,\ |\{V = 0\}|,\ |\{V = 1\}| \tag{A.38}$$

among complete cases is smaller than 5. When unsupported, the coefficient matrix is zeroed while enrichment and type metadata may still be filled.

A.10 Variable-type metadata (slots 34–39)

slots 34–36 store

$$\text{type}(U) = \text{one-hot}(\text{binary}, \text{count}, \text{continuous}), \tag{A.39}$$

and slots 37–39 store the analogous one-hot vector for V. These indicators are needed because zero-padding has different meanings for different pair types: for a binary–binary pair most of the 4×4 coefficient matrix is structurally absent, whereas for a continuous–continuous pair a zero coefficient may represent a genuinely small estimated dependence mode.

A.11 Final composition

The complete vector is

$$\begin{aligned} h(U,V) = \Big[&\text{vec}(A_{\text{pad}}) \parallel D_{\text{tot}} \parallel D_{\text{mean}} \parallel R_{\text{low}} \parallel M_{\text{max}} \\ &\parallel m(U) \parallel m(V) \parallel p_{++} \parallel \Delta \parallel L \parallel R_{++} \\ &\parallel S \parallel I_{\text{sup}} \parallel t_U \parallel t_V\Big] \in \mathbb{R}^{40}, \end{aligned} \tag{A.40}$$

where $\parallel$ denotes concatenation and vec is row-major flattening. Dimension check:

$$16 + 4 + 4 + 4 + 4 + 1 + 1 + 3 + 3 = 40, \tag{A.41}$$

where the four after the two marginal blocks are the joint-activity features.

A.12 Connection to the Task A motif

For candidate $A \to G$ with local context node Z, the statistical branch constructs independently

$$h_{AZ} = h(A,Z), \qquad h_{AG} = h(A,G), \qquad h_{ZG} = h(Z,G), \tag{A.42}$$

each in $\mathbb{R}^{40}$. These three vectors are not immediately concatenated into the fusion input. Each role is first passed through its own $40 \to 16 \to 8$ role encoder, and absent motif roles are masked,

$$e_r = m_r\, E_r(h_r), \tag{A.43}$$

yielding

$$g_{\text{stat}} = \Big[e_{AZ} \parallel e_{AG} \parallel e_{ZG}\Big] \in \mathbb{R}^{24}. \tag{A.44}$$

A.13 Source and inspiration note

The coefficient construction is inspired by Hierarchical Correlation Reconstruction: dependence is encoded through expectations of products of orthonormal basis functions [10, 9, 14]. The fixed 40-dimensional representation used here is an application-specific extension of that principle rather than a direct reproduction of a canonical HCR feature vector. In particular, the marginal, joint-activation, support, rarity and type descriptors were introduced to provide a common interface for heterogeneous binary, count and continuous variables and are not attributed to the original HCR formulation.

Appendix B

Synthetic Endpoint Oracle Reference

Table B.1 lists the thirteen clinical endpoints in the synthetic cohort together with prevalence and generator-based oracle AUC on the complete dataset. Task B macro-averaged reporting uses the eight endpoints that exclude `Hospitalization`, `Falls`, `Serotonin_syndrome`, `Rhabdomyolysis`, and `Lactic_acidosis`.

Table B.1: Endpoint prevalence and generator-based oracle AUC in the complete synthetic cohort.

Endpoint	Positive cases	Prevalence (%)	Oracle AUC (%)	Parents
AKI	2,620	13.10	81.20	6
DILI	361	1.81	69.30	7
Delirium	739	3.70	65.73	4
Depression	1,693	8.47	66.27	6
Falls	1,558	7.79	68.09	7
GI bleeding	425	2.13	61.28	5
Hospitalization	4,502	22.51	76.23	14
Hyperkalemia	969	4.85	59.63	2
Hyponatremia	1,574	7.87	62.69	3
QT arrhythmia	319	1.60	61.66	4
Serotonin syndrome	51	0.26	52.16	1
Rhabdomyolysis	69	0.35	58.64	1
Lactic acidosis	111	0.56	52.45	1

List of Figures

1.1 Intended research direction: an iterative loop between Task A (evaluation of graph relations) and Task B (patient-level endpoint prediction). 9

4.1 Number of nodes in each semantic layer of Synthetic Pharmacotherapy Graph v3. 18
4.2 Overview of the synthetic pharmacotherapy DAG with node colours by semantic type. 19
4.3 AKI-related fragment of the Synthetic Pharmacotherapy Graph v3: all audited ancestors of the AKI endpoint (32 nodes / 79 edges), drawn with NetworkX. Node colours follow the v3 ontology (patient context, drug exposure, mechanism, ADR/intermediate state, clinical endpoint). The full graph is larger; this excerpt is for orientation. 20

5.1 Solution architecture for Task A. Patient records yield train-only node features $x_v = [\mu, \sigma, m]$ and pair statistics; candidate edges are split independently at edge level. The structural branch maps G_{train} through HGT to $g_{\text{graph}} \in \mathbb{R}^{64}$; the statistical branch encodes roles $AZ/AG/ZG$ with separate encoders and masks absent roles to $g_{\text{stat}} \in \mathbb{R}^{24}$; fusion yields an edge logit. . 25
5.2 Role-aware motif for $A \to G$: three ordered pair roles, each with a separate encoder and role mask, concatenated to $g_{\text{stat}} \in \mathbb{R}^{24}$. 27
5.3 Task A: final performance across observational scenarios. Panel (a): AP on the training partition (at the validation-selected epoch), on validation, and on the sealed test set. Panel (b): AUROC on validation and sealed test. Bars are means over five training seeds; whiskers are one standard deviation. The hatched scenario is excluded from official macro averages. Official macro test AP is 0.894 and test AUROC is 0.967 (five-scenario mean; hidden confounder excluded). 29

6.1 Illustrative fragment of the Last-FM* collaborative knowledge graph: a user, training interactions, and Freebase-style entity neighbours of catalog items. 32
6.2 Last-FM* architecture. Structural HGT yields $[q_{uX} \parallel \langle z_u, z_X \rangle] \in \mathbb{R}^{257}$. A5 uses H_u, X and N_X; routed Top25 associations feed both H3 and LEG_{K2}, with LEG entering only as residual δ_{LEG}. Late fusion is 265-dimensional ($\parallel$ = concatenation; $\oplus$ = residual addition). The right column summarizes training and the two development evaluation levels; the sealed external benchmark is defined in Section 6.4. 33
6.3 Candidate-conditioned history selection: A5 summarises whole-history context with N_X; Top25 routing by cross-fitted A_{11}/ϕ feeds both H3 and LEG_{K2} from the same selected associations. 34

6.4 Development training dynamics for seed 303 of the full Last-FM* model: training loss and sampled validation NDCG@20 versus epoch. The dashed line marks the selected checkpoint (epoch 43). Seeds 101 and 202 were frozen under the same sampled rule at epochs 36 and 34. 36
6.5 Last-FM* development ablation ladder. Panel (a): sampled validation NDCG@20 used for checkpoint selection. Panel (b): full-catalog development NDCG@20 of the same checkpoints. Error bars show the seed standard deviation over three training seeds. The panels use different y-axis scales because they evaluate different candidate universes under the same NDCG@20 formula. This figure is not an external baseline comparison. . . 39
6.6 Sealed external Last-FM* benchmark. NDCG@20 and Recall@20 for classical recommendation baselines and the proposed model under the same full-catalog holdout evaluation protocol. The proposed-model bars report the mean across the three prespecified training seeds, with error bars indicating one standard deviation. 41

7.1 Exemplary per-patient causal DAG for one synthetic patient (id 101). The full SPG v3 is shared across the cohort; this figure keeps only the AKI-relevant fragment so that the left-to-right causal layers remain readable. Filled nodes are active for this patient; outlined nodes are inactive mediators on the same paths. 44
7.2 Order of operations in the Task B encoder. 47
7.3 Task B architecture: heterogeneous GNN path with a shared endpoint head, HCR-inspired evidence tensors with pair encoders, and optional concatenation or logit-level fusion. 48
7.4 Statistical-evidence gate matrix: influence of parent node type per endpoint. 57

8.1 Hetionet v1.0 knowledge graph (adapted from [18]). 60
8.2 Hetionet layer mapping after reconstruction onto the synthetic schema. . . 62

List of Tables

1 Symbols and abbreviations by category. 6

4.1 Node types in Synthetic Pharmacotherapy Graph v3. 18
4.2 Observational scenarios derived from the same pharmacotherapy graph. . . 21

5.1 Grouped composition of the 40-dimensional HCR-inspired pair representation. 26
5.2 Task A: model and training configuration. 28

6.1 Last-FM dump versus leakage-corrected Last-FM*. 31
6.2 Working splits used in the Last-FM* experiment. Official test remains sealed. 31
6.3 Last-FM*: model and training configuration. 38
6.4 Last-FM*: development ablation results. Sampled NDCG@20 is the checkpoint-selection metric; full-catalog NDCG@20 and Recall@20 evaluate the same checkpoints on the eligible validation catalog. These are development validation results and are used to isolate the contribution of explicit candidate-specific context. They are not the final external benchmark. . . 39
6.5 Full-model full-catalog development metrics by seed (model train → validation). Sampled NDCG@20 selected the epoch; full-catalog metrics are evaluation only. 40
6.6 Full-catalog development NDCG@20 by seed for the ablation layers without LEG_{K2}. 40
6.7 Last-FM*: sealed external benchmark results for classical recommenders fitted on TRAIN_EXTERNAL and evaluated once on the corrected Last-FM* test with the pinned holdout evaluator. Primary comparison metrics are NDCG@20 and Recall@20. 41
6.8 Last-FM*: sealed external benchmark results for the proposed full model under the frozen epoch schedule. Mean ± SD is over the three prespecified training seeds. 41

7.1 Components of the per-patient heterogeneous graph representation. 45
7.2 Layout of the Task B `edge_attr` vector. 46
7.3 Use of `edge_attr` by convolution backbone in Task B. 46
7.4 Fusion modes for the statistical evidence branch in Task B. 50
7.5 Task B: shared model and training configuration. Values were fixed across the experiments reported in this chapter. 52
7.6 Task B: architecture-specific configuration. 52
7.7 Task B: message-passing architecture comparison without residual connections. 53
7.8 Task B: message-passing architecture comparison with residual connections. 54
7.9 Notation used for the statistical-evidence variants in the result tables. . . . 55

7.10 Task B: statistical-evidence variants without residual connections (TransformerConv backbone). 55
7.11 Task B: statistical-evidence variants with residual connections (TransformerConv backbone). 56
7.12 Task B: per-endpoint results for the selected model (*typed+triple* at $L = 4$ without residual connections). 58

8.1 Mapping of Hetionet entities onto the synthetic layer schema. 61
8.2 Hetionet proxy: model and training configuration (differences from the Task B synthetic protocol). 63
8.3 Hetionet proxy: endpoint-prediction results under the reconstructed layer schema. 64

A.1 Block layout of the 40-dimensional HCR-inspired pair representation. . . . 74

B.1 Endpoint prevalence and generator-based oracle AUC in the complete synthetic cohort. 81

Bibliography

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Jesús Barrasa, Maya Natarajan, and Jim Webber. *Building Knowledge Graphs: A Practitioner's Guide*. O'Reilly Media, 2023.

[3] G. Bazhenov, O. Platonov, and L. Prokhorenkova. TabGraphs: A benchmark and strong baselines for learning on graphs with tabular node features. *arXiv preprint arXiv:2409.14500*, 2024. Later extended as GraphLand (NeurIPS 2025 Datasets and Benchmarks).

[4] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations*, 2022.

[5] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pages 7–10, 2016.

[6] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pages 7–10, 2016.

[7] Hejie Cui, Jiaying Lu, Ran Xu, Shiyu Wang, Wenjing Ma, Shaojun Yu, Yue Yu, Xuan Kan, Chen Ling, Liang Zhao, Zhaohui S. Qin, Joyce C. Ho, Tianfan Fu, Jing Ma, Mengdi Huai, and Carl Yang. A review on knowledge graphs for healthcare: Resources, applications, and promises. *Journal of Biomedical Informatics*, 169:104861, 2025.

[8] Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 233–240, 2006.

[9] J. Duda. Exploiting statistical dependencies of time series with hierarchical correlation reconstruction. *arXiv preprint arXiv:1807.04119*, 2018.

[10] J. Duda. Hierarchical correlation reconstruction with missing data, for example for biology-inspired neuron. *arXiv preprint arXiv:1804.06218*, 2018.

[11] J. Duda. Hierarchical correlation reconstruction: between statistics and machine learning, 2023. Wolfram Community Staff Pick.

[12] J. Duda, D. Bracha, and M. Przybysz. Linear cost mutual information estimation and independence test of similar performance as HSIC. *arXiv preprint arXiv:2508.18338*, 2025.

[13] J. Duda and S. Podlewska. Low cost prediction of probability distributions of molecular properties for early virtual screening. *arXiv preprint arXiv:2207.11174*, 2022. Journal version: Molecular Diversity 28 (2024) 437–448.

[14] J. Duda and A. Szulc. Credibility evaluation of income data with hierarchical correlation reconstruction. *CoRR*, abs/1812.08040, 2018. Listed in DBLP as CoRR abs/1812.08040; no verified conference DOI used here.

[15] Z. Gao, F. Gama, and A. Ribeiro. Wide and deep graph neural network with distributed online learning. *IEEE Transactions on Signal Processing*, 70, 2022. arXiv:2107.09203 (2021).

[16] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[17] Daniel S. Himmelstein, Antoine Lizee, Christine Hessler, Leo Brueggeman, Sabrina L. Chen, Dexter Hadley, Ari Green, Pouya Khankhanian, and Sergio E. Baranzini. Hetionet: an integrative network of disease. https://github.com/hetio/hetionet, 2017. GitHub repository, version 1.0.

[18] Daniel S. Himmelstein, Antoine Lizee, Christine Hessler, Leo Brueggeman, Sabrina L. Chen, Dexter Hadley, Ari Green, Pouya Khankhanian, and Sergio E. Baranzini. Systematic integration of biomedical knowledge prioritizes drugs for repurposing. *eLife*, 6:e26726, 2017.

[19] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labaj, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. *Knowledge Graphs*. Synthesis Lectures on Data, Semantics, and Knowledge. Morgan & Claypool Publishers, 2021.

[20] Jeroen Hoogland, Joanna IntHout, Michail Belias, Maroeska M. Rovers, Richard D. Riley, Frank E. Harrell, Karel G. M. Moons, Thomas P. A. Debray, and Johannes B. Reitsma. A tutorial on individualized treatment effect prediction from randomized trials with a binary endpoint. *Statistics in Medicine*, 40(26):5961–5981, 2021.

[21] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference 2020*, pages 2704–2710, 2020.

[22] Pengcheng Jiang, Cao (Danica) Xiao, Adam Cross, and Jimeng Sun. GraphCare: Enhancing healthcare predictions with personalized knowledge graphs. In *International Conference on Learning Representations*, 2024.

[23] Ruth Johnson, Michelle M. Li, Ayush Noori, Owen Queen, and Marinka Zitnik. Graph artificial intelligence in medicine. *Annual Review of Biomedical Data Science*, 7, 2024.

[24] So Yeon Kim. GNN-surv: Discrete-time survival prediction using graph neural networks. *Bioengineering*, 10(9):1046, 2023.

[25] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2015. ICLR 2015.

[26] Finlay MacLean. Knowledge graphs and their applications in drug discovery. *Expert Opinion on Drug Discovery*, 2021.

[27] Munib Mesinovic, Max Buhlan, and Tingting Zhu. Causal graph neural networks for healthcare. *Nature Biomedical Engineering*, 10:1536–1556, 2026. Also available as arXiv:2511.02531 (2025).

[28] F. Pedregosa et al. Scikit-learn: Machine learning in Python. `https://scikit-learn.org/`. Documentation of `average_precision_score`.

[29] T. Konstantin Rusch, Michael M. Bronstein, and Siddhartha Mishra. A survey on oversmoothing in graph neural networks. *arXiv preprint arXiv:2303.10993*, 2023.

[30] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web (ESWC 2018)*, pages 593–607, 2018.

[31] Faisal Shehzad, Maurizio Ferrari Dacrema, and Dietmar Jannach. A worrying reproducibility study of intent-aware recommendation models. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3155–3164, 2025.

[32] A. Szulc. Polish inequality statistics reconsidered: are the poor really that poor? *Statistics in Transition new series*, 23(3):79–94, 2022.

[33] A. Tharzeen, A. Vafaei Sadr, N. Radfar, W. Hwang, V. Abedi, and R. Zand. A heterogeneous graph neural network framework for multi-horizon stroke mortality prediction. *medRxiv*, 2026. StrokeTHG: temporal heterogeneous graph vs tabular baselines.

[34] Sumyyah Toonsi, Paul N. Schofield, and Robert Hoehndorf. Causal knowledge graph analysis identifies adverse drug effects. *arXiv preprint arXiv:2505.06949*, 2025.

[35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.

[36] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. KGAT: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 950–958, 2019.

[37] Filip Wójcik. *Grafowe sieci neuronowe. Teoria i praktyka*. Helion, Gliwice, 2026. 224 pp.